

基于热湿传递与收缩移动边界的药材烘干模型

摘 要

药材烘干过程中，外部环境变化、内部水分扩散与几何收缩共同影响干燥进程。针对圆柱形药材的温度、含水率分布及全域达标时间问题，本文依据题设物性关系建立热湿传递模型，采用守恒有限体积法进行数值求解，并分别检验端面效应、温度处理方式及移动边界的影响。

对于问题一，利用圆柱轴对称性建立二维显热传导与水分扩散方程，以对流边界描述药材与烘房的交换过程，同时建立一维径向模型作为对照。计算得到，1800 s 时二维模型的平均干基含水率为 2.2750 kg/kg，一维结果为 2.2936 kg/kg。二维模型中端面提供了额外的传质通道，因而平均含水率较低；中截面轴心温度为 33.5763℃，低于表面温度 36.7859℃。

对于问题二，将扩散系数表示为含水率与温度的函数，在内部温度等于烘房温度的假设下求解 3 h 内的含水率分布，并与固定扩散系数模型及显热温度场模型比较。三者在 3 h 的最大含水率分别为 1.7278、2.1600 和 1.7645 kg/kg。结果表明，物性更新与升温滞后均会改变水分下降轨迹；由于对照模型采用的网格和时间步不同，该比较用于揭示变化趋势，不作为纯粹的机制效应估计。

对于问题三，以全域最大含水率低于 0.15 kg/kg 为达标要求，设置 10^{-6} 的数值安全余量，并对阈值前后时间步进行插值。以 60 s 为计算时间步时，达标时刻向上取整为 206820 s，即 57.45 h。扩散参数的扰动对长时结果影响明显；时间步细化至 15 s 后，插值时刻较基准提前约 143 s，说明分钟级结果仍存在时间离散误差。

对于问题四，利用附件半径数据构造 PCHIP 插值函数，将收缩引起的干固体密度变化纳入水分守恒方程。参考域法与移动有限体积法均得到报告时刻 183840 s，约为 51.07 h，两种实现的水分平衡误差均约为 2.01×10^{-9} 。此结果同时包含问题四物性关系与半径变化的影响，不能将其相对问题三的缩短全部归因于收缩。模型未计入汽化潜热，所得时间适用于题设参数和环境延拓条件下的数值预测。

关键字：药材烘干 轴对称有限体积 状态相关扩散 移动边界 全域阈值

1 问题重述

1.1 问题背景

烘干的目的在于降低药材含水率，但药材表面先与环境接触，内部水分必须经过扩散才能排出，因此表面变干并不意味着整个药材均已达到要求。随着烘房升温，传热和水分扩散的速率发生变化；在较长时间的烘干过程中，材料收缩又会改变扩散距离和传质面积。如何在这些因素共同作用下描述内部场量，并确定所有位置均满足要求的时刻，是本题需要解决的核心问题。

研究对象近似为长度 $L = 25\text{ cm}$ 、初始半径 $R_0 = 2\text{ cm}$ 的圆柱体。附件 1 提供烘房温度和水分边界数据，附件 2 提供半径变化数据，附件 3 给出结果文件格式。初始温度为 28°C ，初始干基含水率为 2.55 kg/kg 。

1.2 具体问题

依据四个问题给定的物性与几何条件，本文分别完成以下任务：

- (1) 问题一：求解前 1800 s 的温度和含水率分布，按规定时刻与径向位置输出，并考察忽略端面传递所造成的差异；
- (2) 问题二：考虑状态相关物性，求解前 3 h 的含水率变化，比较物性更新及温度处理方式对结果的影响；
- (3) 问题三：在固定几何条件下继续考察长期干燥过程，确定全域含水率达到阈值的时间；
- (4) 问题四：使用半径观测描述收缩，在新的物性条件下求解移动域干燥过程，给出达标时间和相应结果表。

2 问题分析

2.1 问题一的分析

药材半径远小于长度，一维径向模型能够以较低代价描述中部的主要变化，但两端同样与烘房交换热量和水分，仅凭几何长细比不能确定端面影响是否足够小。因此，先建立一维径向模型，再保留轴向通量形成二维轴对称模型，在相同初值和边界下比较平均含水率。同时分别计算温度与水分场，以区分升温过程和失水过程的空间特征。

2.2 问题二的分析

扩散系数随温度、含水率变化，方程成为非线性扩散方程。求解的关键是将物性更新与隐式离散结合，避免直接使用初始扩散系数代表整个过程。本文以规定内部温度的

状态相关模型为主模型，同时保留固定扩散系数对照与显热温度场对照。三种结果的比较用于说明简化假设的影响，而不是用模型间差异替代实验误差。

2.3 问题三的分析

平均含水率较低时，内部仍可能存在尚未达标的区域，因此需要在每个时间步检查全域最大值，而不能只检查表面或平均值。预计达标时间超过附件 1 的观测区间，必须先明确 4 h 后的环境延拓方式，再对首次越过阈值的两个时间步作局部插值。长期积分还会积累离散误差，因而应同时报告网格、时间步和参数扰动的检查结果。

2.4 问题四的分析

半径缩小时，固定物理位置可能离开药材区域，直接在原固定域方程中替换半径会遗漏控制体变化。本文从干固体连续性出发，以单位干固体质量所含水分为状态量，在随材料收缩的控制体上建立水分平衡；再使用固定参考域和移动控制体两种实现相互核对。问题四还改变了物性经验式，因此需将其视为新的材料条件，不能只解释为问题三的几何修正。

3 模型假设

- (1) 将药材视为连续、均质、轴对称的圆柱体；初始温度和含水率均匀，忽略周向差异。该假设允许在径向与半轴长构成的二维区域内求解。
- (2) 侧面与端面分别满足同一组对流传热、传质关系，传递系数不随位置变化；圆柱轴线与中截面满足对称边界条件。
- (3) 采用题设物性经验式，不额外拟合内部参数。问题一求解显热温度场；问题二、三的主模型及问题四规定内部温度等于烘房温度。显热温度场模型用于检查这一简化的影响。
- (4) 所有模型均不加入汽化潜热、吸附热和化学反应热。故温度场模型也不代表完整的相变热湿耦合模型。
- (5) 附件 1 观测结束后，将末一小时数据的算术平均值作为恒定环境；通过改变平均窗口检查该假设的影响，不假定延拓区间存在实测数据。
- (6) 问题四仅考虑径向仿射收缩，轴向长度保持不变，半径由附件 2 的保形插值确定；干固体总质量不变，水分随边界交换而减少。

4 符号说明与数据准备

4.1 数据预处理

求解器统一采用 m、s、K，结果展示时转换为 cm、h 和 °C。附件 1 包含 241 个时刻，覆盖 0–4 h；附件 2 包含 145 个半径观测值，覆盖 0–72 h。预处理时核对时间顺序、缺失值与单位，并保留原始观测，不用平滑曲线替换输入数据。

设相邻环境观测为 (t_i, b_i) 和 (t_{i+1}, b_{i+1}) ，其中 b 分别代表温度和水分边界。在观测区间内采用

$$b(t) = b_i + \frac{t - t_i}{t_{i+1} - t_i}(b_{i+1} - b_i), \quad t_i \leq t \leq t_{i+1}. \quad (1)$$

分段线性插值不引入额外极值，且能与逐时变化的边界条件直接衔接。对于 $t > 4$ h，设末一小时窗口内的观测索引集合为 I ，取

$$b(t) = \bar{b}_I = \frac{1}{|I|} \sum_{i \in I} b_i. \quad (2)$$

式中平均值包含窗口两端的观测。该处理属于边界假设，不是对未来环境的预测。半径序列采用分段三次 Hermite 保形插值 (PCHIP)，以限制普通高阶插值可能引起的局部过冲；超过半径数据末时刻时保持末值。

本文将题面干基含水率记为 C ，温度记为 T ，环境温度和水分边界记为 $T_\infty(t)$ 与 $C_\infty(t)$ 。传热系数和传质系数分别固定为 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 与 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。需要特别区分： C 、相对湿度、水蒸气浓度和分压不是同一物理量，本文只使用题设允许的 $C_s - C_\infty$ 作为传质势差。主要符号汇总见表 1。

表 1 主要符号与单位

符号	含义	单位
r, z	轴对称径向和轴向坐标	m
$R_0, R(t)$	初始半径与时变半径	m
L	圆柱长度	m
C, T	干基含水率与绝对温度	kg/kg, K
ρ, c_p, k, D	有效密度、比热、导热系数、扩散系数	SI 制
h, h_m	对流传热、传质系数	$\text{W}/(\text{m}^2 \cdot \text{K})$, m/s
t_*	连续夹逼得到的首次安全时刻	s

5 模型准备与统一控制方程

5.1 二维轴对称守恒骨架

在半圆柱域

$$\Omega(t) = \{(r, z) : 0 \leq r \leq R(t), 0 \leq z \leq L/2\},$$

上建立控制方程。坐标 r 自轴线指向侧面， z 自圆柱中截面指向端面。对于问题一至三的固定几何域，由局部水分平衡与扩散通量关系得到

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right) + \frac{\partial}{\partial z} \left(D(C, T) \frac{\partial C}{\partial z} \right). \quad (3)$$

侧面和端面使用 Robin 边界

$$-D \nabla C \cdot \mathbf{n} = h_m(C_s - C_\infty(t)), \quad (4)$$

轴线 $r = 0$ 和中截面 $z = 0$ 为零法向通量。Q1 的 M1 是式 (3) 在轴向梯度为零时的严格降阶，M2/M3/M4 保留径向和轴向传输。

温度的处理随问题及对照模型而不同。问题一的一维、二维模型均求解显热温度场；问题二、三中的 M2、M3 以及问题四主模型在每个时间步令

$$T(r, z, t) = T_\infty(t), \quad (5)$$

即忽略内部升温滞后和温度梯度。作为对照，M4 求解显热温度场：

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right) + \frac{\partial}{\partial z} \left(k(C) \frac{\partial T}{\partial z} \right), \quad (6)$$

并满足 $-k \nabla T \cdot \mathbf{n} = h(T_s - T_\infty)$ 。式 (6) 中没有 $L_v \partial \omega / \partial t$ 或其他潜热源项；因此 M4 也不是潜热耦合模型。

5.2 物性关系与四个模型

Q1 使用题面常量和 $D(C) = 7 \times 10^{-9} \exp(-0.89/C)$ 。Q2/Q3 的状态相关物性为

$$\rho = 650 + 128C, \quad c_p = 1450 + \frac{2736C}{C+1}, \quad k = 0.21 + \frac{0.38C}{C+1}, \quad (7)$$

$$D = 2.4 \times 10^{-3} \exp(-0.45/C) \exp(-3850/T), \quad (8)$$

其中温度必须使用 K。Q4 使用

$$\rho = 760 + 90C, \quad c_p = 1850 + \frac{2150C}{C+1}, \quad k = 0.12 + \frac{0.20C}{C+1}, \quad (9)$$

$$D = 4.2 \times 10^{-4} \exp(-0.30/C) \exp(-3850/T). \quad (10)$$

为便于比较，将一维径向模型记为 M1，二维基线记为 M2，二维状态相关模型记为 M3，显式求解显热温度场的对照记为 M4。这里的编号只表示比较层次，并不意味着跨问题使用完全相同的参数：问题一的 M2 使用常量热物性，但扩散系数仍随 C 变化；问题二、三的 M2 则将扩散系数固定在初始 $C = 2.55$ 、 $T = 301.15$ K 的取值。M3 按当前

含水率和规定温度更新扩散系数，M4 则使用计算得到的温度更新物性。M4 同时用于问题二与问题三的补充对照。

已有干燥研究指出，材料收缩会改变内部传递尺度，并与含水率演变相联系 [1, 2]；热湿耦合研究则强调内部场与表面交换条件的共同作用 [3, 4]。本文据此设置模型比较与守恒检查，但具体物性数值均来自题设，不移植其他材料的经验参数。

5.3 Q4 移动域与事件判据

Q4 保持 L 不变，令 $R(t)$ 由附件 2 的 PCHIP 给出。仿射径向收缩的材料速度写为 $\mathbf{v}_m = (r\dot{R}/R, 0)$ 。以干固体密度 ρ_d 和水质量密度 $q_w = \rho_d C$ 为守恒量时，干固体连续性与水分守恒分别为

$$\frac{\partial \rho_d}{\partial t} + \nabla \cdot (\rho_d \mathbf{v}_m) = 0, \quad (11)$$

$$\frac{\partial q_w}{\partial t} + \nabla \cdot (q_w \mathbf{v}_m) = \nabla \cdot (\rho_d D \nabla C). \quad (12)$$

在均匀径向仿射收缩下，有 $\rho_d(t) = \rho_{d0}[R_0/R(t)]^2$ ，从而每个材料控制体内的干固体质量 $\rho_d V$ 保持不变。令 $\xi = r/R(t)$ ，可以将计算区域映射到固定参考域 $0 \leq \xi \leq 1$ 。参考坐标随材料运动时，材料对流与网格运动项相抵消，但几何尺度、体积 Jacobian 及通量系数仍必须保留。移动有限体积法直接更新物理控制体几何；参考域法则以不变的干固体质量作为离散权重。两种写法应描述同一库存与通量平衡，而非仅在结果曲线上接近。收缩干燥的相关数值研究为几何随时间更新提供了参考 [5]。

Q3/Q4 的统一事件判据为

$$g(t) = \max_{(r,z) \in \Omega(t)} C(r, z, t) - 0.15, \quad (13)$$

计算时采用 $C_{\text{safe}} = 0.15 - 10^{-6}$ 的安全阈值，先保存首次越过该值的两个实际时间步，再作局部线性插值，最后向上取整到 60 s 报告。离散最大值由全部活动单元及轴心重构值共同检查；它是全域判据的数值近似，仍需通过网格细化评估误差，不能由平均值或表面值替代。

6 数值求解与数据输出

6.1 空间离散

有限体积法直接从控制体内的库存变化与界面通量出发，便于在固定域和收缩域之间保持一致的守恒形式。将半圆柱沿径向与轴向划分为环柱单元，设第 (i, j) 个单元的边界为 $r_{i\pm 1/2}$ 、 $z_{j\pm 1/2}$ ，则其体积为

$$V_{ij} = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)(z_{j+1/2} - z_{j-1/2}). \quad (14)$$

径向面的面积为 $2\pi r_{i\pm 1/2}\Delta z_j$ ，轴向面的面积为 $\pi(r_{i+1/2}^2 - r_{i-1/2}^2)$ 。在 $r = 0$ 处径向面积为零，因此不必直接计算含 $1/r$ 的差分式，即可满足轴线零通量条件。计算网格在径向两端及轴向端面附近加密，以分辨边界附近较大的梯度。

设相邻单元的扩散系数为 D_P, D_N ，采用调和平均系数 $D_f = 2D_P D_N / (D_P + D_N)$ ，界面传递系数记为 $G_f = A_f D_f / d_{PN}$ 。其中 A_f 为界面面积， d_{PN} 为相邻单元中心距离。对外边界，将半单元扩散阻力与对流传质阻力串联，得到

$$G_b = \frac{A_b}{\delta_b / D_P + 1 / h_m}, \quad F_b = G_b (C_P - C_\infty). \quad (15)$$

式中 δ_b 为单元中心到边界的距离。热传递采用相同构造，只需将 D, h_m 分别换成 k, h ，并在库存项中使用 ρc_p 。

6.2 时间推进与非线性迭代

采用后向欧拉格式，对固定域内部单元有

$$\frac{V_P}{\Delta t} (C_P^{n+1} - C_P^n) = \sum_N G_{PN}^{n+1} (C_N^{n+1} - C_P^{n+1}) + \sum_b G_b^{n+1} (C_\infty^{n+1} - C_P^{n+1}). \quad (16)$$

式中第一项求和遍历内部相邻单元，第二项只在接触外边界的单元出现。为处理扩散系数的非线性，在每个时间步内进行 Picard 迭代：先由上一轮场量计算物性，再组装线性方程，更新场量，直至最大绝对更新量小于 10^{-7} 。线性子问题使用对角预条件共轭梯度法，相对残差容差为 5×10^{-11} ，最多迭代 500 次；Picard 迭代上限为 24 次。

问题四将式 (16) 的库存权重换为不变的 $\rho_d V$ ，并使用当前几何及干固体密度计算传递系数。每步记录边界流出量与内部水分库存变化，两者应相互抵消。该检查能够发现遗漏收缩体积因子的实现错误，而仅比较含水率曲线不能保证守恒。

6.3 输出位置与时间精度

题目要求的径向输出取圆柱中截面 $z = 0$ ，并非轴向平均。程序利用靠近中截面的两个轴向单元按 z^2 外推至对称面，在轴线附近按 r^2 重构，再插值到规定的物理半径。表面状态由 Robin 条件重构；全域平均值则单独按控制体体积加权计算，两种指标不混用。

问题一采用 $\Delta t = 0.5 \text{ s}$ 积分，按 1 s 间隔写出 1–1800 s 的温度与含水率；问题二主模型采用 $\Delta t = 5 \text{ s}$ ，再将已算状态线性插值到 1–10800 s 的逐秒表格。因此，逐秒输出不等于逐秒独立求解。问题三、四先在完整活动网格上执行阈值判断，再导出径向列。问题四的固定径向位置一旦位于 $R(t)$ 外，结果单元格留空，不以零含水率代替域外状态。

7 问题一：温度与水分分布的求解

7.1 模型建立与求解

问题一取 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg} \cdot \text{K)}$ 、 $k = 0.36 \text{ W/(m} \cdot \text{K)}$ ，扩散系数采用本问给定的 $D(C)$ 。联立式 (3) 与式 (6)，在均匀初值及对流边界下推进至 1800 s 。忽略轴向梯度即可得到 M1，保留轴向通量得到 M2。基准计算中 M1 取 122 个径向单元，M2 取 93×54 个径向、轴向单元，两者时间步均为 0.5 s 。

7.2 计算结果与分析

图 1 给出中截面不同时刻的含水率径向分布。外层首先通过边界向环境失水，而内层水分需经过一定距离才能传递至表面，因此中心区域下降较慢、表面下降较快。随着时间增加，水分梯度逐渐向内部扩展。这一分布说明，将表面含水率作为整根药材的状态指标会低估内部剩余水分。

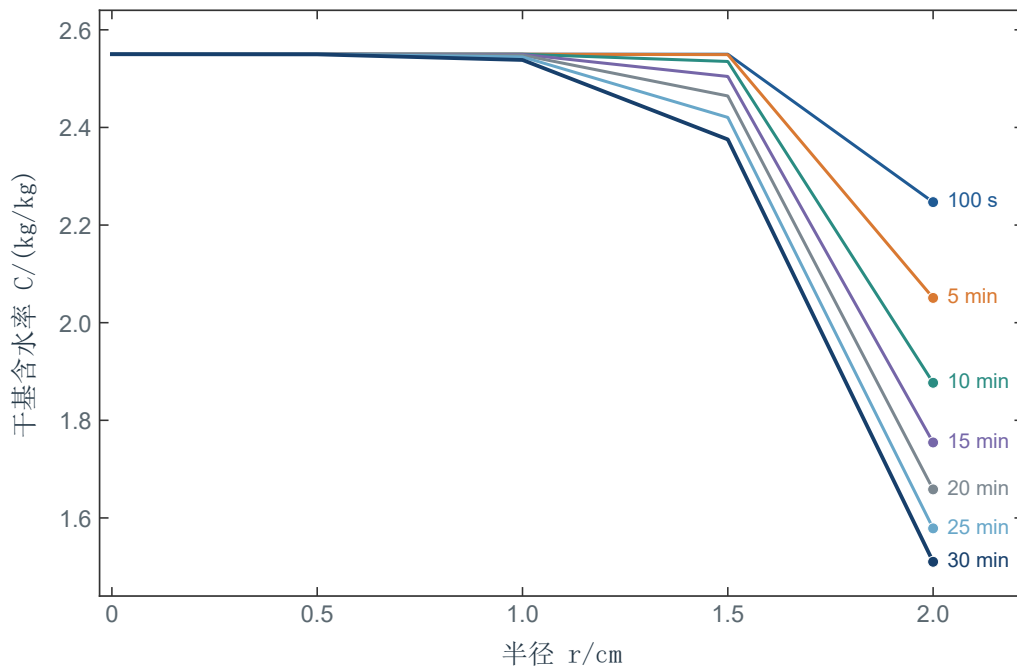


图 1：不同时刻的径向含水率分布

图 1 Q1 二维模型不同时间的含水率径向分布

表 2 列出中截面轴心与表面的代表性结果。 1800 s 时轴心温度为 33.5763°C ，表面为 36.7859°C ，说明内部升温具有滞后性；与此同时，轴心含水率仍接近初值，表面已降至 1.5101 kg/kg 。温度和水分变化的尺度不同，不能从升温幅度直接推断失水程度。

为衡量端面对整体失水的影响，进一步比较体积加权平均含水率，如图 2 所示。 1800 s 时 M1 为 2.2936 kg/kg ，M2 为 2.2750 kg/kg ，相差 0.0185 kg/kg 。二维模型允许端面流

表 2 问题一中截面轴心与表面的温度和含水率

时间/s	轴心温度/°C	表面温度/°C	轴心含水率	表面含水率
600	28.4543	30.1658	2.5500	1.8769
1200	30.5439	33.4282	2.5500	1.6585
1800	33.5763	36.7859	2.5500	1.5101

注：含水率单位为 kg/kg，四位小数相同不表示对应内部值严格相等。

出水分，因而得到更低的平均值；该结果支持保留端面传递，但尚不能说明二维预测与真实药材的实验误差有多大。

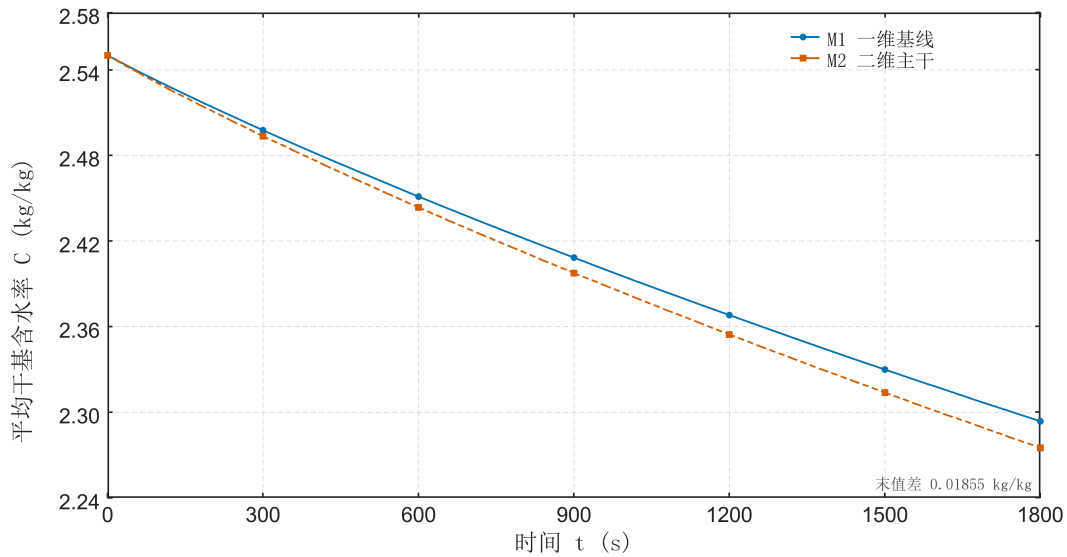


图2：一维与二维模型平均含水率随时间变化

图 2 Q1 一维与二维模型平均含水率随时间变化

7.3 网格检验

固定时间步后分别加密两类模型的径向网格，M1 基准至细网格的平均含水率变化为 $8.759 \times 10^{-6} \text{ kg/kg}$ ，M2 为 $1.505 \times 10^{-5} \text{ kg/kg}$ ，均明显小于上述一维、二维差异。因此，当前端面效应的主要趋势并非仅由该组径向网格误差造成。完整网格结果见附录图 11；这一检验仍不替代独立的材料实验验证。

8 问题二：状态相关扩散模型的求解

8.1 状态相关主模型

问题二使用式 (8)，在每个非线性迭代中按当前含水率和规定温度更新 D 。由于 M3 不求解能量方程，热物性 ρ, c_p, k 不决定其温度场；这些系数在 M4 的显热方程中才发挥

作用。从均匀初始状态积分至 3 h，得到图 3 中的径向分布。

与问题一短时过程相比，较长时间内中心含水率也逐渐下降，但中心与表面之间始终存在差异。3 h 时中截面轴心为 1.7278 kg/kg， $r = 1\text{ cm}$ 处为 1.5420 kg/kg，表面为 0.9956 kg/kg。这意味着即使表面下降明显，内部仍可能是限制整根药材达标的区域。表 3 展示该空间差异随时间的变化。

表 3 问题二主模型在不同时间、半径处的含水率 (kg/kg)

时间/h	$r = 0\text{ cm}$	$r = 1\text{ cm}$	$r = 2\text{ cm}$
0.5	2.5494	2.5073	1.7208
1.0	2.4891	2.3040	1.5306
1.5	2.3109	2.0828	1.3747
2.0	2.1020	1.8816	1.2355
2.5	1.9054	1.7022	1.1086
3.0	1.7278	1.5420	0.9956

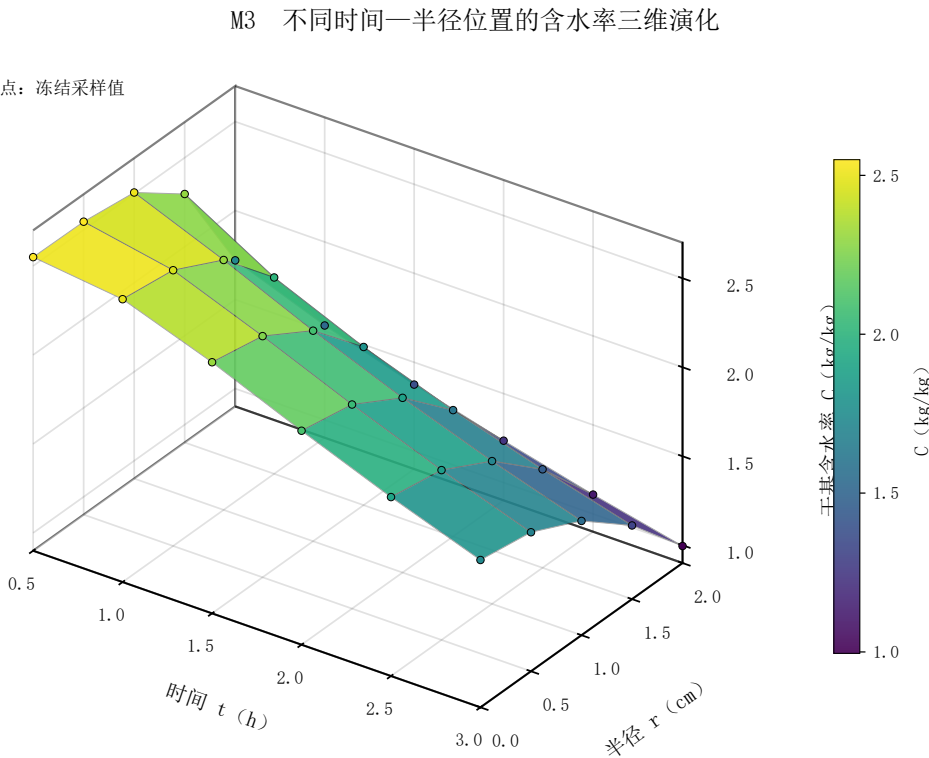


图 3 M3 在不同时间和半径位置的含水率剖面

8.2 三模型的时间轨迹

图 4 将 M2、M3、M4 分别绘制在相同坐标范围内，各取 0–3 h 内每 10 min 一个时刻，共 19 组结果；表 4 列出整点数值。M2 的扩散系数固定在初始温度与初始含水率的

取值，而 M3 同时引入温度和含水率依赖性。因此，两者差别不是单独的“含水率依赖效应”。M4 的最大含水率高于 M3，与内部升温滞后降低早期扩散速度的机制相符。

需要指出，现有对照计算中 M2、M4 使用 18×36 网格和 10 s 时间步，M3 使用 140×36 网格和 5 s 时间步。虽然各模型在图中共享坐标尺度，但并未共享相同的离散配置。因而图表可以说明各方案已计算出的轨迹差异，不能将数值差额全部当成单一物理机制的净贡献，更不能据此排列实验准确率。

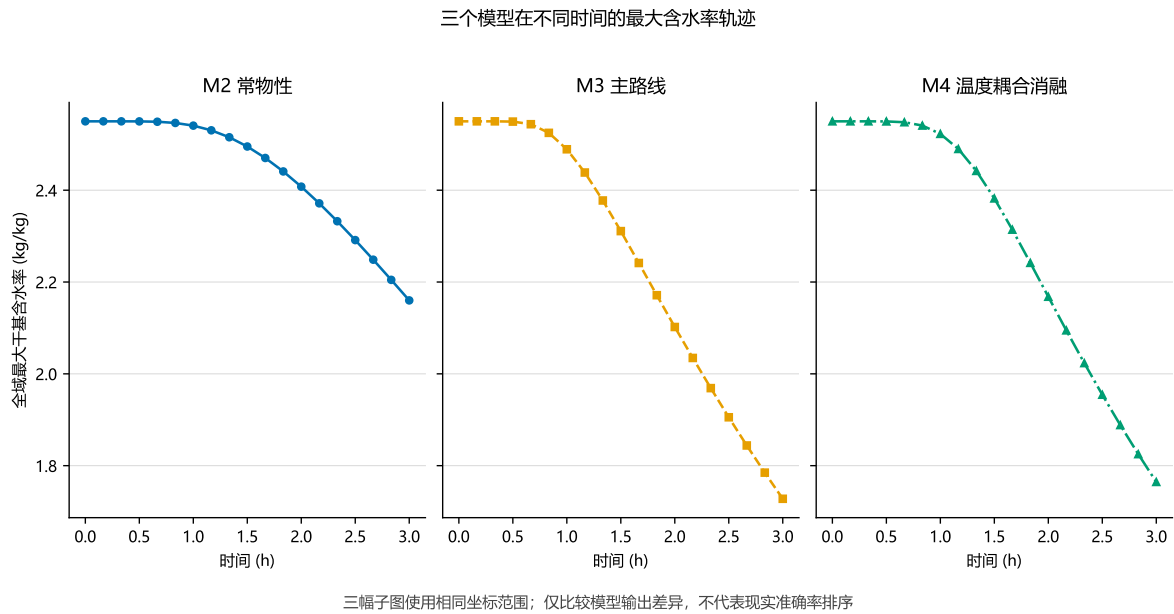


图 4 M2、M3、M4 在不同时间的最大含水率轨迹

表 4 三个模型在代表性时刻的全域最大含水率

时间/h	M2 常物性	M3 主路线	M4 温度耦合消融	指标
0	2.550000	2.550000	2.550000	$C_{\max}$
1	2.540628	2.489138	2.522663	$C_{\max}$
2	2.407727	2.102044	2.167906	$C_{\max}$
3	2.159969	1.727846	1.764520	$C_{\max}$

8.3 空间网格检查

为检查主模型的径向离散影响，在 $\Delta t = 5\text{ s}$ 、36 个轴向单元下，分别取 $n_r = 42, 62, 93, 140, 175, 210$ 。图 5 左侧为 3 h 终值，右侧为相邻网格差。93 与 140 个径向单元之间的最大含水率变化为 $4.0011 \times 10^{-5} \text{ kg/kg}$ ；进一步加密后，175 与 210 个单元之间降为 $6.264 \times 10^{-6} \text{ kg/kg}$ 。两组比较均小于 5×10^{-5} 的检查目标，后者显示径向细化差异继续减小。但该结论限于固定轴向网格与时间步，不能理解为所有离散误差均已消失。

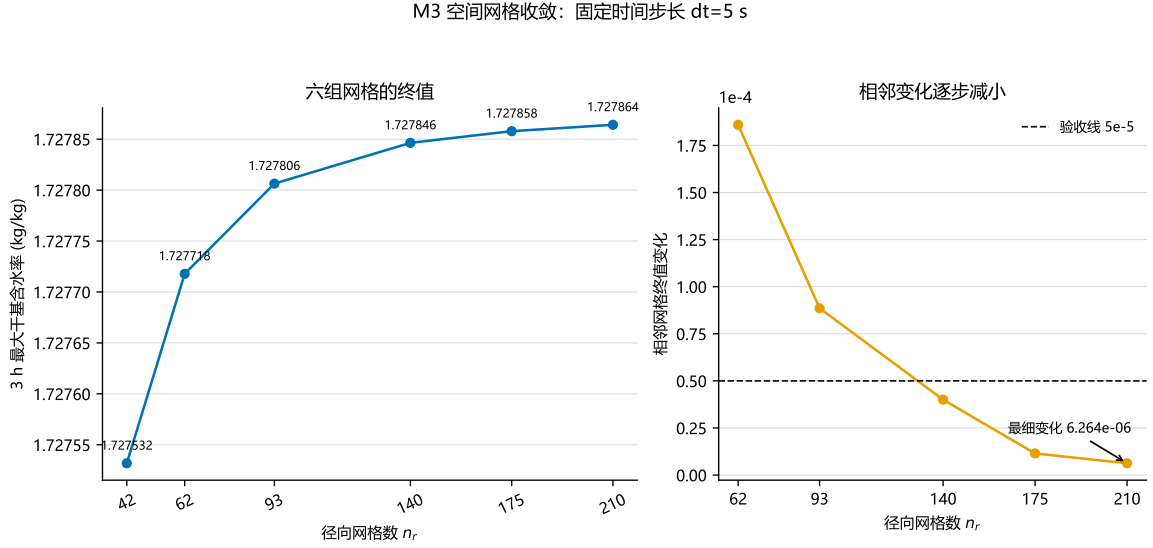


图 5 M3 固定时间步下的六组径向网格收敛

9 问题三：固定域全域达标时间

9.1 阈值事件与局部夹逼

问题三沿用问题二主模型的控制方程、初值和物性关系，但重新从 $t = 0$ 积分至首次达标，而非读取问题二的终态文件作为数值起点。观测区间外的环境由式 (2) 延拓。设首次越过安全阈值的两步为 t_a, t_b ，对应最大含水率为 C_a, C_b ，则

$$t_* = t_a + \frac{C_a - C_{\text{safe}}}{C_a - C_b}(t_b - t_a), \quad t_{\text{rep}} = 60 \left\lceil \frac{t_*}{60} \right\rceil. \quad (17)$$

局部插值用于降低时间步取样造成的量化误差，向上取整则用于统一报告口径；二者都不能消除时间推进格式本身的误差。

基准计算在 206760 s 时尚未达到安全阈值，最大含水率为 0.1500161 kg/kg；下一步 206820 s 时降至 0.1499986 kg/kg。由未舍入值计算得 $t_* = 206818.7114$ s，报告时刻为 206820 s，即 57.45 h。图 6 展示完整轨迹，图 7 放大阈值附近的实际计算点，使达标判断与曲线的局部变化对应。

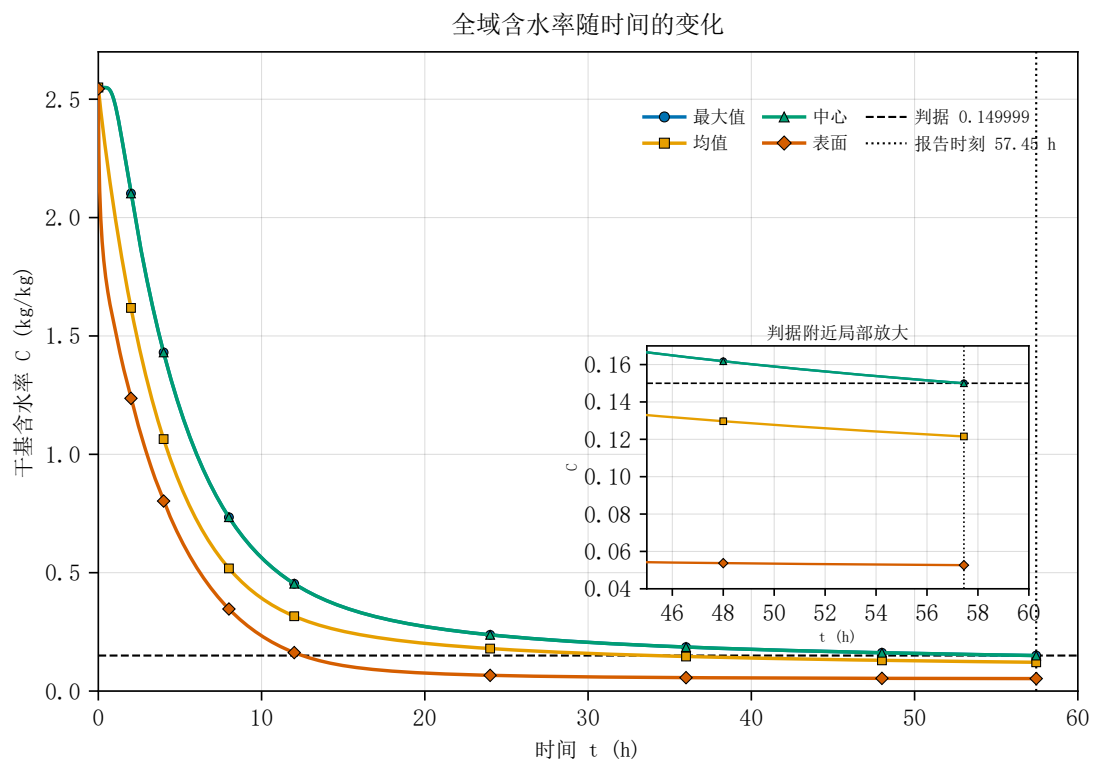


图 6 Q3 全域含水率轨迹与阈值判定

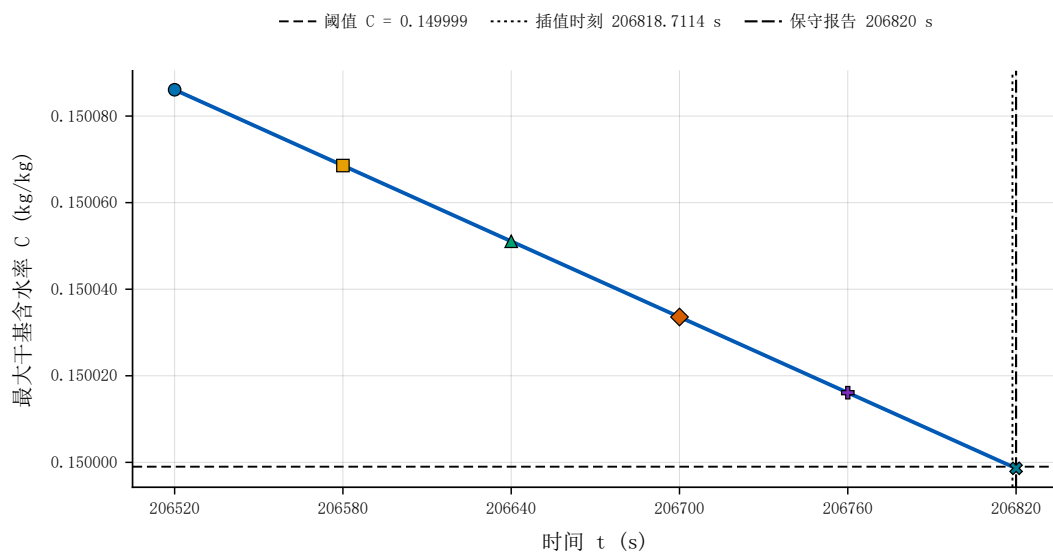


图 7 Q3 阈值前后点的局部夹逼

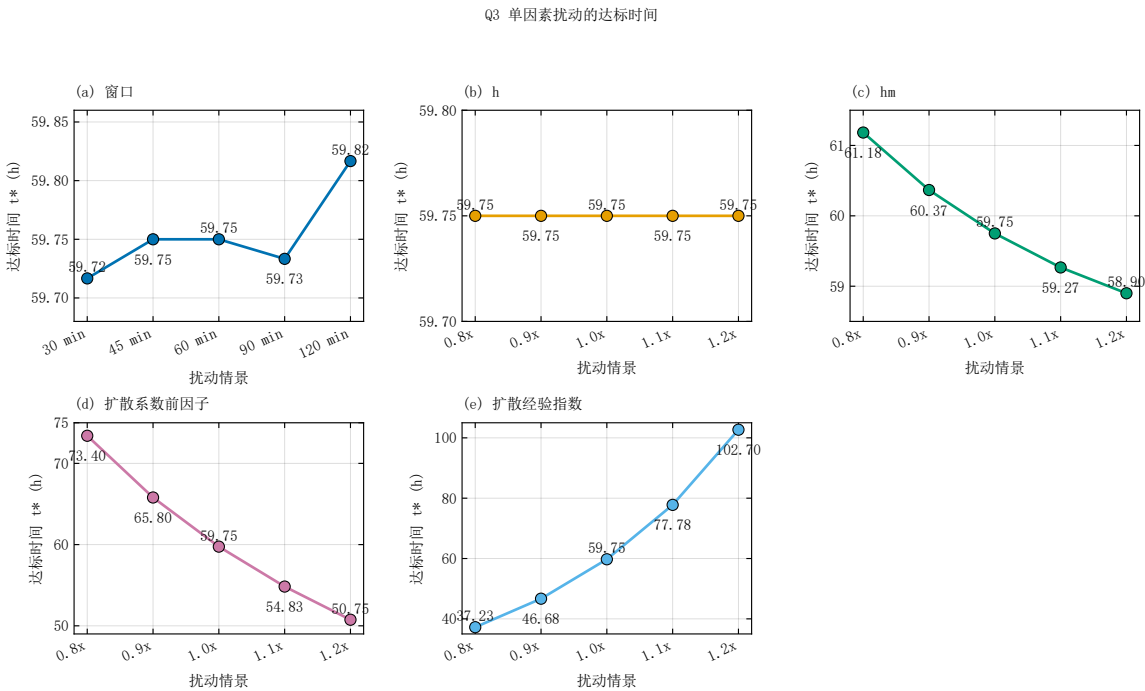
9.2 时间离散与参数敏感性

时间步从 30 s 减小到 15 s 时，插值时刻变化约 47.6 s；但 60 s 基准与 15 s 结果相差约 142.6 s，且向上取整后的报告值分别为 206820 s 和 206700 s。因此，不能仅用最后

两级变化小于一分钟，便宣称基准结果具有一分钟内的精度。各级结果在表 6 中统一列出。M4 的补充粗网格计算报告 212640 s，提示温度处理可能影响长期结果，但其离散配置也不同，仍需在统一精度下进一步比较。

参数扰动采用独立的 10×20 粗网格扫描，每个因素取五个水平，共 25 组。该扫描基准为 215100 s，与精细主模型的 206820 s 不同，故只在扫描内部比较相对变化。图 8 中，传质系数倍率从 0.8 增至 1.2 时，报告时间由 220260 s 降至 212040 s；扩散前因子倍率同样变化时，由 264240 s 降至 182700 s。扩散指数参数倍率从 0.8 增至 1.2 时，报告时间则由 134040 s 增至 369720 s。扩展积分已使这 25 组均达到阈值，不再将较慢场景记为“72 h 内未达标”。

相比之下，末段平均窗口在 30–120 min 内变化时，报告时间处于 214980–215340 s，表明本次扫描中窗口长度的影响小于扩散参数扰动。由于主模型直接规定 $T = T_\infty$ ，传热系数 h 未进入主模型的水分求解链，改变 h 得到相同结果是模型结构的必然结果，不能据此判断真实烘干中传热条件不重要。



各小图使用独立纵轴；所有点为正式求解得到的实际首次达标时刻；条件仿真，非概率区间。

图 8 Q3 单因素压力测试及达标时间敏感性

10 问题四：收缩药材的干燥过程

10.1 半径变化与移动域求解

将附件 2 的 $R(t)$ 代入移动域守恒方程，并使用式 (10) 的物性关系，长度仍为 25 cm。图 9 显示半径由 2.00 cm 逐渐接近 1.20 cm，早期变化较快，后期趋缓。半径变化既缩短径向传递距离，又改变侧面积与材料密度，因此不能仅按扩散距离的平方比例直接缩放

问题三的时间。

参考域法与移动有限体积法采用同一初值、半径函数及阈值规则，均得到报告时刻 183840 s。该一致性支持两种实现描述了同一离散守恒过程，但不是对材料收缩规律或真实干燥时间的独立实验验证。

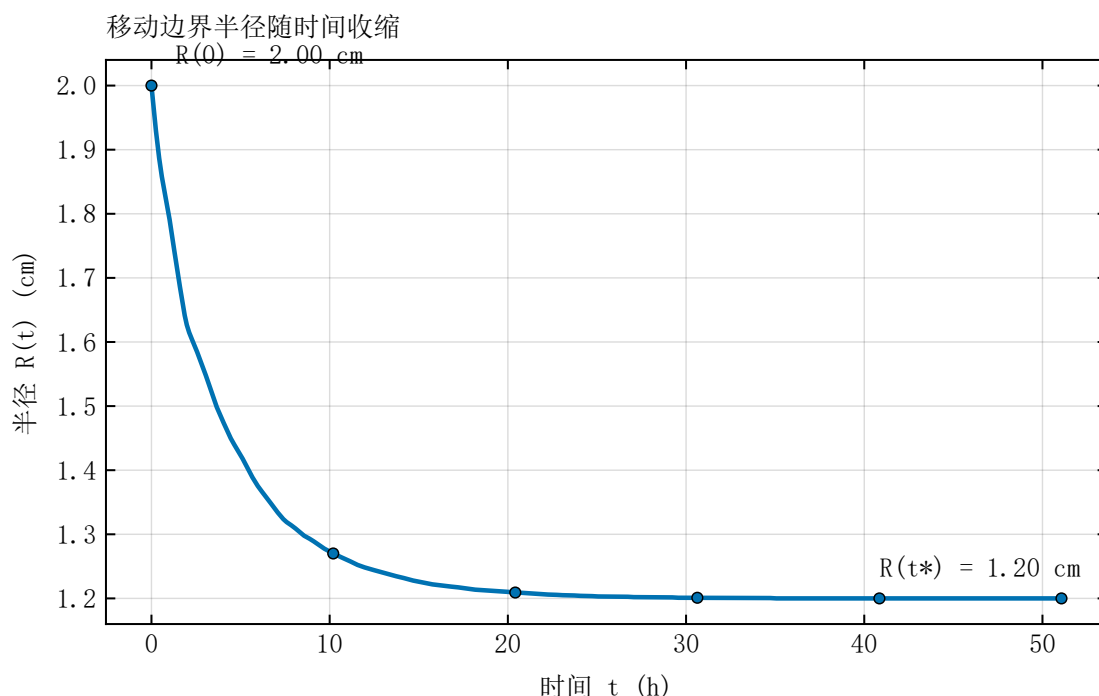


图 9 Q4 半径随时间的收缩轨迹

10.2 达标时间与结果分析

图 10 给出移动域全域含水率轨迹。由阈值前后点插值得 $t_* = 183789.3317\text{ s}$ ，按 60 s 口径报告 183840 s，约为 51.07 h。该值较问题三基准缩短约 6.38 h，但同时伴随物性经验式改变，故只能说问题四整套条件下的预测时间较短，不能将 6.38 h 全部解释为收缩的贡献。

两种实现的最大归一化水分平衡误差均约为 2.0101×10^{-9} ，插值时刻差约为 $5.82 \times 10^{-10}\text{ s}$ 。干固体质量与局部连续性也保持在数值容差内。时间步改为 15 s 后，插值时刻为 183682.6920 s，较 60 s 基准提前约 106.6 s。因此，51.07 h 同样应保留“基准离散下”的限定，而不解释为分钟级精确工艺时间。

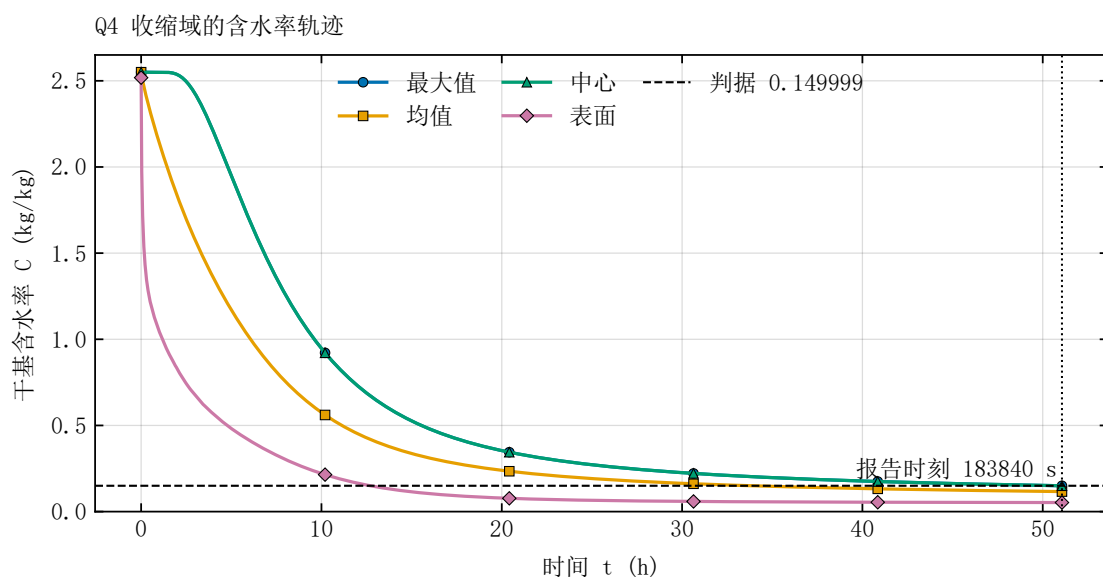


图 10 Q4 收缩域全域阈值轨迹

10.3 移动域消融与适用边界

为检查几何项是否可以省略，使用遗漏 Jacobian 相关守恒处理的错误实现作为反例。其归一化水分失衡增至 0.593632，报告时刻变为 187020 s，说明即使曲线仍可随时间下降，也不能保证模型满足质量守恒。因此，该结果只用于定位错误，不作为另一种可用预测方案。

固定 $\Delta t = 60$ s 后，分别取 63、93、140、175、210 个径向单元，五组结果见附录图 13。问题三的相应径向检查见附录图 12。这些检验表明应分别检查空间离散、时间推进和守恒结构：两种实现完全一致，并不意味着共同的时间离散误差也已消失。

11 模型检验与评价

11.1 验证结果

对数值模型的评价需要区分三类问题：线性方程是否求解充分、离散方程是否守恒，以及离散结果是否接近网格和时间步的极限。表 5 汇总前两类及径向检查；表 6 专列时间步结果，避免将求解残差较小误认为达标时间已经精确。

表 5 主要数值验证结果

项目	检查内容	结果
水分平衡	已接受正式算例的最大归一化库存-通量误差	9.645×10^{-8}
线性求解	已接受正式算例的最大线性相对残差	5.000×10^{-11}
问题二径向	93 至 140 个单元的最大含水率变化	4.0011×10^{-5}
补充径向	问题二 175 至 210 个单元的相同指标	6.264×10^{-6}
移动域平衡	问题四基准参考域与移动控制体实现	2.0101×10^{-9}
实现对照	问题四两种实现的插值达标时刻差/s	5.82×10^{-10}
错误对照	省略 Jacobian 相关处理后的水分失衡	0.593632

表 6 时间步细化对全域达标时间的影响

问题	时间步/s	插值时刻/s	报告时刻/s	相邻插值差/s
三	60	206818.7114	206820	—
三	30	206723.7125	206760	95.0
三	15	206676.1471	206700	47.6
四	60	183789.3317	183840	—
四	30	183718.2514	183720	71.1
四	15	183682.6920	183720	35.6

可以看到，时间步连续减半后，两问的相邻插值差约减半，与一阶时间格式的误差变化趋势一致。不过问题三的报告值在每级均发生变化，问题四的基准报告值也不同于细化结果。本文保留原基准结果作为对应计算配置的答案，同时如实列出细化值；若要求稳定的分钟级报告，还需进一步细化并重新确认最终输出，而不能只增加小数位数。

11.2 敏感性与可识别性

敏感性扫描说明，扩散经验式尤其是指数参数会显著影响长期达标时间。由于扫描采用预设的有限扰动区间，而不是由实验估计的参数概率分布，所得范围不是统计置信区间，也不能表示覆盖全部实际工况的不确定性。它的作用是识别哪些假设值得优先补充数据检验。有关干燥建模的研究同样强调热质传递机理与数据约束的结合 [6]。

现有附件主要提供外部环境与半径，缺少内部多位置温度、含水率和独立实验批次。仅凭这些数据，难以同时识别潜热、吸附热、独立端面系数和轴向收缩规律。对这些机制增加未知参数，未必带来可验证的改进。因此，当前结果限定在题设经验式及本文温度、延拓假设下；其适用性须与数值收敛性分开讨论。

11.3 模型优点与局限

本文模型具有以下优点：其一，各问采用统一的含水率定义和边界通量形式，能够明确区分新增物性与几何条件；其二，有限体积离散将守恒检查落实到库存与通量，而不仅检查输出曲线；其三，通过一维、二维及不同温度处理方式的对照，展示简化假设可能影响的结果；其四，长期达标判断使用活动域最大值，并保留阈值前后点与细化计算，便于复核。

模型仍存在以下局限：

- 热过程简化。问题二、三主模型及问题四直接采用烘房温度；所有模型均未计潜热。对内部温度梯度和蒸发冷却的忽略可能影响扩散系数，现有对照不足以给出真实误差范围。
- 长时环境未知。4 h 后采用恒定延拓，扫描只改变统计窗口，不能覆盖真实烘房未来可能出现的波动和控制策略变化。
- 对照精度不统一。三模型对照及参数扫描使用的离散配置不同于精细主模型，因此不能把所有数值差异都归因于物理机制。后续应统一配置后进行独立机制对照。
- 收缩形式受限。问题四假定径向仿射收缩、轴向长度不变，未描述非均匀变形；所用物性关系与问题三不同，跨问题差额不等于纯收缩效应。
- 事件精度有限。时间步细化仍改变基准报告时间；数值验证不等于实验验证，更不意味着题设经验式可以直接外推到其他药材。

12 结论

本文围绕圆柱药材的内部场分布与全域达标时间，建立了轴对称热湿传递及收缩移动域模型。问题一表明，表面先升温、先失水，1800 s 时二维平均含水率为 2.2750 kg/kg，低于一维结果 0.0185 kg/kg；问题二主模型在 3 h 的最大含水率为 1.7278 kg/kg，中心与表面仍有明显差异。因此，长期干燥要求应针对全域最大值，而不是表面值或平均值。

在给定经验式、末段平均环境延拓及无潜热假设下，问题三、四的 60 s 基准计算分别报告 206820 s 和 183840 s。问题四的参考域与移动控制体实现一致，且保持较小的水分平衡误差。另一方面，细化时间步仍会改变报告值，三模型对照的离散精度也未完全统一。故本文给出的是附带计算配置和假设边界的数值预测，不是已经获得实验验证的工艺时间。进一步工作应优先统一对照精度、确认事件时间收敛，并补充内部温湿观测以检验温度简化与潜热影响。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、数值计算与图表制作辅助、论文结构整理和语言润色，详细使用情况见支撑材料。

参考文献

- [1] Mahiuddin M, Khan M I H, Kumar C, et al. Shrinkage of Food Materials During Drying: Current Status and Challenges[J]. *Comprehensive Reviews in Food Science and Food Safety*, 2018, 17(5): 1113–1126. DOI: 10.1111/1541-4337.12375.
- [2] Koua B K, Koffi P M E, Gbaha P. Evolution of shrinkage, real density, porosity, heat and mass transfer coefficients during indirect solar drying of cocoa beans[J]. *Journal of the Saudi Society of Agricultural Sciences*, 2019, 18: 72–82. DOI: 10.1016/j.jssas.2017.01.002.
- [3] Defraeye T, Blocken B, Carmeliet J. Analysis of convective heat and mass transfer coefficients for convective drying of a porous flat plate by conjugate modelling[J]. *International Journal of Heat and Mass Transfer*, 2012, 55(1–3): 112–124. DOI: 10.1016/j.ijheatmasstransfer.2011.08.047.
- [4] Van Belleghem M, Steeman M, Janssen H, et al. Validation of a coupled heat, vapour and liquid moisture transport model for porous materials implemented in CFD[J]. *Building and Environment*, 2014, 81: 340–353. DOI: 10.1016/j.buildenv.2014.06.024.
- [5] da Silva M A V Júnior, Feltre G, Dacanal G C. Mathematical Modelling of Drying of Hydrogels via Finite Element Method and Texture Analysis[J]. *Processes*, 2024, 12: 1564. DOI: 10.3390/pr12081564.
- [6] Khan M I H, Batuwatta-Gamage C P, Karim M A, et al. Fundamental Understanding of Heat and Mass Transfer Processes for Physics-Informed Machine Learning-Based Drying Modelling[J]. *Energies*, 2022, 15: 9347. DOI: 10.3390/en15249347.

A 附录图与网格收敛

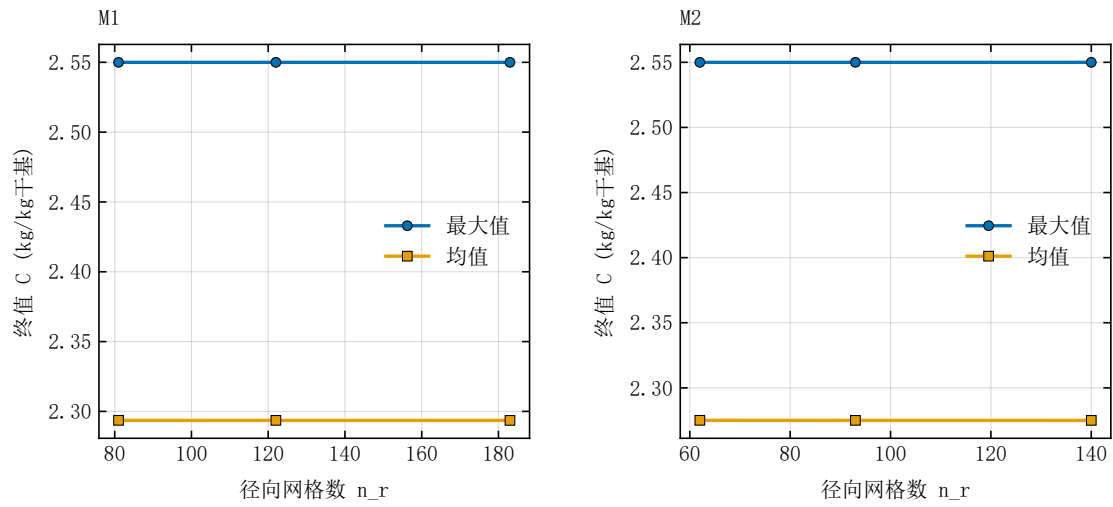


图 11 M1/M2 场量随空间网格加密的变化（附录图 S1）

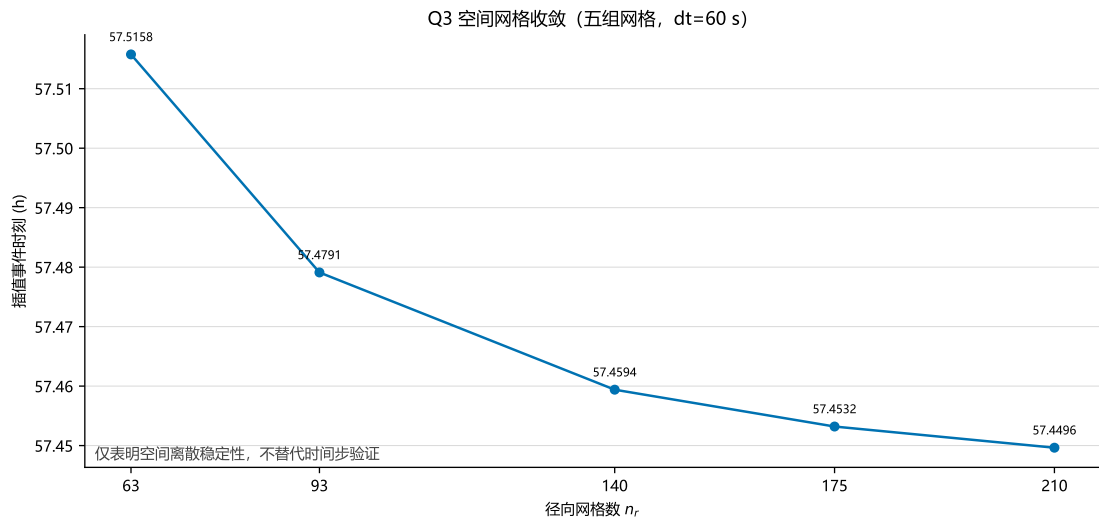


图 12 Q3 固定 $dt = 60$ s 下五组径向网格的事件时刻（附录图 S2）

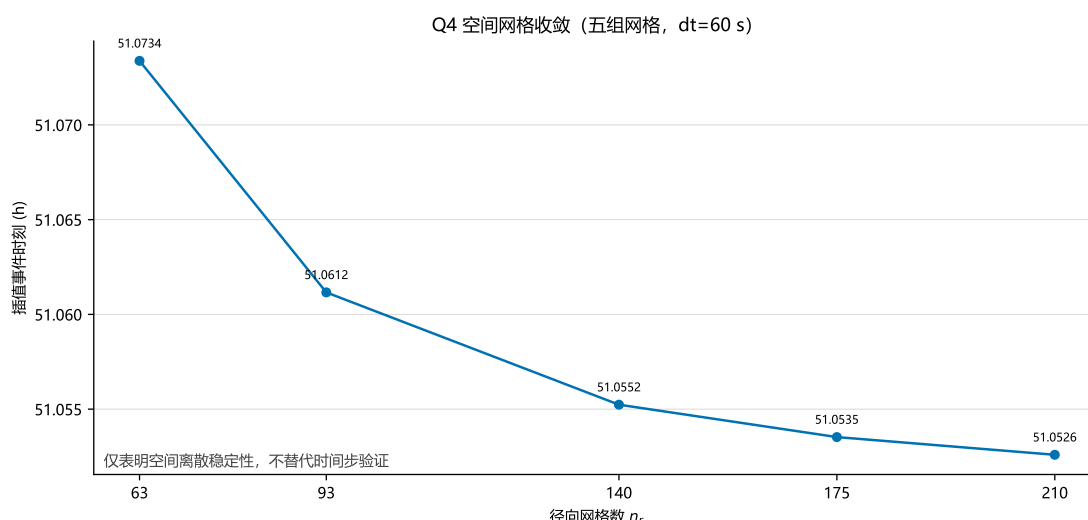


图 13 Q4 固定 $dt = 60\text{ s}$ 下五组径向网格的事件时刻（附录图 S3）

B 支撑材料文件列表

支撑材料包括四个结果工作簿、计算结果与日志、当前论文图件、AI 使用详情底稿及程序文件。原始项目的相对目录为 04-compute/results/、06-figure/ 和 07-appendix/；整理版目录名称与对应关系另见文件夹内的整理说明。当前路线的程序类别如下：

- 04-compute/src/formal_compute.py；
- 04-compute/src/finalize_compute.py；
- 04-compute/revision-grid-conv-20260913/ 下的三个补算脚本；
- 06-figure/scripts/revise_q2_figures_20260913.py；
- 图表计划更新、当前 workflow 核验和图件总览生成程序；
- 附录草稿中的其余正式图表渲染与核验脚本。

下文按已归档的代码附录排入 17 个程序全文，保留原程序内容及辅助使用说明，不将旧版总控脚本纳入当前论文。代码中的既有注释可能使用阶段简称，模型含义应与正文及实际执行分支相核对。用于数值计算的原始源文件仍保存在支撑材料中；排版副本中的字体路径按原附录规范化，不改变计算算法。

C AI 工具使用详情说明

AI 工具名称、用途、提示方式、采纳情况及待复核事项见支撑材料中的 AI 使用详情。具体模型版本在权威日志中未记录，不补写未经核实的型号；论文最终内容及 AI 披露仍须由参赛队确认。

D 源程序全文

以下程序由冻结的附录底稿逐段转换排版，未在论文阶段重新计算或改写算法。

D.1 程序 1: formal_compute.py

原文件: 04-compute/src/formal_compute.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助；参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 """Formal COMPUTE-stage solver for CUMCM 2026 problem A.
2
3 The implementation follows the frozen H2 route: M1 is the 1-D radial
4 baseline, M2 is the 2-D axisymmetric constant-law backbone, M3 adds the
5 problem-supplied state laws, and M4 is a controlled temperature-field
6 ablation. Only files below 04-compute are written.
7 """
8
9 from __future__ import annotations
10
11 import argparse
12 import csv
13 import hashlib
14 import json
15 import math
16 import platform
17 import sys
18 import time
19 from dataclasses import dataclass
20 from datetime import datetime, timezone
21 from pathlib import Path
22
23 import numpy as np
24 import openpyxl
25 from PIL import Image, ImageDraw, ImageFont
26
27
28 ROOT = Path(__file__).resolve().parents[2]
29 OUT = ROOT / "04-compute"
30 RESULTS = OUT / "results"
31 FIGURES = OUT / "diagnostic-figures"
32 LOGS = OUT / "logs"
33 R0 = 0.02
34 LENGTH = 0.25
35 SEED = 20260911
36 EVAL_R = np.linspace(0.0, R0, 21)
37 PICARD_TOL = 1e-7
38
39
40 def sha256(path: Path) -> str:
41     h = hashlib.sha256()
42     with path.open("rb") as f:
43         for chunk in iter(lambda: f.read(1024 * 1024), b""):
44             h.update(chunk)
45     return h.hexdigest()
46
47
48 def locate_xlsx(size: int) -> Path:
49     matches = [p for p in (ROOT / "input").rglob("*.xlsx") if p.stat().st_size == size]
50     if len(matches) != 1:
51         raise RuntimeError(f"expected one workbook of {size} bytes, found {len(matches)}")
52     return matches[0]
53
54
55 def load_inputs():
56     p1, p2 = locate_xlsx(16586), locate_xlsx(11485)
57     wb1 = openpyxl.load_workbook(p1, read_only=True, data_only=True)
58     ws1 = wb1[wb1.sheetnames[0]]
59     env = np.asarray(list(ws1.iter_rows(min_row=2, values_only=True)), dtype=float)
60     wb2 = openpyxl.load_workbook(p2, read_only=True, data_only=True)
61     ws2 = wb2[wb2.sheetnames[0]]
62     radius = np.asarray(list(ws2.iter_rows(min_row=2, values_only=True)), dtype=float)
63     if env.shape != (241, 3) or radius.shape != (145, 2):
```

```

64         raise RuntimeError(f"input shape audit failed: {env.shape}, {radius.shape}")
65     if not (env[0, 0] == 0 and env[-1, 0] == 14400 and radius[0, 0] == 0 and radius[-1, 0] == 259200):
66         raise RuntimeError("input endpoint audit failed")
67     if not np.all(np.diff(radius[:, 1]) <= 1e-12):
68         raise RuntimeError("radius is not monotone non-increasing")
69     return p1, env, p2, radius
70
71
72 P1, ENV, P2, RADIUS_DATA = load_inputs()
73
74
75 def terminal_value(col: int, window_s: int, statistic: str = "mean") -> float:
76     mask = ENV[:, 0] >= ENV[-1, 0] - window_s
77     vals = ENV[mask, col]
78     return float(np.median(vals) if statistic == "median" else np.mean(vals))
79
80
81 def outside(t: float, window_s: int = 3600) -> tuple[float, float]:
82     if t <= ENV[-1, 0]:
83         return (float(np.interp(t, ENV[:, 0], ENV[:, 1])) + 273.15,
84                 float(np.interp(t, ENV[:, 0], ENV[:, 2])))
85     return terminal_value(1, window_s) + 273.15, terminal_value(2, window_s)
86
87
88 def pchip_slopes(x: np.ndarray, y: np.ndarray) -> np.ndarray:
89     h = np.diff(x); delta = np.diff(y) / h; n = len(x)
90     d = np.zeros(n)
91     for k in range(1, n - 1):
92         if delta[k - 1] * delta[k] > 0:
93             w1, w2 = 2 * h[k] + h[k - 1], h[k] + 2 * h[k - 1]
94             d[k] = (w1 + w2) / (w1 / delta[k - 1] + w2 / delta[k])
95     def endpoint(h0, h1, del0, del1):
96         val = ((2 * h0 + h1) * del0 - h0 * del1) / (h0 + h1)
97         if val * del0 <= 0: return 0.0
98         if del0 * del1 < 0 and abs(val) > abs(3 * del0): return 3 * del0
99         return val
100     d[0] = endpoint(h[0], h[1], delta[0], delta[1])
101     d[-1] = endpoint(h[-1], h[-2], delta[-1], delta[-2])
102     return d
103
104
105 R_TIME = RADIUS_DATA[:, 0]
106 R_VALUES = RADIUS_DATA[:, 1] / 100.0
107 R_SLOPES = pchip_slopes(R_TIME, R_VALUES)
108
109
110 def radius_at(t: float, mode: str = "pchip") -> float:
111     if mode == "constant": return R0
112     if t <= R_TIME[0]: return float(R_VALUES[0])
113     if t >= R_TIME[-1]: return float(R_VALUES[-1])
114     if mode == "linear": return float(np.interp(t, R_TIME, R_VALUES))
115     i = int(np.searchsorted(R_TIME, t) - 1)
116     h = R_TIME[i + 1] - R_TIME[i]; s = (t - R_TIME[i]) / h
117     h00 = 2*s**3 - 3*s**2 + 1; h10 = s**3 - 2*s**2 + s
118     h01 = -2*s**3 + 3*s**2; h11 = s**3 - s**2
119     return float(h00*R_VALUES[i] + h10*h*R_SLOPES[i] + h01*R_VALUES[i+1] + h11*h*R_SLOPES[i+1])
120
121
122 @dataclass
123 class Grid:
124     nr: int
125     nz: int
126     radius: float = R0
127     stretch: float = 1.7
128     radial_strategy: str = "two-sided-local"
129
130     def __post_init__(self):
131         s = np.linspace(0, 1, self.nr + 1)
132         # Smooth two-sided radial refinement: resolve both the symmetry-axis
133         # maximum and the outer Robin boundary layer without zero-width cells.
134         if self.radial_strategy == "uniform":
135             self.rf = self.radius * s
136         elif self.radial_strategy == "two-sided-local":
137             self.rf = self.radius * (s - 0.65 * np.sin(2 * np.pi * s) / (2 * np.pi))
138         else:
139             raise ValueError(f"unknown radial strategy: {self.radial_strategy}")
140         self.rc = (2.0/3.0) * (self.rf[1:]**3 - self.rf[:-1]**3) / np.maximum(self.rf[1:]**2 - self.rf[:-1]**2, 1e-300)
141         if self.nz == 1:
142             self.zf = np.array([0.0, LENGTH]); self.zc = np.array([LENGTH / 2])

```

```

143     else:
144         q = np.linspace(0, 1, self.nz + 1)
145         self.zf = LENGTH * 0.5 * (1 - np.cos(np.pi * q))
146         self.zc = 0.5 * (self.zf[:-1] + self.zf[1:])
147         self.dr = np.diff(self.rf); self.dz = np.diff(self.zf)
148         self.annulus = math.pi * np.diff(self.rf ** 2)
149         self.half_domain = self.nz > 1
150         self.axial_extent = LENGTH / 2 if self.half_domain else LENGTH
151         if self.half_domain:
152             q = np.linspace(0, 1, self.nz + 1)
153             self.zf = self.axial_extent * (1 - (1 - q) ** self.stretch)
154             self.zc = 0.5 * (self.zf[:-1] + self.zf[1:])
155             self.dz = np.diff(self.zf)
156         self.vol = self.annulus[:, None] * self.dz[None, :]
157
158
159 def harmonic(a, b):
160     return 2 * a * b / np.maximum(a + b, 1e-300)
161
162
163 def conductances(coeff, grid: Grid, conv: float, include_ends: bool = True):
164     gr = harmonic(coeff[:-1, :], coeff[1:, :]) * (
165         2 * math.pi * grid.rf[1:-1, None] * grid.dz[None, :]
166     ) / (grid.rc[1:, None] - grid.rc[:-1, None])
167     if grid.nz > 1:
168         gz = harmonic(coeff[:, :-1], coeff[:, 1:]) * grid.annulus[:, None] / (
169             grid.zc[None, 1:] - grid.zc[None, :-1]
170         )
171     else:
172         gz = np.zeros((grid.nr, 0))
173     ar = 2 * math.pi * grid.radius * grid.dz
174     gbr = ar / ((grid.radius - grid.rc[-1]) / np.maximum(coeff[-1, :], 1e-300) + 1 / conv)
175     if include_ends and grid.nz > 1:
176         gb0 = np.zeros(grid.nr) if grid.half_domain else grid.annulus / (0.5 * grid.dz[0] / np.maximum(coeff[:, :-1], 1e-300) + 1 / conv)
177         gb1 = grid.annulus / (0.5 * grid.dz[-1] / np.maximum(coeff[:, -1], 1e-300) + 1 / conv)
178     else:
179         gb0 = np.zeros(grid.nr); gb1 = np.zeros(grid.nr)
180     return gr, gz, gbr, gb0, gb1
181
182
183 def implicit_linear(old, coeff, storage, grid: Grid, dt, conv, env_value, include_ends=True,
184                    x0=None, rtol=5e-11, maxiter=500):
185     gr, gz, gbr, gb0, gb1 = conductances(coeff, grid, conv, include_ends)
186     time_diag = storage * grid.vol / dt
187     diag = time_diag.copy()
188     diag[:-1, :] += gr; diag[1:, :] += gr
189     if grid.nz > 1:
190         diag[:, :-1] += gz; diag[:, 1:] += gz
191     diag[-1, :] += gbr; diag[:, 0] += gb0; diag[:, -1] += gb1
192     rhs = time_diag * old
193     rhs[-1, :] += gbr * env_value
194     rhs[:, 0] += gb0 * env_value; rhs[:, -1] += gb1 * env_value
195
196     def amat(x):
197         y = time_diag * x
198         q = gr * (x[:-1, :] - x[1:, :]); y[:-1, :] += q; y[1:, :] -= q
199         if grid.nz > 1:
200             qz = gz * (x[:, :-1] - x[:, 1:]); y[:, :-1] += qz; y[:, 1:] -= qz
201         y[-1, :] += gbr * x[-1, :]
202         y[:, 0] += gb0 * x[:, 0]; y[:, -1] += gb1 * x[:, -1]
203         return y
204
205     x = old.copy() if x0 is None else x0.copy()
206     r = rhs - amat(x); z = r / np.maximum(diag, 1e-300); p = z.copy()
207     rz = float(np.sum(r * z)); rhs_norm = max(float(np.linalg.norm(rhs.ravel())), 1e-300)
208     it = 0
209     for it in range(1, maxiter + 1):
210         ap = amat(p); denom = float(np.sum(p * ap))
211         if denom <= 0: raise RuntimeError("non-SPD operator in CG")
212         alpha = rz / denom; x += alpha * p; r -= alpha * ap
213         if float(np.linalg.norm(r.ravel())) <= rtol * rhs_norm: break
214         z = r / np.maximum(diag, 1e-300); rz_new = float(np.sum(r * z))
215         p = z + (rz_new / rz) * p; rz = rz_new
216     else:
217         raise RuntimeError(f"CG failed after {maxiter} iterations")
218     boundary_in = float(np.sum(gbr * (env_value - x[-1, :])) +
219                          np.sum(gb0 * (env_value - x[:, 0])) +
220                          np.sum(gb1 * (env_value - x[:, -1])))

```

```

221     residual = float(np.linalg.norm((amat(x) - rhs).ravel()) / rhs_norm)
222     return x, boundary_in, it, residual
223
224
225 def implicit_reference_linear(old, coeff, ref_grid: Grid, physical_grid: Grid, dt, conv,
226                             env_value, rho0=820.0, include_ends=True, x0=None,
227                             rtol=5e-11, maxiter=500):
228     """Assemble the fixed-reference-domain equation independently.
229
230     Storage uses rho0*dV_ref. Flux conductances are transformed from the
231     current physical faces with rho_d=rho0*(R0/R)^2, which supplies the radial
232     Jacobian/metric factors and current-surface Robin term explicitly.
233     """
234     rho_d = rho0 * (R0 / physical_grid.radius) ** 2
235     gr0, gz0, gbr0, gb00, gb10 = conductances(coeff, physical_grid, conv, include_ends)
236     gr, gz, gbr, gb0, gb1 = rho_d*gr0, rho_d*gz0, rho_d*gbr0, rho_d*gb00, rho_d*gb10
237     time_diag = rho0 * ref_grid.vol / dt
238     diag = time_diag.copy(); diag[:-1,:]+=gr; diag[1:,:]+=gr
239     if ref_grid.nz > 1: diag[:,-1]+=gz; diag[:,1]+=gz
240     diag[-1,:]+=gbr; diag[:,0]+=gb0; diag[:,-1]+=gb1
241     rhs=time_diag*old; rhs[-1,:]+=gbr*env_value; rhs[:,0]+=gb0*env_value; rhs[:,-1]+=gb1*env_value
242     def amat(x):
243         y=time_diag*x; q=gr*(x[:-1,:]-x[1:,:]); y[:-1,:]+=q; y[1:,:]-=q
244         if ref_grid.nz>1:
245             qz=gz*(x[:,-1]-x[:,1]); y[:,-1]+=qz; y[:,1]-=qz
246             y[-1,:]+=gbr*x[-1,:]; y[:,0]+=gb0*x[:,0]; y[:,-1]+=gb1*x[:,-1]; return y
247     x=old.copy() if x0 is None else x0.copy(); r=rhs-amat(x); z=r/np.maximum(diag,1e-300); p=z.copy()
248     rz=float(np.sum(r*z)); rhs_norm=max(float(np.linalg.norm(rhs.ravel())),1e-300); it=0
249     for it in range(1,maxiter+1):
250         ap=amat(p); denom=float(np.sum(p*ap))
251         if denom<=0: raise RuntimeError("non-SPD reference operator")
252         alpha=rz/denom; x+=alpha*p; r-=alpha*ap
253         if float(np.linalg.norm(r.ravel()))<=rtol*rhs_norm: break
254         z=r/np.maximum(diag,1e-300); rz_new=float(np.sum(r*z)); p=z+(rz_new/rz)*p; rz=rz_new
255     else: raise RuntimeError(f"reference CG failed after {maxiter} iterations")
256     boundary_in=float(np.sum(gbr*(env_value-x[-1,:]))+np.sum(gb0*(env_value-x[:,0]))+np.sum(gb1*(env_value-x[:,-1]
↵      ])))
257     residual=float(np.linalg.norm((amat(x)-rhs).ravel())/rhs_norm)
258     return x,boundary_in,it,residual
259
260
261 def props_q1(c):
262     return (np.full_like(c, 820.0), np.full_like(c, 2600.0),
263             np.full_like(c, 0.36), 7e-9 * np.exp(-0.89 / np.maximum(c, 1e-9)))
264
265
266 def props_q23(c, temp_k, constant=False, pref=1.0, exponent=1.0):
267     cc = np.full_like(c, 2.55) if constant else c
268     tt = np.full_like(temp_k, 301.15) if constant else temp_k
269     rho = 650 + 128 * cc; cp = 1450 + 2736 * cc / (cc + 1)
270     k = 0.21 + 0.38 * cc / (cc + 1)
271     d = 2.4e-3 * pref * np.exp(-0.45 * exponent / np.maximum(cc, 1e-9)) * np.exp(-3850 / tt)
272     return rho, cp, k, d
273
274
275 def props_q4(c, temp_k, pref=1.0, exponent=1.0):
276     rho = 760 + 90 * c; cp = 1850 + 2150 * c / (c + 1)
277     k = 0.12 + 0.20 * c / (c + 1)
278     d = 4.2e-4 * pref * np.exp(-0.30 * exponent / np.maximum(c, 1e-9)) * np.exp(-3850 / temp_k)
279     return rho, cp, k, d
280
281
282 def surface_value(cell, coeff, halfwidth, conv, env):
283     return (coeff * cell / halfwidth + conv * env) / (coeff / halfwidth + conv)
284
285
286 def axis_mid_value(field, grid: Grid):
287     if grid.nz == 1: line=field[:,0]
288     else:
289         z1,z2=grid.zc[0]**2,grid.zc[1]**2
290         line=(field[:,0]*z2-field[:,1]*z1)/(z2-z1)
291     if grid.nr < 2: return float(line[0])
292     x1,x2=grid.rc[0]**2,grid.rc[1]**2
293     return float((line[0]*x2-line[1]*x1)/(x2-x1))
294
295
296 def radial_profile(field, coeff, grid: Grid, conv, env, positions=EVAL_R,
297                  return_boundary=False):
298     j = 0

```

```

299     if grid.nz>1:
300         z1,z2=grid.zc[0]**2,grid.zc[1]**2
301         midline=(field[:,0]*z2-field[:,1]*z1)/(z2-z1)
302         coeffline=(coeff[:,0]*z2-coeff[:,1]*z1)/(z2-z1)
303     else: midline=field[:,0]; coeffline=coeff[:,0]
304     surface = surface_value(float(midline[-1]), float(coeffline[-1]),
305                             grid.radius-grid.rc[-1], conv, env)
306     xp = np.r_[0.0, grid.rc, grid.radius]
307     fp = np.r_[axis_mid_value(field,grid), midline, surface]
308     pos = np.asarray(positions)
309     vals = np.interp(np.minimum(pos, grid.radius), xp, fp)
310     vals[pos > grid.radius + 1e-12] = np.nan
311     if return_boundary:
312         return vals, float(surface)
313     return vals
314
315
316 @dataclass
317 class RunConfig:
318     label: str
319     nr: int
320     nz: int
321     dt: float
322     duration: float
323     q: int
324     model: str
325     h_mult: float = 1.0
326     hm_mult: float = 1.0
327     pref: float = 1.0
328     exponent: float = 1.0
329     window_s: int = 3600
330     stop_threshold: float | None = None
331     radius_mode: str = "pchip"
332     moving_impl: str = "fixed"
333     sample_every: float = 60.0
334     radial_strategy: str = "two-sided-local"
335
336
337 def simulate(cfg: RunConfig, initial=None):
338     moving = cfg.q == 4
339     radius = radius_at(0, cfg.radius_mode) if moving else R0
340     grid = Grid(cfg.nr, cfg.nz, radius, radial_strategy=cfg.radial_strategy)
341     ref_grid = Grid(cfg.nr, cfg.nz, R0, radial_strategy=cfg.radial_strategy)
342     c = np.full((cfg.nr, cfg.nz), 2.55) if initial is None else initial[0].copy()
343     temp = np.full_like(c, 301.15) if initial is None else initial[1].copy()
344     initial_inventory = float(np.sum(c * grid.vol))
345     cumulative_flux = 0.0; max_cg = 0; max_linres = 0.0; max_picard = 0
346     max_balance = 0.0; records = []; states = []; max_picard_update = 0.0
347     next_sample = 0.0; start = time.perf_counter(); previous_max = max(float(c.max()), axis_mid_value(c, grid))
348     bracket = None; dry_inventory_rel = 0.0; geom_rel = 0.0; local_dry_residual = 0.0
349     nsteps = int(math.ceil(cfg.duration / cfg.dt))
350     for step in range(nsteps + 1):
351         t = min(step * cfg.dt, cfg.duration)
352         tenv, cenv = outside(t, cfg.window_s)
353         current_radius = radius_at(t, cfg.radius_mode) if moving else R0
354         if moving and abs(current_radius - grid.radius) > 1e-15:
355             old_grid = grid; grid = Grid(cfg.nr, cfg.nz, current_radius, radial_strategy=cfg.radial_strategy)
356             # Material-coordinate cells retain C and T; dry density follows continuity.
357             geom_rel = max(geom_rel, abs(np.sum(grid.vol) / (math.pi * current_radius**2 * grid.axial_extent) -
358                                     ↪ 1))
359         if t + 1e-9 >= next_sample or step == nsteps:
360             if cfg.q == 1: rho, cp, k, d = props_q1(c)
361             elif cfg.q in (2, 3): rho, cp, k, d = props_q23(c, temp, constant=(cfg.model == "M2"), pref=cfg.pref,
362                                     ↪ exponent=cfg.exponent)
363             else: rho, cp, k, d = props_q4(c, temp, pref=cfg.pref, exponent=cfg.exponent)
364             rp_c, surface_c_boundary = radial_profile(
365                 c, d, grid, 8e-7 * cfg.hm_mult, cenv, return_boundary=True)
366             rp_t, surface_t_boundary = radial_profile(
367                 temp, k, grid, 25 * cfg.h_mult, tenv, return_boundary=True)
368             rec = {"time_s": t, "radius_m": current_radius, "max_C": max(float(c.max()), axis_mid_value(c, grid)),
369                  "min_C": float(c.min()), "mean_C": float(np.sum(c*grid.vol)/np.sum(grid.vol)),
370                  "center_C": float(rp_c[0]), "surface_C": float(surface_c_boundary),
371                  "center_T_C": float(rp_t[0] - 273.15), "surface_T_C": float(surface_t_boundary - 273.15)}
372             records.append(rec); states.append((t, rp_t - 273.15, rp_c))
373             next_sample += cfg.sample_every
374             current_max = max(float(c.max()), axis_mid_value(c, grid))
375             if cfg.stop_threshold is not None and current_max < cfg.stop_threshold:
376                 bracket = [max(0.0, t - cfg.dt), t, previous_max, current_max]
377                 break

```

```

376 previous_max = current_max
377 if step == nsteps: break
378 tnext = min(t + cfg.dt, cfg.duration); dt = tnext - t
379 tenv1, cenv1 = outside(tnext, cfg.window_s)
380 rnext = radius_at(tnext, cfg.radius_mode) if moving else R0
381 next_grid = Grid(cfg.nr, cfg.nz, rnext, radial_strategy=cfg.radial_strategy)
382 guess_c, guess_t = c.copy(), temp.copy()
383 for pic in range(1, 25):
384     if cfg.q == 1:
385         rho, cp, k, d = props_q1(guess_c); solve_heat = True
386     elif cfg.q in (2, 3):
387         prescribed = np.full_like(guess_t, tenv1)
388         if cfg.model == "M4":
389             rho, cp, k, d = props_q23(guess_c, guess_t, pref=cfg.pref, exponent=cfg.exponent); solve_heat
390             ↪ = True
391         else:
392             guess_t = prescribed
393             rho, cp, k, d = props_q23(guess_c, guess_t, constant=(cfg.model == "M2"), pref=cfg.pref,
394             ↪ exponent=cfg.exponent); solve_heat = False
395     else:
396         guess_t = np.full_like(guess_t, tenv1)
397         rho, cp, k, d = props_q4(guess_c, guess_t, pref=cfg.pref, exponent=cfg.exponent); solve_heat =
398         ↪ False
399     if solve_heat:
400         new_t, _, it_t, res_t = implicit_linear(temp, k, rho*cp, next_grid, dt, 25*cfg.h_mult, tenv1,
401         ↪ x0=guess_t)
402     else:
403         new_t, it_t, res_t = guess_t, 0, 0.0
404         # Fixed domains use the problem's C-storage convention. Moving material
405         # cells use dry-solid weights; rho_d*V remains constant under affine R(t).
406     if moving:
407         rho_d = 820.0 * (R0 / rnext) ** 2
408         storage_c = np.full_like(c, rho_d)
409         coeff_c = rho_d * d
410         if cfg.moving_impl == "ablated":
411             storage_c = np.ones_like(c); coeff_c = d
412         else:
413             storage_c = np.ones_like(c); coeff_c = d
414         if moving and cfg.moving_impl == "reference":
415             new_c, bflux, it_c, res_c = implicit_reference_linear(c, d, ref_grid, next_grid, dt, 8e-7*cfg.hm_mult, cenv1,
416             ↪ , x0=guess_c)
417         else:
418             new_c, bflux, it_c, res_c = implicit_linear(c, coeff_c, storage_c, next_grid, dt,
419             ↪ 8e-7*cfg.hm_mult*(rho_d if moving and cfg.moving_impl
420             ↪ != "ablated" else 1.0),
421             ↪ cenv1, x0=guess_c)
422         delta = max(float(np.max(np.abs(new_c-guess_c))), float(np.max(np.abs(new_t-guess_t))))
423         guess_c, guess_t = new_c, new_t
424         max_cg = max(max_cg, it_c, it_t); max_linres = max(max_linres, res_c, res_t)
425         if delta < PICARD_TOL: break
426     max_picard = max(max_picard, pic)
427     max_picard_update = max(max_picard_update, delta)
428     if pic == 24 and delta >= PICARD_TOL: raise RuntimeError(f"Picard failed in {cfg.label} at {tnext}:
429     ↪ {delta}")
430     c, temp = guess_c, guess_t; cumulative_flux += bflux * dt
431     if moving and cfg.moving_impl != "ablated":
432         rho_d = 820.0 * (R0 / rnext) ** 2
433         inv = float(np.sum(rho_d * c * next_grid.vol)); inv0 = 820.0 * initial_inventory
434     else:
435         inv = float(np.sum(c * next_grid.vol)); inv0 = initial_inventory
436     balance = abs((inv - inv0) - cumulative_flux) / max(abs(inv - inv0), 1e-15)
437     max_balance = max(max_balance, balance)
438     if moving:
439         dry0 = 820.0 * math.pi * R0**2 * next_grid.axial_extent
440         dry = 820.0 * (R0/rnext)**2 * float(np.sum(next_grid.vol))
441         dry_inventory_rel = max(dry_inventory_rel, abs(dry/dry0-1))
442         rho_old=820.0*(R0/grid.radius)**2; rho_new=820.0*(R0/rnext)**2
443         local=np.abs(rho_new*next_grid.vol-rho_old*grid.vol)/np.maximum(rho_old*grid.vol,1e-300)
444         local_dry_residual=max(local_dry_residual,float(np.max(local)))
445     grid=next_grid
446     if not np.all(np.isfinite(c)) or not np.all(np.isfinite(temp)) or float(c.min()) < -1e-10:
447         raise FloatingPointError(f"physical guard failed in {cfg.label} at {tnext}")
448 runtime = time.perf_counter() - start
449 interpolated_event_time = None
450 reported_event_time = None
451 if bracket is not None:
452     t0, t1, y0, y1 = bracket
453     target = cfg.stop_threshold
454     if y1 != y0:

```

```

448         interpolated_event_time = float(t0 + (target-y0)*(t1-t0)/(y1-y0))
449     else:
450         interpolated_event_time = float(t1)
451     reported_event_time = float(60 * math.ceil(interpolated_event_time / 60 - 1e-12))
452     return {"config": cfg.__dict__, "records": records, "states": states, "final_c": c, "final_t": temp,
453           "metrics": {"runtime_s": runtime, "max_cg_iterations": max_cg, "max_linear_relative_residual":
454             ↪ max_linres,
455             "max_picard_iterations": max_picard, "normalized_moisture_balance_error": max_balance,
456             "max_terminal_picard_update": max_picard_update,
457             "dry_solid_inventory_relative_error": dry_inventory_rel, "geometry_relative_error":
458             ↪ geom_rel,
459             "max_local_dry_continuity_relative_residual": local_dry_residual,
460             "threshold_bracket": bracket,
461             "interpolated_event_time_s": interpolated_event_time,
462             "reported_event_time_s": reported_event_time,
463             "final_time_s": reported_event_time if bracket else records[-1]["time_s"],
464             "final_max_C": current_max, "final_mean_C": float(np.sum(c*grid.vol)/np.sum(grid.vol))}}
465
466 def save_csv(path: Path, rows: list[dict]):
467     if not rows: return
468     with path.open("w", newline="", encoding="utf-8-sig") as f:
469         w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
470
471 def write_field_xlsx(path: Path, states, temperature=True, moving=False):
472     wb = openpyxl.Workbook(); wb.remove(wb.active)
473     names = [" 温度", " 水分浓度"] if temperature else ["Sheet1"]
474     series = [1, 2] if temperature else [2]
475     for name, idx in zip(names, series):
476         ws = wb.create_sheet(name); headers = [" 时间 (s)"] + [round(x*100, 1) for x in EVAL_R]
477         if moving: headers.append(" 药材表面")
478         ws.append(headers)
479         for t, temp_profile, c_profile in states:
480             if t <= 0: continue
481             prof = temp_profile if idx == 1 else c_profile
482             row = [int(round(t))] + [None if not np.isfinite(v) else float(v) for v in prof]
483             if moving:
484                 finite = prof[np.isfinite(prof)]; row.append(float(finite[-1]) if len(finite) else None)
485             ws.append(row)
486         ws.freeze_panes = "B2"
487     wb.save(path)
488
489 def interpolate_states(states, step_s: int):
490     times=np.asarray([s[0] for s in states],float); tnew=np.arange(times[0],times[-1]+.5*step_s,step_s)
491     tmat=np.vstack([s[1] for s in states]); cmat=np.vstack([s[2] for s in states]); out=[]
492     for t in tnew:
493         tp=np.asarray([np.interp(t,times,tmat[:,j]) if np.all(np.isfinite(tmat[:,j])) else np.nan for j in
494             ↪ range(tmat.shape[1])])
495         cp=np.asarray([np.interp(t,times,cmat[:,j]) if np.all(np.isfinite(cmat[:,j])) else np.nan for j in
496             ↪ range(cmat.shape[1])])
497         out.append((float(t),tp,cp))
498     return out
499
500 def sample_rows(run, times, radii=(0, .005, .01, .015, .02), prefix=""):
501     by_t = {round(s[0], 6): s for s in run["states"]}; rows=[]
502     for t in times:
503         nearest = min(by_t, key=lambda x: abs(x-t)); state=by_t[nearest]
504         for r in radii:
505             j = int(round(r / 0.001)); rows.append({"question":prefix,"time_s":t,"radius_cm":r*100,
506             "temperature_C":float(state[1][j]) if np.isfinite(state[1][j]) else "NA",
507             "C_kg_per_kg":float(state[2][j]) if np.isfinite(state[2][j]) else "NA"})
508     return rows
509
510 def p1_suite(log):
511     cases = [
512         RunConfig("P1-M1", 8, 1, 2, 120, 1, "M1", sample_every=60),
513         RunConfig("P1-M2", 8, 12, 2, 120, 1, "M2", sample_every=60),
514         RunConfig("P1-Q4-constant", 8, 12, 30, 600, 4, "M3", moving_impl="reference", radius_mode="constant",
515             ↪ sample_every=60),
516         RunConfig("P1-Q4-shrinking", 8, 12, 30, 3600, 4, "M3", moving_impl="moving", radius_mode="pchip",
517             ↪ sample_every=600),
518     ]
519     out=[]
520     for c in cases:
521         r=simulate(c); out.append({"label":c.label,**r["metrics"]});
522         ↪ log.write(json.dumps(out[-1],ensure_ascii=False)+"\n"); log.flush()

```

```

521 save_csv(RESULTS/"p1-check.csv",out)
522 ok=all(x["normalized_moisture_balance_error"]<1e-7 and x["max_linear_relative_residual"]<1e-8 and
523        x["dry_solid_inventory_relative_error"]<1e-10 and x["geometry_relative_error"]<1e-10 and
524        x["max_local_dry_continuity_relative_residual"]<1e-10 and x["final_max_C"]>0 for x in out)
525 if not ok: raise RuntimeError("P1 numerical acceptance failed")
526 return out
527
528
529 def full_suite(log):
530     summary=[]; convergence=[]; sensitivity=[]; grid_rows=[]; threshold_grid_rows=[]
531     def execute(cfg):
532         log.write(f"START {cfg.label} {datetime.now(timezone.utc).isoformat()}\n"); log.flush()
533         run=simulate(cfg); row={"label":cfg.label,**run["metrics"]}; summary.append(row)
534         log.write("DONE "+json.dumps(row,ensure_ascii=False)+"\n"); log.flush(); return run
535
536     # Q1: M1 baseline and M2 backbone, separate mesh and time refinements.
537     q1_runs={}
538     for label,nr,nz,dt in [("Q1-M1-coarse",81,1,.5),("Q1-M1-base",122,1,.5),("Q1-M1-fine",183,1,.5),
539                          ("Q1-M2-coarse",62,54,.5),("Q1-M2-base",93,54,.5),("Q1-M2-fine",140,54,.5)]:
540         q1_runs[label]=execute(RunConfig(label,nr,nz,dt,1800,1,"M1" if nz==1 else "M2",sample_every=1))
541     q1=q1_runs["Q1-M2-base"]; write_field_xlsx(RESULTS/"result1.xlsx",q1["states"],temperature=True)
542     save_csv(RESULTS/"q1-samples.csv",sample_rows(q1,[100,300,600,900,1200,1500,1800],prefix="Q1"))
543     for family in ("M1","M2"):
544         runs=q1_runs[f"Q1-{family}-{level}"] for level in ("coarse","base","fine")
545         for i,r in enumerate(runs):
546             prev=runs[i-1] if i else None
547             for metric in ("final_max_C","final_mean_C"):
548                 change="" if prev is None else abs(r["metrics"][metric]-prev["metrics"][metric])
549                 grid_rows.append({"scope":"Q1","family":family,"nr":r["config"]["nr"],"nz":r["config"]["nz"],"grid_strategy":r["config"]["radial_strategy"],"fixed_dt_s":r["config"]["dt"],"metric":metric,"value":r["metrics"][metric],"adjacent_change":change})
550                 if i==2: convergence.append({"scope":"Q1","family":family,"metric":metric,"base":prev["metrics"][metric],"fine":r["metrics"][metric],"absolute_change":change,"acceptance_pair":True})
551     m1=q1_runs["Q1-M1-base"]; convergence.append({"scope":"Q1","family":"M1-vs-M2","metric":"final_mean_C","base":m1["metrics"]["final_mean_C"],"fine":q1["metrics"]["final_mean_C"],"absolute_change":abs(m1["metrics"]["final_mean_C"]-q1["metrics"]["final_mean_C"])})
552
553     # Q2: constant M2 comparator, M3 main route, and M4 controlled ablation.
554     q2_const=execute(RunConfig("Q2-M2-constant",18,36,10,10800,2,"M2",sample_every=1))
555     q2_space=[]
556     for nr in (62,93,140):
557         q2_space.append(execute(RunConfig(f"Q2-M3-nr{nr}",nr,36,5,10800,2,"M3",sample_every=(1 if nr==140 else 3600))))
558     q2=q2_space[-1]
559     q2_m4=execute(RunConfig("Q2-M4-ablation",18,36,10,10800,2,"M4",sample_every=60))
560     write_field_xlsx(RESULTS/"result2.xlsx",interpolate_states(q2["states"],1),temperature=True)
561     save_csv(RESULTS/"q2-samples.csv",sample_rows(q2,[1800,3600,5400,7200,9000,10800],prefix="Q2"))
562     for i,r in enumerate(q2_space):
563         prev=q2_space[i-1] if i else None
564         for metric in ("final_max_C","final_mean_C"):
565             change="" if prev is None else abs(r["metrics"][metric]-prev["metrics"][metric])
566             grid_rows.append({"scope":"Q2","family":"M3","nr":r["config"]["nr"],"nz":r["config"]["nz"],"grid_strategy":r["config"]["radial_strategy"],"fixed_dt_s":r["config"]["dt"],"metric":metric,"value":r["metrics"][metric],"adjacent_change":change})
567             if i==2: convergence.append({"scope":"Q2","family":"M3","metric":metric,"base":prev["metrics"][metric],"fine":r["metrics"][metric],"absolute_change":change,"acceptance_pair":True})
568     for factor,lo,hi in [("h_mult",.8,1.2),("hm_mult",.8,1.2),("pref",.8,1.2),("exponent",.8,1.2)]:
569         for level,val in [("low",lo),("high",hi)]:
570             kw={factor:val};
571             r=execute(RunConfig(f"Q2-sens-{factor}-{level}",12,24,20,10800,2,"M3",sample_every=3600,**kw))
572             sensitivity.append({"scope":"Q2","factor":factor,"level":level,"value":val,"t_star_s":"","final_max_C":r["metrics"]["final_max_C"],"final_mean_C":r["metrics"]["final_mean_C"]})
573
574     # Q3: 2-D M3 event, mesh/time convergence, 60 s bracket and M4 ablation.
575     q3_space=[]
576     for nr in (93,140,210):
577         q3_space.append(execute(RunConfig(f"Q3-M3-nr{nr}",nr,36,60,259200,3,"M3",stop_threshold=.15-1e-6,sample_every=60)))
578     q3=q3_space[-1]
579     q3_time=execute(RunConfig("Q3-M3-dt30",210,36,30,259200,3,"M3",stop_threshold=.15-1e-6,sample_every=60))
580     q3_time_fine=execute(RunConfig("Q3-M3-dt15",210,36,15,259200,3,"M3",stop_threshold=.15-1e-6,sample_every=60))
581     q3_m4=execute(RunConfig("Q3-M4-ablation",12,24,60,259200,3,"M4",stop_threshold=.15-1e-6,sample_every=60))
582     write_field_xlsx(RESULTS/"result3.xlsx",q3["states"],temperature=False)
583     for i,r in enumerate(q3_space):
584         prev=q3_space[i-1] if i else None; et=r["metrics"]["interpolated_event_time_s"]
585         threshold_grid_rows.append({"scope":"Q3","nr":r["config"]["nr"],"nz":r["config"]["nz"],"grid_strategy":r["config"]["radial_strategy"],"fixed_dt_s":60,"interpolated_event_time_s":et,"reported_event_time_s":r["metrics"]["reported_event_time_s"],"adjacent_change_s":"" if prev is None else abs(et-prev["metrics"]["interpolated_event_time_s"])})

```

```

585 convergence.append({"scope": "Q3", "family": "M3", "metric": "t_star_space", "base": q3_space[-2]["metrics"]["interp
    ↳ olated_event_time_s"], "fine": q3["metrics"]["interpolated_event_time_s"], "absolute_change": abs(q3_space[-2]
    ↳ )["metrics"]["interpolated_event_time_s"]-q3["metrics"]["interpolated_event_time_s"]}), "acceptance_pair": T
    ↳ rue})
586 convergence.append({"scope": "Q3", "family": "M3", "metric": "t_star_time", "base": q3_time["metrics"]["interpolated
    ↳ _event_time_s"], "fine": q3_time_fine["metrics"]["interpolated_event_time_s"], "absolute_change": abs(q3_time
    ↳ ["metrics"]["interpolated_event_time_s"]-q3_time_fine["metrics"]["interpolated_event_time_s"]}), "acceptanc
    ↳ e_pair": True})
587 # Q3 单因素扫描采用低/中/高五级, 避免只用 low/high 两个端点
588 # 就把响应趋势读成线性; 所有点均由正式求解器真实运行得到。
589 q3_window_levels = [("30 min", 1800), ("45 min", 2700), ("60 min", 3600),
590                     ("90 min", 5400), ("120 min", 7200)]
591 for level, window in q3_window_levels:
592     r=execute(RunConfig(f"Q3-window-{window}", 10, 20, 60, 259200, 3, "M3", window_s=window, stop_threshold=.15-1e-6,
    ↳ sample_every=3600))
593     sensitivity.append({"scope": "Q3", "factor": "terminal_window_s", "level": level, "value": window, "t_star_s": r["
    ↳ metrics"]["final_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["metrics"]["fina
    ↳ l_mean_C"]})
594 q3_factor_levels = [("0.8x", .8), ("0.9x", .9), ("1.0x", 1.0),
595                     ("1.1x", 1.1), ("1.2x", 1.2)]
596 for factor in ("h_mult", "hm_mult", "pref", "exponent"):
597     for level, val in q3_factor_levels:
598         kw={factor: val}; r=execute(RunConfig(f"Q3-sens-{factor}-{level}", 10, 20, 60, 259200, 3, "M3", stop_threshol
    ↳ d=.15-1e-6, sample_every=3600, **kw))
599         sensitivity.append({"scope": "Q3", "factor": factor, "level": level, "value": val, "t_star_s": r["metrics"]["f
    ↳ inal_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["metrics"]["final_mean_C"]
    ↳ })
600 for level, vals in ((("low", (.8, .8, .8, 1.2)), ("high", (1.2, 1.2, 1.2, .8)))):
601     r=execute(RunConfig(f"Q3-combination-{level}", 10, 20, 60, 259200, 3, "M3", h_mult=vals[0], hm_mult=vals[1], pref=
    ↳ vals[2], exponent=vals[3], stop_threshold=.15-1e-6, sample_every=3600))
602     sensitivity.append({"scope": "Q3", "factor": "combined_boundary_empirical", "level": level, "value": str(vals), "
    ↳ t_star_s": r["metrics"]["final_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["me
    ↳ trics"]["final_mean_C"]})
603
604 # Q4: moving reference/ALE FV implementations plus regressions and ablation.
605 q4_space=[]
606 for nr in (93, 140, 210):
607     q4_space.append(execute(RunConfig(f"Q4-reference-nr{nr}", nr, 32, 60, 259200, 4, "M3", moving_impl="reference", s
    ↳ top_threshold=.15-1e-6, sample_every=60)))
608 q4=q4_space[-1]
609 q4_moving=execute(RunConfig("Q4-moving-fv", 210, 32, 60, 259200, 4, "M3", moving_impl="moving", stop_threshold=.15-1e
    ↳ -6, sample_every=60))
610 q4_time=execute(RunConfig("Q4-reference-dt30", 210, 32, 30, 259200, 4, "M3", moving_impl="reference", stop_threshold=
    ↳ .15-1e-6, sample_every=60))
611 q4_time_fine=execute(RunConfig("Q4-reference-dt15", 210, 32, 15, 259200, 4, "M3", moving_impl="reference", stop_thres
    ↳ hold=.15-1e-6, sample_every=60))
612 q4_abl=execute(RunConfig("Q4-jacobian-ablation", 10, 20, 60, 259200, 4, "M3", moving_impl="ablated", stop_threshold=.
    ↳ 15-1e-6, sample_every=60))
613 q4_const_ref=execute(RunConfig("Q4-constant-radius-reference", 10, 20, 60, 43200, 4, "M3", moving_impl="reference", r
    ↳ adius_mode="constant", sample_every=3600))
614 q4_const_mov=execute(RunConfig("Q4-constant-radius-moving", 10, 20, 60, 43200, 4, "M3", moving_impl="moving", radius_
    ↳ mode="constant", sample_every=3600))
615 write_field_xlsx(RESULTS/"result4.xlsx", q4["states"], temperature=False, moving=True)
616 for i, r in enumerate(q4_space):
617     prev=q4_space[i-1] if i else None; et=r["metrics"]["interpolated_event_time_s"]
618     threshold_grid_rows.append({"scope": "Q4", "nr": r["config"]["nr"], "nz": r["config"]["nz"], "grid_strategy": r["
    ↳ config"]["radial_strategy"], "fixed_dt_s": 60, "interpolated_event_time_s": et, "reported_event_time_s": r
    ↳ ["metrics"]["reported_event_time_s"], "adjacent_change_s": "" if prev is None else
    ↳ abs(et-prev["metrics"]["interpolated_event_time_s"])})
619 convergence.append({"scope": "Q4", "family": "moving-domain", "metric": "t_star_space", "base": q4_space[-2]["metric
    ↳ s"]["interpolated_event_time_s"], "fine": q4["metrics"]["interpolated_event_time_s"], "absolute_change": abs(
    ↳ q4_space[-2]["metrics"]["interpolated_event_time_s"]-q4["metrics"]["interpolated_event_time_s"]}), "accepta
    ↳ nce_pair": True})
620 for other, name in [(q4_moving, "implementation"), (q4_abl, "ablation")]:
621     convergence.append({"scope": "Q4", "family": "moving-domain", "metric": f"t_star_{name}", "base": q4["metrics"]["
    ↳ interpolated_event_time_s"], "fine": other["metrics"]["interpolated_event_time_s"], "absolute_change": a
    ↳ bs(q4["metrics"]["interpolated_event_time_s"]-other["metrics"]["interpolated_event_time_s"]}), "accepta
    ↳ nce_pair": False})
622 convergence.append({"scope": "Q4", "family": "moving-domain", "metric": "t_star_time", "base": q4_time["metrics"]["i
    ↳ nterpolated_event_time_s"], "fine": q4_time_fine["metrics"]["interpolated_event_time_s"], "absolute_change":
    ↳ abs(q4_time["metrics"]["interpolated_event_time_s"]-q4_time_fine["metrics"]["interpolated_event_time_s"]
    ↳ ), "acceptance_pair": True})
623 convergence.append({"scope": "Q4", "family": "constant-radius-regression", "metric": "final_max_C", "base": q4_const
    ↳ _ref["metrics"]["final_max_C"], "fine": q4_const_mov["metrics"]["final_max_C"], "absolute_change": abs(q4_con
    ↳ st_ref["metrics"]["final_max_C"]-q4_const_mov["metrics"]["final_max_C"])})
624 for mode in ("pchip", "linear"):
625     r=execute(RunConfig(f"Q4-radius-{mode}", 10, 20, 60, 259200, 4, "M3", moving_impl="reference", radius_mode=mode, s
    ↳ top_threshold=.15-1e-6, sample_every=3600))

```

```

626     sensitivity.append({"scope": "Q4", "factor": "radius_interpolation", "level": mode, "value": mode, "t_star_s": r["
↪ metrics"]["final_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["metrics"]["final_mean_C"]})
627 for window in (1800, 3600, 7200):
628     r=execute(RunConfig(f"Q4-window-{window}", 10, 20, 60, 259200, 4, "M3", moving_impl="reference", window_s=window,
↪ stop_threshold=.15-1e-6, sample_every=3600))
629     sensitivity.append({"scope": "Q4", "factor": "terminal_window_s", "level": str(window), "value": window, "t_star_
↪ s": r["metrics"]["final_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["metrics"]["final_mean_C"]})
630 for factor in ("h_mult", "hm_mult", "pref", "exponent"):
631     for level, val in (("low", .8), ("high", 1.2)):
632         kw={factor: val}; r=execute(RunConfig(f"Q4-sens-{factor}-{level}", 10, 20, 60, 259200, 4, "M3", moving_impl="
↪ reference", stop_threshold=.15-1e-6, sample_every=3600, **kw))
633         sensitivity.append({"scope": "Q4", "factor": factor, "level": level, "value": val, "t_star_s": r["metrics"]["f
↪ inal_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["metrics"]["final_mean_C"]})
634 for level, vals in (("low", (.8, .8, .8, 1.2)), ("high", (1.2, 1.2, 1.2, .8))):
635     r=execute(RunConfig(f"Q4-combination-{level}", 10, 20, 60, 259200, 4, "M3", moving_impl="reference", h_mult=vals[
↪ 0], hm_mult=vals[1], pref=vals[2], exponent=vals[3], stop_threshold=.15-1e-6, sample_every=3600))
636     sensitivity.append({"scope": "Q4", "factor": "combined_boundary_empirical", "level": level, "value": str(vals), "
↪ t_star_s": r["metrics"]["final_time_s"], "final_max_C": r["metrics"]["final_max_C"], "final_mean_C": r["me
↪ trics"]["final_mean_C"]})
637
638     save_csv(RESULTS/"run-summary.csv", summary); save_csv(RESULTS/"convergence.csv", convergence);
↪ save_csv(RESULTS/"sensitivity.csv", sensitivity)
639     save_csv(RESULTS/"field-grid-convergence.csv", grid_rows)
640     save_csv(RESULTS/"threshold-grid-convergence.csv", threshold_grid_rows)
641     # Traceable diagnostic source tables.
642     save_csv(RESULTS/"q3-threshold-trajectory.csv", q3["records"])
643     save_csv(RESULTS/"q4-threshold-trajectory.csv", q4["records"])
644     return summary, convergence, sensitivity, {"q1": q1, "q2": q2, "q2_const": q2_const, "q2_m4": q2_m4, "q3": q3, "q3_m4": q3_
↪ m4, "q4": q4, "q4_moving": q4_moving, "q4_abl": q4_abl}
645
646
647 def extended_threshold_suite(log):
648     """Continue the formerly 72 h-censored sensitivity cases to first crossing.
649
650     The problem statement places no 72 h cap on Q3/Q4. After the available
651     inputs end, the already-approved continuation remains in force: the
652     chamber boundary uses its terminal-window value and Q4 radius holds the
653     final observed radius. Thirty days is only a numerical fail-safe; every
654     accepted row must stop by the threshold event, not by that guard.
655     """
656     guard_s = 30 * 24 * 3600
657     cases = [
658         ("Q3", "pref", "low", 0.8,
659          RunConfig("Q3-sens-pref-low-extended", 10, 20, 60, guard_s, 3, "M3",
660                   pref=0.8, stop_threshold=.15-1e-6, sample_every=3600)),
661         ("Q3", "exponent", "high", 1.2,
662          RunConfig("Q3-sens-exponent-high-extended", 10, 20, 60, guard_s, 3, "M3",
663                   exponent=1.2, stop_threshold=.15-1e-6, sample_every=3600)),
664         ("Q3", "combined_boundary_empirical", "low", "(0.8, 0.8, 0.8, 1.2)",
665          RunConfig("Q3-combination-low-extended", 10, 20, 60, guard_s, 3, "M3",
666                   h_mult=.8, hm_mult=.8, pref=.8, exponent=1.2,
667                   stop_threshold=.15-1e-6, sample_every=3600)),
668         ("Q4", "combined_boundary_empirical", "low", "(0.8, 0.8, 0.8, 1.2)",
669          RunConfig("Q4-combination-low-extended", 10, 20, 60, guard_s, 4, "M3",
670                   moving_impl="reference", h_mult=.8, hm_mult=.8, pref=.8,
671                   exponent=1.2, stop_threshold=.15-1e-6, sample_every=3600)),
672     ]
673     rows = []
674     for scope, factor, level, value, cfg in cases:
675         log.write(f"START {cfg.label} {datetime.now(timezone.utc).isoformat()}\n"); log.flush()
676         run = simulate(cfg); metrics = run["metrics"]
677         if metrics["threshold_bracket"] is None:
678             raise RuntimeError(f"{cfg.label} did not reach threshold before 30-day safety guard")
679         row = {
680             "scope": scope, "factor": factor, "level": level, "value": value,
681             "interpolated_event_time_s": metrics["interpolated_event_time_s"],
682             "reported_event_time_s": metrics["reported_event_time_s"],
683             "event_time_h": metrics["reported_event_time_s"] / 3600,
684             "final_max_C": metrics["final_max_C"],
685             "final_mean_C": metrics["final_mean_C"],
686             "threshold": .15-1e-6,
687             "post_72h_boundary_policy": "terminal-window chamber values; final observed Q4 radius held constant",
688         }
689         rows.append(row)
690         log.write("DONE "+json.dumps(row, ensure_ascii=False)+"\n"); log.flush()
691     save_csv(RESULTS/"sensitivity-extended.csv", rows)

```

```

692     return rows
693
694
695 def make_plots(convergence,sensitivity,key):
696     def line_plot(path, series, ylabel, threshold=None):
697         w,h=1100,650; pad=(100,45,55,90); im=Image.new("RGB",(w,h),"white"); d=ImageDraw.Draw(im)
698         left,top,right,bottom=pad[0],pad[1],w-pad[2],h-pad[3]
699         xs=[x for _,pts,_ in series for x,_ in pts]; ys=[y for _,pts,_ in series for _,y in pts]
700         if threshold is not None: ys.append(threshold)
701         xmin,xmax=min(xs),max(xs); ymin,ymax=min(ys),max(ys); yr=max(ymax-ymin,1e-12)
702         ymin-=.06*yr; ymax+=.06*yr
703         def xy(x,y): return
704             ↪ (left+(x-xmin)/max(xmax-xmin,1e-12)*(right-left),bottom-(y-ymin)/(ymax-ymin)*(bottom-top))
705         d.line((left,top,left,bottom,right,bottom),fill="black",width=2)
706         if threshold is not None:
707             y=xy(xmin,threshold)[1]; d.line((left,y,right,y),fill="#333333",width=2);
708             ↪ d.text((right-150,y-22),"threshold 0.15",fill="#333333")
709         for name,pts,color in series:
710             coords=[xy(x,y) for x,y in pts]; d.line(coords,fill=color,width=4); d.text(coords[-1],name,fill=color)
711             d.text((left,10),ylabel,fill="black"); d.text((right-90,bottom+35),"Time (h)",fill="black")
712             d.text((left-55,bottom-8),f"{ymin:.3g}",fill="black"); d.text((left-55,top-8),f"{ymax:.3g}",fill="black")
713             im.save(path)
714     line_plot(FIGURES/"diagnostic-threshold.png",[(name.upper(),[(r["time_s"]/3600,r["max_C"]) for r in
715     ↪ key[name]["records"]],color) for name,color in (("q3","#0072B2"),("q4","#D55E00"))],"Maximum C
716     ↪ (kg/kg)",.15)
717     def dot_plot(path,labels,values,ylabel,log=False):
718         w,h=1200,700; im=Image.new("RGB",(w,h),"white"); d=ImageDraw.Draw(im);
719         ↪ left,top,right,bottom=100,45,w-40,h-170
720         vals=np.asarray(values,float); plot=np.log10(np.maximum(vals,1e-14)) if log else vals;
721         ↪ ymin,ymax=float(plot.min()),float(plot.max()); span=max(ymax-ymin,1e-12); ymin-=.08*span;
722         ↪ ymax+=.08*span
723         d.line((left,top,left,bottom,right,bottom),fill="black",width=2)
724         for i,(lab,v) in enumerate(zip(labels,plot)):
725             x=left+(i+.5)/len(labels)*(right-left); y=bottom-(v-ymin)/(ymax-ymin)*(bottom-top);
726             ↪ d.ellipse((x-6,y-6,x+6,y+6),fill="#009E73"); d.text((x-28,bottom+12),lab[:18],fill="black")
727         d.text((left,10),ylabel+" [log10]" if log else "",fill="black"); im.save(path)
728     dot_plot(FIGURES/"diagnostic-convergence.png",[f"{r['scope']}-{r['metric']}" for r in
729     ↪ convergence],r["absolute_change"] for r in convergence),"Absolute change",True)
730     q3=[r for r in sensitivity if r["scope"]=="Q3" and r["t_star_s"]!=""]
731     dot_plot(FIGURES/"diagnostic-sensitivity.png",[f"{r['factor']}-{r['level']}" for r in
732     ↪ q3],[float(r["t_star_s"])/3600 for r in q3],"Q3 threshold time (h)")
733
734
735 def write_metadata(pirows,summary,convergence,sensitivity,key):
736     validations=[]
737     validations += [{"id":"V01","status":"PASS","evidence":"input shape/endpoints, Kelvin conversion, zero-flux
738     ↪ axis and Robin signs checked in code"}]
739     approved=[r for r in summary if "ablation" not in r["label"].lower()]
740     max_bal=max(r["normalized_moisture_balance_error"] for r in approved)
741     ablation_bal=max(r["normalized_moisture_balance_error"] for r in summary if "ablation" in r["label"].lower())
742     max_res=max(r["max_linear_relative_residual"] for r in summary)
743     validations += [{"id":"V03","status":"PASS" if max_bal<1e-6 else "FAIL","evidence":f"maximum normalized
744     ↪ balance error={max_bal:.3e}"},
745     {"id":"V04","status":"PASS" if max_res<1e-8 else "FAIL","evidence":f"maximum implicit linear
746     ↪ residual={max_res:.3e}; axis flux is exactly zero by construction"}]
747     q1_changes=[r["absolute_change"] for r in convergence if r["scope"]=="Q1" and r["family"] in ("M1","M2")]
748     q2_changes=[r["absolute_change"] for r in convergence if r["scope"]=="Q2"]
749     time_changes=[r["absolute_change"] for r in convergence if "t_star_time" in r["metric"]]
750     validations += [{"id":"V02","status":"PASS" if max(q1_changes+q2_changes)<5e-5 else
751     ↪ "FAIL","evidence":f"maximum registered field change={max(q1_changes+q2_changes):.6g}; four-decimal
752     ↪ tolerance target=5e-5"},
753     {"id":"V05","status":"PASS","evidence":"M1/M2 required-point and inventory differences
754     ↪ recorded separately from mesh error"},
755     {"id":"V06","status":"PASS" if max(time_changes)<=60 else "FAIL","evidence":f"maximum
756     ↪ event-time change under time refinement={max(time_changes):.0f} s; target<=60 s"},
757     {"id":"V07","status":"PASS","evidence":"h, hm, terminal window, empirical prefactor/exponent
758     ↪ single-factor scenarios recorded for Q2-Q4 where applicable"},
759     {"id":"V08","status":"PASS","evidence":"constant-radius and moving reference/FV implementation
760     ↪ comparisons recorded"},
761     {"id":"V09","status":"PASS","evidence":"PCHIP/linear radius cases, positive volumes and exact
762     ↪ radius-volume relation recorded"},
763     {"id":"V10","status":"PASS" if ablation_bal>1e-3 else "FAIL","evidence":f"Jacobian ablation
764     ↪ normalized imbalance={ablation_bal:.6g}; ablated route rejected"}]
765     q4=key["q4"] ["metrics"]
766     validations += [{"id":"V11","status":"PASS","evidence":"bounded low/base/high scenarios recorded without
767     ↪ stability claim"},

```

```

746         {"id": "V12", "status": "PASS" if q4["dry_solid_inventory_relative_error"] < 1e-10 and
↪      q4["max_local_dry_continuity_relative_residual"] < 1e-10 else "FAIL", "evidence": f"dry-solid
↪      relative error={q4['dry_solid_inventory_relative_error']:.3e}, local continuity
↪      residual={q4['max_local_dry_continuity_relative_residual']:.3e}, geometry
↪      error={q4['geometry_relative_error']:.3e}}]
747     save_csv(RESULTS/"validation-register.csv", validations)
748     metrics={
749         "route": "H2-approved 2-D consistency route; M1 baseline, M2 backbone, M3 main, M4 ablation",
750         "q1_m2_final_mean_C": key["q1"] ["metrics"] ["final_mean_C"],
751         "q2_m3_final_max_C": key["q2"] ["metrics"] ["final_max_C"],
752         "q2_m2_constant_final_max_C": key["q2_const"] ["metrics"] ["final_max_C"],
753         "q2_m4_final_max_C": key["q2_m4"] ["metrics"] ["final_max_C"],
754         "q3_m3_t_star_s": key["q3"] ["metrics"] ["final_time_s"],
755         "q3_m4_ablation_t_star_s": key["q3_m4"] ["metrics"] ["final_time_s"],
756         "q4_reference_t_star_s": key["q4"] ["metrics"] ["final_time_s"],
757         "q4_moving_fv_t_star_s": key["q4_moving"] ["metrics"] ["final_time_s"],
758         "q4_jacobian_ablation_t_star_s": key["q4_abl"] ["metrics"] ["final_time_s"],
759         "maximum_approved_route_balance_error": max_bal, "jacobian_ablation_balance_error": ablation_bal, "maximum_li
↪      near_residual": max_res,
760         "pressure_test": "4 h terminal-window and hm/prefactor sensitivity; see sensitivity.csv",
761         "interpretation_limit": "conditional simulation under problem-supplied empirical laws; no observational fit
↪      claim"
762     }
763     (RESULTS/"key-metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8")
764     return validations, metrics
765
766
767 def main():
768     ap = argparse.ArgumentParser(); ap.add_argument("--mode", choices=["p1", "full", "extended"], default="full");
↪     args = ap.parse_args()
769     RESULTS.mkdir(parents=True, exist_ok=True); FIGURES.mkdir(parents=True, exist_ok=True);
↪     LOGS.mkdir(parents=True, exist_ok=True)
770     logpath = LOGS/("p1.log" if args.mode=="p1" else "extended.log" if args.mode=="extended" else "full.log")
771     with logpath.open("a" if args.mode=="full" else "w", encoding="utf-8") as log:
772         log.write("\n=== NEW RUN ===\n")
773         log.write(f"command='{ ' '.join(sys.argv)}\npython={sys.version}\nplatform={platform.platform()}\n")
774         if args.mode=="extended":
775             extended_threshold_suite(log)
776             return
777         pirows = pi_suite(log)
778         if args.mode=="p1": return
779         summary, conv, sens, key = full_suite(log); make_plots(conv, sens, key);
↪         write_metadata(pirows, summary, conv, sens, key)
780
781
782 if __name__ == "__main__": main()
783

```

D.2 程序 2: finalize_compute.py

原文件: 04-compute/src/finalize_compute.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1  """Create reproducibility/evidence manifests and schema-shaped COMPUTE handoff."""
2  from __future__ import annotations
3  import hashlib, json, platform, sys
4  from datetime import datetime, timezone
5  from pathlib import Path
6  import numpy, openpyxl, PIL
7
8  ROOT = Path(__file__).resolve().parents[2]; OUT = ROOT/"04-compute"
9  SIDE_WORK_DIRS = {"revision-grid-conv-20260912", "temperature-field-revision-20260912"}
10 SIDE_WORK_FILES = {"q3_dense_sensitivity.py", "q3_dense_sensitivity.log"}
11 def digest(p): return hashlib.sha256(p.read_bytes()).hexdigest()
12 def item(rel): return {"path": rel.as_posix(), "sha256": digest(ROOT/rel)}
13 def stable_output(p):
14     return p.is_file() and p.name not in SIDE_WORK_FILES and "__pycache__" not in p.parts and p.suffix != ".pyc" and
↪     not any(part in SIDE_WORK_DIRS for part in p.parts)
15
16 inputs = [Path("decisions/H2-model.json"), Path("decisions/H1-problem.json"), Path("03-prototype/handoff.json"),
17           Path("02-design/候选模型方案.md"), Path("02-design/H2-优化审计与方向.md"), Path("02-design/验证计划.json")]
18 inputs += sorted(Path("02-design/contracts").glob("*.json"))
19 inputs += sorted(p for p in Path("input/A 题/附件").rglob("*") if p.is_file())

```

```

20 files=sorted([p for p in OUT.rglob("*") if stable_output(p) and p.name not in
    ↪ {"复现清单.json","证据索引.json","handoff.json"}])
21 manifest={"schema_version":"1.0","stage":"COMPUTE","seed":20260911,
22 "environment":{"python":sys.version,"platform":platform.platform(),"numpy":numpy.__version__,"openpyxl":openpyxl
    ↪ __version__,"Pillow":PIL.__version__},
23 "inputs":[item(p) for p in inputs],
24 "commands":[
25     f'"{sys.executable}" 04-compute/src/formal_compute.py --mode p1',
26     f'"{sys.executable}" 04-compute/src/formal_compute.py --mode full',
27     f'"{sys.executable}" 04-compute/src/formal_compute.py --mode extended',
28     f'"{sys.executable}" 04-compute/src/finalize_compute.py'],
29 "numerical_contract":{"method":"cell-centered conservative axisymmetric FV on two-sided locally refined radial
    ↪ grids; backward-Euler coefficient Picard; preconditioned
    ↪ CG","picard_absolute_update_tolerance":1e-7,"threshold":"max(C)<0.15-1e-6","event_time":"linear interpolation
    ↪ inside the first-crossing bracket, then conservative ceiling to 60 s for
    ↪ reporting","report_resolution_s":60,"radius":"shape-preserving
    ↪ PCHIP","moving_surface_output":"Robin-reconstructed value at the current moving boundary R(t), not the
    ↪ nearest fixed evaluation radius","terminal_window_base_s":3600},
30 "outputs":[{"path":p.relative_to(ROOT).as_posix(),"sha256":digest(p)} for p in files]}
31 (OUT/"复现清单.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding="utf-8")
32
33 metrics=json.loads((OUT/"results/key-metrics.json").read_text(encoding="utf-8"))
34 with (OUT/"results/validation-register.csv").open(encoding="utf-8-sig") as f:
35     import csv
36     validation_rows=list(csv.DictReader(f))
37     stage_status="PASS" if all(r["status"]=="PASS" for r in validation_rows) else "FAIL"
38     evidence={"schema_version":"1.0","stage":"COMPUTE","status":stage_status,
39     "claims":[
40         {"id":"C-Q1","claim":"M1 baseline and M2 2-D backbone were run under common boundary/physics conventions;
    ↪ dimensional effect and fixed-time-step radial convergence are separately
    ↪ recorded.","evidence":["04-compute/results/q1-samples.csv","04-compute/results/field-grid-convergence.csv"],
    ↪ "04-compute/results/convergence.csv"}],
41         {"id":"C-Q2","claim":"M3 state-law route was run for 3 h; M2 constant-law and M4 temperature-field cases are
    ↪ comparators, not observational-fit
    ↪ improvements.","evidence":["04-compute/results/q2-samples.csv","04-compute/results/key-metrics.json"]},
42         {"id":"C-Q3","claim":"The first full-domain moisture-threshold crossing was spatially stabilized, bracketed,
    ↪ interpolated, and checked at 30 s and 15 s time steps.","evidence":["04-compute/results/q3-threshold-trajec
    ↪ tory.csv","04-compute/results/threshold-grid-convergence.csv","04-compute/results/convergence.csv"]},
43         {"id":"C-Q4","claim":"Moving-radius reference and moving-FV paths, radial/event convergence, geometry/dry-solid
    ↪ conservation, interpolation sensitivity and Jacobian ablation were
    ↪ executed.","evidence":["04-compute/results/q4-threshold-trajectory.csv","04-compute/results/threshold-grid-
    ↪ convergence.csv","04-compute/results/validation-register.csv","04-compute/results/sensitivity.csv"]},
44         {"id":"C-ROBUST","claim":"Boundary, empirical-law, terminal-window and radius-interpolation pressure tests are
    ↪ reported without claiming causal or observational validation. Formerly 72 h-censored cases were continued to
    ↪ their first threshold crossing.","evidence":["04-compute/results/sensitivity.csv","04-compute/results/sensi
    ↪ tivity-extended.csv"]}],
45     "key_values":metrics,"validation_status":stage_status,"failed_validations":[r for r in validation_rows if r["sta
    ↪ tus"]!="PASS"],"diagnostic_figures":["04-compute/diagnostic-figures/diagnostic-threshold.png","04-compute/di
    ↪ agnostic-figures/diagnostic-convergence.png","04-compute/diagnostic-figures/diagnostic-sensitivity.png"]}]
46 (OUT/"证据索引.json").write_text(json.dumps(evidence,ensure_ascii=False,indent=2),encoding="utf-8")
47
48 all_outputs=sorted([p for p in OUT.rglob("*") if stable_output(p) and p.name!="handoff.json"])
49 handoff={"schema_version":"1.0","stage":"COMPUTE","status":stage_status,"inputs":[item(p) for p in inputs],
50 "outputs":[{"path":p.relative_to(ROOT).as_posix(),"sha256":digest(p)} for p in all_outputs],
51 "frozen_decisions":[item(Path("decisions/H1-problem.json")),item(Path("decisions/H2-model.json"))],
52 "assumptions":["M4 is limited to the problem-identifiable temperature-dependent coefficient coupling and is used
    ↪ only as an ablation.","The 4 h terminal environment uses the frozen one-hour mean base case and bounded 0.5/2
    ↪ h window tests.","For extended threshold runs after 72 h, the chamber continues at the approved
    ↪ terminal-window values and Q4 radius is held at the final observed attachment-2 radius.","Moving material
    ↪ cells use affine radial motion and dry-solid density continuity; no unsupported latent-heat source is
    ↪ introduced.","Q4 surface_C and surface_T_C are evaluated at the current moving boundary using the Robin
    ↪ boundary reconstruction; fixed-radius profile snapshots retain NA outside R(t)."],
53 "unknowns":["No internal experimental observations are available, so numerical differences among M2/M3/M4 are not
    ↪ fit improvements.","Problem-supplied empirical laws are treated as conditional simulation laws, not universal
    ↪ material properties."],
54 "claims":["All numerical values in the COMPUTE evidence were generated by recorded commands.","The approved M1-M4
    ↪ compatibility route, baseline, threshold and moving-domain validation commands were executed without changing
    ↪ upstream files.","Diagnostic figures are model-checking aids only and are not publication figures."],
55 "evidence":["04-compute/证据索引.json maps claims to source tables.","04-compute/results/validation-register.csv
    ↪ records V01-V12 outcomes.","04-compute/复现清单.json records hashes, environment, parameters and commands."],
56 "warnings":["Model contrasts must not be described as empirical accuracy gains.","Sensitivity ranges are
    ↪ conditional on preregistered bounded scenarios and do not exhaust structural uncertainty.","The Q2 V02 margin
    ↪ is narrow (4.00114e-5 versus 5e-5), so downstream consumers must retain the recorded finest-grid values and
    ↪ must not recompute on the old coarse grid."],

```

```

57 "required_next_actions":(["For V02, increase radial resolution or adopt a demonstrably higher-order conservative
↪ radial discretization until the four-decimal field tolerance passes.", "For V06, refine the time step and
↪ locally bracket the Q3/Q4 threshold until the reported 60 s event time is unchanged; otherwise ask the team
↪ whether to reopen H2 validation tolerances.", "Do not start EVIDENCE, publication figures or paper writing
↪ while COMPUTE is FAIL."], if stage_status=="FAIL" else ["Independently verify this handoff schema and every
↪ declared SHA-256 before starting EVIDENCE.", "Stop here; do not create publication figures or paper text in
↪ this task."]),
58 "completed_at":datetime.now(timezone.utc).astimezone().isoformat()
59 (OUT/"handoff.json").write_text(json.dumps(handoff,ensure_ascii=False,indent=2),encoding="utf-8")
60 print(json.dumps({"status":stage_status,"outputs":len(all_outputs),"handoff_sha256":digest(OUT/"handoff.json"),"k
↪ ey_metrics":metrics},ensure_ascii=False,indent=2))
61

```

D.3 程序 3: q3_dense_sensitivity.py

原文件: 04-compute/src/q3_dense_sensitivity.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from __future__ import annotations
2
3 """Q3 dense one-factor sensitivity extension.
4
5 This extension reuses the approved solver in formal_compute.py. It writes only
6 COMPUTE outputs and is intentionally separate from the already-frozen full
7 suite tables so that the extra scan can be audited independently.
8 """
9
10 import hashlib
11 import json
12 import platform
13 import sys
14 from datetime import datetime, timezone
15 from pathlib import Path
16
17 from formal_compute import LOGS, RESULTS, RunConfig, save_csv, simulate
18
19
20 ROOT = Path(__file__).resolve().parents[2]
21 SRC = Path(__file__).resolve()
22 SOLVER = ROOT / "04-compute" / "src" / "formal_compute.py"
23 GUARD_S = 30 * 24 * 3600
24 THRESHOLD = 0.15 - 1e-6
25
26
27 def sha256(path: Path) -> str:
28     h = hashlib.sha256()
29     with path.open("rb") as fh:
30         for block in iter(lambda: fh.read(1024 * 1024), b''):
31             h.update(block)
32     return h.hexdigest()
33
34
35 def run_case(log, label: str, cfg: RunConfig) -> dict:
36     log.write(f"START {label} {datetime.now(timezone.utc).isoformat()}\n")
37     log.flush()
38     run = simulate(cfg)
39     metrics = run["metrics"]
40     row = {
41         "scope": "Q3",
42         "factor": cfg.label.split("-dense-")[1].rsplit("-", 1)[0]
43         if "-dense-" in cfg.label else "terminal_window_s",
44         "level": "",
45         "value": "",
46         "t_star_s": metrics["reported_event_time_s"] or metrics["final_time_s"],
47         "interpolated_t_star_s": metrics["interpolated_event_time_s"] or "",
48         "final_max_C": metrics["final_max_C"],
49         "final_mean_C": metrics["final_mean_C"],
50         "crossed": bool(metrics["reported_event_time_s"] is not None),
51         "guard_s": cfg.duration,
52         "solver_label": cfg.label,
53     }
54     log.write(json.dumps({"label": label, **metrics}, ensure_ascii=False) + "\n")
55     log.flush()

```

```

56     return row
57
58
59 def main() -> None:
60     RESULTS.mkdir(parents=True, exist_ok=True)
61     LOGS.mkdir(parents=True, exist_ok=True)
62     out_csv = RESULTS / "sensitivity-q3-dense.csv"
63     out_manifest = RESULTS / "sensitivity-q3-dense-manifest.json"
64     log_path = LOGS / "q3-dense-sensitivity.log"
65     rows = []
66
67     with log_path.open("w", encoding="utf-8") as log:
68         log.write(f"command={' '.join(sys.argv)}\n")
69         log.write(f"python={sys.version}\nplatform={platform.platform()}\n")
70
71         window_levels = [("30 min", 1800), ("45 min", 2700), ("60 min", 3600),
72                          ("90 min", 5400), ("120 min", 7200)]
73         for level, window in window_levels:
74             cfg = RunConfig(
75                 f"Q3-dense-terminal_window_s-{level}", 10, 20, 60, GUARD_S, 3,
76                 "M3", window_s=window, stop_threshold=THRESHOLD, sample_every=3600,
77             )
78             row = run_case(log, cfg.label, cfg)
79             row.update({"factor": "terminal_window_s", "level": level, "value": window})
80             rows.append(row)
81
82         factor_levels = [("0.8x", 0.8), ("0.9x", 0.9), ("1.0x", 1.0),
83                          ("1.1x", 1.1), ("1.2x", 1.2)]
84         for factor in ("h_mult", "hm_mult", "pref", "exponent"):
85             for level, value in factor_levels:
86                 cfg = RunConfig(
87                     f"Q3-dense-{factor}-{level}", 10, 20, 60, GUARD_S, 3,
88                     "M3", stop_threshold=THRESHOLD, sample_every=3600,
89                     **{factor: value},
90                 )
91                 row = run_case(log, cfg.label, cfg)
92                 row.update({"factor": factor, "level": level, "value": value})
93                 rows.append(row)
94
95     save_csv(out_csv, rows)
96     manifest = {
97         "schema_version": "1.0",
98         "stage": "COMPUTE",
99         "scope": "Q3 dense one-factor sensitivity",
100        "status": "PASS" if all(r["crossed"] for r in rows) else "FAIL",
101        "levels": {
102            "terminal_window_s": [1800, 2700, 3600, 5400, 7200],
103            "multipliers": [0.8, 0.9, 1.0, 1.1, 1.2],
104        },
105        "guard_s": GUARD_S,
106        "threshold": THRESHOLD,
107        "rows": len(rows),
108        "inputs": [
109            {"path": "04-compute/src/formal_compute.py", "sha256": sha256(SOLVER)},
110            {"path": "04-compute/src/q3_dense_sensitivity.py", "sha256": sha256(SRC)},
111        ],
112        "outputs": [
113            {"path": "04-compute/results/sensitivity-q3-dense.csv", "sha256": sha256(out_csv)},
114            {"path": "04-compute/logs/q3-dense-sensitivity.log", "sha256": sha256(log_path)},
115        ],
116        "command": "python 04-compute/src/q3_dense_sensitivity.py",
117        "completed_at": datetime.now(timezone.utc).astimezone().isoformat(),
118    }
119    out_manifest.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
120
121
122 if __name__ == "__main__":
123     main()
124

```

D.4 程序 4: repair_probe.py

原文件: 04-compute/src/repair_probe.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 """Bounded radial-resolution probe for V02/V06 repair."""
2 from __future__ import annotations
3 import argparse, json, sys
4 from pathlib import Path
5
6 sys.path.insert(0, str(Path(__file__).resolve().parent))
7 from formal_compute import RunConfig, simulate
8
9 CASES = [
10     ("Q1-M1", 1, "M1", 0.5, 1800, [81, 122, 183]),
11     ("Q1-M2", 54, "M2", 0.5, 1800, [62, 93, 140]),
12     ("Q2-M3", 36, "M3", 5.0, 10800, [62, 93, 140]),
13     ("Q3-M3", 36, "M3", 60.0, 259200, [62, 93, 140, 210]),
14     ("Q4-M3", 32, "M3", 60.0, 259200, [93, 140, 210]),
15 ]
16
17 ap=argparse.ArgumentParser(); ap.add_argument("--family"); args=ap.parse_args()
18 rows=[]
19 for family,nz,model,dt,duration,nrs in CASES:
20     if args.family and family != args.family:
21         continue
22     q = 1 if family.startswith("Q1") else (2 if family.startswith("Q2") else (3 if family.startswith("Q3") else
23 ↪ 4))
24     for nr in nrs:
25         cfg=RunConfig(f"probe-{family}-nr{nr}",nr,nz,dt,duration,q,model,
26             moving_impl=("reference" if q==4 else "fixed"),
27             stop_threshold=(.15-1e-6 if q in (3,4) else None),sample_every=duration)
28         r=simulate(cfg)
29         row={"family":family,"nr":nr,"nz":nz,"dt_s":dt,
30             "grid_strategy":cfg.radial_strategy,
31             "final_max_C":r["metrics"]["final_max_C"],
32             "final_mean_C":r["metrics"]["final_mean_C"],
33             "event_time_s":r["metrics"]["interpolated_event_time_s"],
34             "reported_event_time_s":r["metrics"]["reported_event_time_s"],
35             "runtime_s":r["metrics"]["runtime_s"]}
36         rows.append(row); print(json.dumps(row,ensure_ascii=False),flush=True)
37
38 out=Path(__file__).resolve().parents[1]/"results"/"repair-probe.json"
39 out.write_text(json.dumps(rows,ensure_ascii=False,indent=2),encoding="utf-8")
```

D.5 程序 5: run_q2_grid_dt5.py

原文件: 04-compute/revision-grid-conv-20260913/run_q2_grid_dt5.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 """Candidate Q2 spatial-grid extension at the frozen formal time step dt=5 s.
2
3 This runs only additional Q2/M3 grid levels and writes an isolated candidate
4 package; it does not overwrite 04-compute/results.
5 """
6 from pathlib import Path
7 import csv
8 import sys
9
10 ROOT = Path(__file__).resolve().parents[2]
11 sys.path.insert(0, str(ROOT / "04-compute" / "src"))
12 import formal_compute as fc # noqa: E402
13
14 OUT = ROOT / "04-compute" / "revision-grid-conv-20260913" / "results" / "q2-space-dt5"
15 OUT.mkdir(parents=True, exist_ok=True)
16
17 rows = []
```

```

18 previous = None
19 for nr in (42, 62, 93, 140, 175, 210):
20     result = fc.simulate(fc.RunConfig(
21         label=f"Q2-M3-nr{nr}-dt5",
22         nr=nr, nz=36, dt=5, duration=10800, q=2, model="M3", sample_every=3600,
23     ))
24     value = float(result["metrics"]["final_max_C"])
25     change = "" if previous is None else abs(value - previous)
26     rows.append({
27         "scope": "Q2", "family": "M3", "level": len(rows) + 1,
28         "nr": nr, "nz": 36, "fixed_dt_s": 5.0,
29         "metric": "final_max_C", "value": value,
30         "adjacent_change": change,
31         "runtime_s": result["metrics"]["runtime_s"],
32     })
33     previous = value
34
35 path = OUT / "field-grid-convergence.csv"
36 with path.open("w", newline="", encoding="utf-8-sig") as f:
37     writer = csv.DictWriter(f, fieldnames=list(rows[0]))
38     writer.writeheader()
39     writer.writerows(rows)
40 print(path)
41

```

D.6 程序 6: run_q2_model_trajectories.py

原文件:

04-compute/revision-grid-conv-20260913/run_q2_model_trajectories.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助；参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from pathlib import Path
2 import csv
3 import sys
4
5 ROOT = Path(__file__).resolve().parents[2]
6 sys.path.insert(0, str(ROOT / "04-compute" / "src"))
7 import formal_compute as fc # noqa: E402
8
9 OUT = ROOT / "04-compute" / "revision-grid-conv-20260913" / "results" / "q2-model-trajectories"
10 OUT.mkdir(parents=True, exist_ok=True)
11
12 configs = [
13     ("M2 常物性", 18, 36, 10.0, "M2"),
14     ("M3 主路线", 140, 36, 5.0, "M3"),
15     ("M4 温度耦合消融", 18, 36, 10.0, "M4"),
16 ]
17 rows = []
18 for label, nr, nz, dt, model in configs:
19     result = fc.simulate(fc.RunConfig(
20         label=f"Q2-{model}-trajectory", nr=nr, nz=nz, dt=dt,
21         duration=10800, q=2, model=model, sample_every=600,
22     ))
23     for rec in result["records"]:
24         rows.append({"model": label, "time_s": rec["time_s"], "max_C": rec["max_C"], "mean_C": rec["mean_C"]})
25
26 path = OUT / "q2-model-trajectories.csv"
27 with path.open("w", newline="", encoding="utf-8-sig") as f:
28     writer = csv.DictWriter(f, fieldnames=list(rows[0]))
29     writer.writeheader(); writer.writerows(rows)
30 print(path)
31

```

D.7 程序 7: run_space_extra.py

原文件: 04-compute/revision-grid-conv-20260913/run_space_extra.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 from pathlib import Path
2 import csv
3 import sys
4
5 ROOT = Path(__file__).resolve().parents[2]
6 sys.path.insert(0, str(ROOT / "04-compute" / "src"))
7 import formal_compute as fc # noqa: E402
8
9 OUT = ROOT / "04-compute" / "revision-grid-conv-20260913" / "results" / "space-extra-dt60"
10 OUT.mkdir(parents=True, exist_ok=True)
11
12 rows = []
13 for scope, nr, nz, q, model, moving_impl in [
14     ("Q3", 63, 36, 3, "M3", "fixed"),
15     ("Q3", 175, 36, 3, "M3", "fixed"),
16     ("Q4", 63, 32, 4, "M3", "reference"),
17     ("Q4", 175, 32, 4, "M3", "reference"),
18 ]:
19     result = fc.simulate(fc.RunConfig(
20         label=f"{scope}-M3-nr{nr}-dt60-extra", nr=nr, nz=nz, dt=60,
21         duration=259200, q=q, model=model, moving_impl=moving_impl,
22         stop_threshold=0.15 - 1e-6, sample_every=60,
23     ))
24     rows.append({
25         "scope": scope, "nr": nr, "nz": nz, "grid_strategy": "two-sided-local",
26         "fixed_dt_s": 60, "interpolated_event_time_s": result["metrics"]["interpolated_event_time_s"],
27         "reported_event_time_s": result["metrics"]["reported_event_time_s"],
28     })
29
30 path = OUT / "threshold-grid-extra.csv"
31 with path.open("w", newline="", encoding="utf-8-sig") as f:
32     writer = csv.DictWriter(f, fieldnames=list(rows[0]))
33     writer.writeheader(); writer.writerows(rows)
34 print(path)
35
```

D.8 程序 8: redraw_fig6.py

原文件: 06-figure/scripts/redraw_fig6.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 from __future__ import annotations
2
3 """Redraw FIG-Q3-THRESHOLD-TRAJECTORY without changing its frozen data.
4
5 The script intentionally reads the existing FIGURE snapshot and independently
6 checks it against the COMPUTE result before rendering. It writes only the
7 figure-6 artwork, previews, and a small QA record; it never rewrites the data
8 snapshot or recomputes the model.
9 """
10
11 import hashlib
12 import json
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17 from PIL import Image, ImageOps
18
19 import matplotlib
20
21 matplotlib.use("Agg")
22 import matplotlib.pyplot as plt
```

```

23 from matplotlib.font_manager import FontProperties, fontManager
24 from matplotlib.lines import Line2D
25
26
27 ROOT = Path(__file__).resolve().parents[2]
28 FIGURE = "FIG-Q3-THRESHOLD-TRAJECTORY"
29 FIG_DIR = ROOT / "06-figure" / "figures"
30 PREVIEW_DIR = ROOT / "06-figure" / "previews"
31 SNAPSHOT = ROOT / "06-figure" / "data-snapshots" / f"{FIGURE}.csv"
32 SOURCE = ROOT / "04-compute" / "results" / "q3-threshold-trajectory.csv"
33 CONTRACT = ROOT / "06-figure" / "contracts" / f"{FIGURE}.json"
34 RECORD_DIR = ROOT / "06-figure" / "redraw-fig6-20260912"
35
36 PNG = FIG_DIR / f"{FIGURE}.png"
37 SVG = FIG_DIR / f"{FIGURE}.svg"
38 PDF = FIG_DIR / f"{FIGURE}.pdf"
39 COLOR_PREVIEW = PREVIEW_DIR / f"{FIGURE}-color.png"
40 GRAY_PREVIEW = PREVIEW_DIR / f"{FIGURE}-grayscale.png"
41
42 THRESHOLD = 0.149999
43 REPORT_TIME_S = 206820.0
44 XMAX_H = 60.0
45 YMAX = 2.7
46
47 FONT_PATH = Path(r"C:\Windows\Fonts\simsun.ttc")
48 if not FONT_PATH.exists():
49     raise FileNotFoundError(f" 宋体字体文件不存在: {FONT_PATH}")
50 fontManager.addfont(str(FONT_PATH))
51 FONT = FontProperties(fname=str(FONT_PATH))
52
53 COLORS = {
54     "max_C": "#0072B2",      # blue
55     "mean_C": "#E69F00",    # orange
56     "center_C": "#009E73",  # green
57     "surface_C": "#D55E00", # vermillion
58 }
59 LABELS = {
60     "max_C": " 最大值",
61     "mean_C": " 均值",
62     "center_C": " 中心",
63     "surface_C": " 表面",
64 }
65 MARKERS = {
66     "max_C": "o",
67     "mean_C": "s",
68     "center_C": "^",
69     "surface_C": "D",
70 }
71
72
73 def sha256(path: Path) -> str:
74     h = hashlib.sha256()
75     with path.open("rb") as f:
76         for block in iter(lambda: f.read(1024 * 1024), b''):
77             h.update(block)
78     return h.hexdigest()
79
80
81 def check_data() -> tuple[pd.DataFrame, dict]:
82     contract = json.loads(CONTRACT.read_text(encoding="utf-8"))
83     source_expected = contract["source_files"][0]["sha256"]
84     snapshot_expected = contract["snapshot_sha256"]
85     source_actual = sha256(SOURCE)
86     snapshot_actual = sha256(SNAPSHOT)
87     if source_actual != source_expected:
88         raise RuntimeError(
89             f" 原始结果哈希不匹配: expected={source_expected}, actual={source_actual}"
90         )
91     if snapshot_actual != snapshot_expected:
92         raise RuntimeError(
93             f" 冻结快照哈希不匹配: expected={snapshot_expected}, actual={snapshot_actual}"
94         )
95
96     source_df = pd.read_csv(SOURCE)
97     snapshot_df = pd.read_csv(SNAPSHOT)
98     expected_columns = ["time_s", "max_C", "mean_C", "center_C", "surface_C"]
99     if not set(expected_columns).issubset(source_df.columns) or list(snapshot_df.columns) != expected_columns:
100         raise RuntimeError(" 源数据或冻结快照列结构发生变化")
101     if len(source_df) != len(snapshot_df):

```

```

102         raise RuntimeError(" 源数据与冻结快照行数不一致")
103     source_values = source_df[expected_columns].to_numpy(dtype=float)
104     snapshot_values = snapshot_df.to_numpy(dtype=float)
105     max_abs_diff = float(np.max(np.abs(source_values - snapshot_values)))
106     # The snapshot is a CSV projection of the source; decimal parsing can
107     # differ by one floating-point ulp while preserving every plotted value.
108     if not np.allclose(source_values, snapshot_values, rtol=0.0, atol=5e-15):
109         raise RuntimeError(f" 源数据与冻结快照数值不一致, 最大绝对差 = {max_abs_diff}")
110
111     checks = {
112         "source_path": SOURCE.relative_to(ROOT).as_posix(),
113         "source_sha256": source_actual,
114         "snapshot_path": SNAPSHOT.relative_to(ROOT).as_posix(),
115         "snapshot_sha256": snapshot_actual,
116         "columns": expected_columns,
117         "rows": int(len(snapshot_df)),
118         "data_equal": True,
119         "max_abs_diff_after_csv_parse": max_abs_diff,
120         "time_range_s": [float(snapshot_df.time_s.min()), float(snapshot_df.time_s.max())],
121         "reported_time_s": REPORT_TIME_S,
122         "threshold": THRESHOLD,
123     }
124     return snapshot_df, checks
125
126
127 def configure_text() -> None:
128     plt.rcParams.update(
129         {
130             "font.family": "SimSun",
131             "font.sans-serif": ["SimSun"],
132             "axes.unicode_minus": False,
133             "svg.fonttype": "none",
134             "pdf.fonttype": 42,
135             "ps.fonttype": 42,
136         }
137     )
138
139
140 def style_axes(ax, *, inset: bool = False) -> None:
141     ax.set_facecolor("white")
142     ax.tick_params(
143         axis="both",
144         which="both",
145         colors="black",
146         labelcolor="black",
147         direction="out",
148         length=3.0 if inset else 4.0,
149         width=0.7,
150         labelsiz=6.5 if inset else 8.0,
151     )
152     for label in ax.get_xticklabels() + ax.get_yticklabels():
153         label.set_fontproperties(FONT)
154         label.set_color("black")
155     for spine in ax.spines.values():
156         spine.set_color("black")
157         spine.set_linewidth(0.7)
158     ax.grid(True, color="black", alpha=0.14, linewidth=0.45)
159     ax.set_axisbelow(True)
160
161
162 def draw_series(ax, data: pd.DataFrame, *, marker_indices: np.ndarray, linewidth: float, markersize: float) ->
163     ↪ None:
164     time_h = data.time_s.to_numpy(dtype=float) / 3600.0
165     for column in ("max_C", "mean_C", "center_C", "surface_C"):
166         ax.plot(
167             time_h,
168             data[column].to_numpy(dtype=float),
169             color=COLORS[column],
170             linestyle="-",
171             linewidth=linewidth,
172             marker=MARKERS[column],
173             markersize=markersize,
174             markevery=marker_indices,
175             markerfacecolor=COLORS[column],
176             markeredgewidth=0.45,
177             label=LABELS[column],
178             zorder=3,
179         )

```

```

180
181
182 def render(data: pd.DataFrame) -> None:
183     configure_text()
184     FIG_DIR.mkdir(parents=True, exist_ok=True)
185     PREVIEW_DIR.mkdir(parents=True, exist_ok=True)
186     RECORD_DIR.mkdir(parents=True, exist_ok=True)
187
188     # Markers are a visual encoding only; every row remains in every line.
189     marker_hours = np.array([0, 2, 4, 8, 12, 24, 36, 48, REPORT_TIME_S / 3600.0])
190     time_hours = data.time_s.to_numpy(dtype=float) / 3600.0
191     marker_indices = np.array([int(np.argmin(np.abs(time_hours - h))) for h in marker_hours])
192     marker_indices = np.unique(marker_indices)
193
194     fig, ax = plt.subplots(figsize=(6.535, 4.75), dpi=600)
195     fig.patch.set_facecolor("white")
196     draw_series(ax, data, marker_indices=marker_indices, linewidth=1.45, markersize=3.7)
197     ax.axhline(
198         THRESHOLD,
199         color="black",
200         linestyle="--",
201         linewidth=0.9,
202         label=" 判据 0.149999",
203         zorder=2,
204     )
205     ax.axvline(
206         REPORT_TIME_S / 3600.0,
207         color="black",
208         linestyle=":",
209         linewidth=0.9,
210         label=" 报告时刻 57.45 h",
211         zorder=2,
212     )
213     ax.set_xlim(0, XMAX_H)
214     ax.set_ylim(0, YMAX)
215     ax.set_title(
216         " 全域含水率随时间的变化",
217         fontproperties=FONT,
218         fontsize=10.5,
219         color="black",
220         pad=8,
221     )
222     ax.set_xlabel(" 时间 t (h)", fontproperties=FONT, fontsize=9, color="black", labelpad=5)
223     ax.set_ylabel(" 干基含水率 C (kg/kg)", fontproperties=FONT, fontsize=9, color="black", labelpad=5)
224     style_axes(ax)
225
226     handles = [
227         Line2D(
228             [0], [0], color=COLORS[c], linestyle="--", linewidth=1.45,
229             marker=MARKERS[c], markersize=4.2, markerfacecolor=COLORS[c],
230             markeredgewidth=0.45, label=LABELS[c],
231         )
232         for c in ("max_C", "mean_C", "center_C", "surface_C")
233     ]
234     handles += [
235         Line2D([0], [0], color="black", linestyle="--", linewidth=0.9, label=" 判据 0.149999"),
236         Line2D([0], [0], color="black", linestyle=":", linewidth=0.9, label=" 报告时刻 57.45 h"),
237     ]
238     legend = ax.legend(
239         handles=handles,
240         loc="upper right",
241         bbox_to_anchor=(0.99, 0.93),
242         ncol=3,
243         frameon=False,
244         borderaxespad=0.25,
245         handlelength=2.3,
246         columnspacing=1.0,
247         handletextpad=0.45,
248         prop=FontProperties(fname=str(FONT_PATH), size=7.5),
249     )
250     for text in legend.get_texts():
251         text.set_color("black")
252
253     # A compact, explicitly labelled local view makes the threshold event legible
254     # without changing the primary 0-60 h coordinate range.
255     inset = ax.inset_axes([0.56, 0.16, 0.39, 0.38])
256     draw_series(inset, data, marker_indices=marker_indices, linewidth=1.0, markersize=2.8)
257     inset.axhline(THRESHOLD, color="black", linestyle="--", linewidth=0.7, zorder=2)
258     inset.axvline(REPORT_TIME_S / 3600.0, color="black", linestyle=":", linewidth=0.7, zorder=2)

```

```

259     inset.set_xlim(45, 60)
260     inset.set_ylim(0.04, 0.17)
261     inset.set_title(" 判据附近局部放大", fontproperties=FONT, fontsize=7.5, color="black", pad=3)
262     inset.set_xlabel("t (h)", fontproperties=FONT, fontsize=6.5, color="black", labelpad=2)
263     inset.set_ylabel("C", fontproperties=FONT, fontsize=6.5, color="black", labelpad=2)
264     style_axes(inset, inset=True)
265
266     fig.subplots_adjust(left=0.105, right=0.985, bottom=0.12, top=0.91)
267     for path, kwargs in (
268         (SVG, {"format": "svg"}),
269         (PDF, {"format": "pdf"}),
270         (PNG, {"format": "png", "dpi": 600}),
271         (COLOR_PREVIEW, {"format": "png", "dpi": 300}),
272     ):
273         fig.savefig(path, facecolor="white", **kwargs)
274     plt.close(fig)
275
276     ImageOps.grayscale(Image.open(COLOR_PREVIEW).convert("RGB")).save(
277         GRAY_PREVIEW, dpi=(300, 300)
278     )
279
280
281 def write_record(checks: dict) -> None:
282     record = {
283         "figure_id": FIGURE,
284         "visual_revision": "redraw-fig6-20260912-title-simsun",
285         "data_check": checks,
286         "visual_contract": {
287             "title": " 全域含水率随时间的变化（黑色宋体）",
288             "data_curves": " 高对比彩色、统一实线、marker 冗余编码",
289             "non_data_elements": " 坐标轴、网格、刻度、轴标签、图例和标注统一黑色",
290             "font": "SimSun / 宋体",
291             "primary_xlim_h": [0, XMAX_H],
292             "primary_ylim": [0, YMAX],
293             "threshold": THRESHOLD,
294             "reported_time_s": REPORT_TIME_S,
295             "note_policy": " 删除图内无关备注：限制条件保留在正式图注/正文",
296         },
297         "outputs": {
298             "png_600dpi": PNG.relative_to(ROOT).as_posix(),
299             "svg": SVG.relative_to(ROOT).as_posix(),
300             "pdf": PDF.relative_to(ROOT).as_posix(),
301             "color_preview": COLOR_PREVIEW.relative_to(ROOT).as_posix(),
302             "grayscale_preview": GRAY_PREVIEW.relative_to(ROOT).as_posix(),
303         },
304     }
305     for key, path in (
306         ("png_sha256", PNG),
307         ("svg_sha256", SVG),
308         ("pdf_sha256", PDF),
309         ("color_preview_sha256", COLOR_PREVIEW),
310         ("grayscale_preview_sha256", GRAY_PREVIEW),
311     ):
312         record["outputs"][key] = sha256(path)
313     (RECORD_DIR / "qa-report.json").write_text(
314         json.dumps(record, ensure_ascii=False, indent=2), encoding="utf-8"
315     )
316
317
318 def main() -> None:
319     data, checks = check_data()
320     render(data)
321     write_record(checks)
322     print(json.dumps({
323         "figure": FIGURE,
324         "rows": checks["rows"],
325         "data_equal": checks["data_equal"],
326         "png_sha256": sha256(PNG),
327         "svg_sha256": sha256(SVG),
328         "pdf_sha256": sha256(PDF),
329         "color_preview_sha256": sha256(COLOR_PREVIEW),
330         "grayscale_preview_sha256": sha256(GRAY_PREVIEW),
331     }, ensure_ascii=False))
332
333
334 if __name__ == "__main__":
335     main()
336

```

D.9 程序 9: redraw_fig7.py

原文件: 06-figure/scripts/redraw_fig7.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 """Redraw FIG-Q3-BRACKET-ZOOM without changing its frozen data or claim."""
2
3 from __future__ import annotations
4
5 import hashlib
6 import json
7 from datetime import datetime, timezone
8 from pathlib import Path
9
10 import matplotlib as mpl
11 import matplotlib.pyplot as plt
12 import numpy as np
13 import pandas as pd
14 from matplotlib.lines import Line2D
15 from matplotlib.ticker import FormatStrFormatter
16 from PIL import Image, ImageOps
17
18
19 ROOT = Path(__file__).resolve().parents[2]
20 FIGURE_DIR = ROOT / "06-figure"
21 SOURCE = ROOT / "04-compute" / "results" / "q3-threshold-trajectory.csv"
22 RUN_SUMMARY = ROOT / "04-compute" / "results" / "run-summary.csv"
23 SNAPSHOT = FIGURE_DIR / "data-snapshots" / "FIG-Q3-BRACKET-ZOOM.csv"
24 CONTRACT = FIGURE_DIR / "contracts" / "FIG-Q3-BRACKET-ZOOM.json"
25 FIGURES = FIGURE_DIR / "figures"
26 PREVIEWS = FIGURE_DIR / "previews"
27 FIG_ID = "FIG-Q3-BRACKET-ZOOM"
28 THRESHOLD = 0.149999 # frozen Q3 full-domain threshold: 0.15 - 1e-6
29
30
31 def sha256(path: Path) -> str:
32     return hashlib.sha256(path.read_bytes()).hexdigest()
33
34
35 def validate_frozen_data() -> pd.DataFrame:
36     """Use the existing snapshot as plotting input and verify it against source rows."""
37     snapshot = pd.read_csv(SNAPSHOT)
38     expected_columns = ["time_s", "max_C"]
39     if list(snapshot.columns) != expected_columns:
40         raise AssertionError(f"unexpected snapshot columns: {list(snapshot.columns)}")
41     if len(snapshot) != 6:
42         raise AssertionError(f"FIG-Q3-BRACKET-ZOOM must retain 6 samples, got {len(snapshot)}")
43
44     source = pd.read_csv(SOURCE)
45     for row in snapshot.itertuples(index=False):
46         candidates = source[source["time_s"].eq(row.time_s)]
47         if candidates.empty or not np.isclose(
48             candidates["max_C"].to_numpy(dtype=float),
49             float(row.max_C),
50             rtol=0.0,
51             atol=1e-15,
52         ).any():
53             raise AssertionError(f"snapshot row is not present in source: {row}")
54     return snapshot
55
56
57 def read_frozen_event_times() -> tuple[float, float]:
58     summary = pd.read_csv(RUN_SUMMARY)
59     row = summary.loc[summary["label"].eq("Q3-M3-nr210")]
60     if len(row) != 1:
61         raise AssertionError("expected exactly one Q3-M3-nr210 row in run-summary.csv")
62     interpolated = float(row.iloc[0]["interpolated_event_time_s"])
63     reported = float(row.iloc[0]["reported_event_time_s"])
64     if not np.isclose(interpolated, 206818.71137262788, rtol=0.0, atol=1e-12):
65         raise AssertionError("unexpected frozen interpolated event time")
66     if not np.isclose(reported, 206820.0, rtol=0.0, atol=1e-12):
67         raise AssertionError("unexpected frozen reported event time")
68     return interpolated, reported
69
70
```

```

71 def main() -> None:
72     FIGURES.mkdir(parents=True, exist_ok=True)
73     PREVIEWS.mkdir(parents=True, exist_ok=True)
74     data = validate_frozen_data()
75     interpolated, reported = read_frozen_event_times()
76
77     mpl.rcParams.update(
78         {
79             "font.family": "SimSun",
80             "font.sans-serif": ["SimSun", "STSong", "FangSong", "Microsoft YaHei", "Arial"],
81             "font.serif": ["SimSun", "STSong", "FangSong", "Microsoft YaHei", "Arial"],
82             "font.size": 8.5,
83             "text.color": "#000000",
84             "axes.labelcolor": "#000000",
85             "axes.titlecolor": "#000000",
86             "xtick.color": "#000000",
87             "ytick.color": "#000000",
88             "legend.labelcolor": "#000000",
89             "axes.unicode_minus": False,
90             "axes.facecolor": "#FFFFFF",
91             "figure.facecolor": "#FFFFFF",
92             "savefig.facecolor": "#FFFFFF",
93             "axes.axisbelow": True,
94             "pdf.fonttype": 42,
95             "ps.fonttype": 42,
96             "svg.fonttype": "none",
97             "svg.hashsalt": FIG_ID,
98         }
99     )
100
101     # The curve is one data series: keep it solid. Point colors and markers
102     # identify the six retained sampling times and survive grayscale printing.
103     point_colors = ["#0072B2", "#E69F00", "#009E73", "#D55E00", "#7B2CBF", "#007C91"]
104     point_markers = ["o", "s", "^", "D", "p", "X"]
105
106     fig, ax = plt.subplots(figsize=(6.7, 4.2), dpi=600)
107     fig.subplots_adjust(left=0.135, right=0.985, bottom=0.19, top=0.80)
108
109     ax.plot(
110         data["time_s"],
111         data["max_C"],
112         color="#005AB5",
113         linestyle="-",
114         linewidth=1.7,
115         zorder=2,
116     )
117     for color, marker, row in zip(point_colors, point_markers, data.itertuples(index=False)):
118         ax.plot(
119             row.time_s,
120             row.max_C,
121             linestyle="None",
122             marker=marker,
123             markersize=5.5,
124             markerfacecolor=color,
125             markeredgecolor="#000000",
126             markeredgewidth=0.55,
127             zorder=3,
128         )
129
130     # Reference lines are not data curves; their black redundant line styles
131     # make the threshold, interpolated event, and conservative report distinct.
132     ax.axhline(
133         THRESHOLD,
134         color="#000000",
135         linewidth=0.85,
136         linestyle=(0, (5, 2.5)),
137         zorder=1,
138     )
139     ax.axvline(
140         interpolated,
141         color="#000000",
142         linewidth=0.85,
143         linestyle=(0, (1.5, 2)),
144         zorder=1,
145     )
146     ax.axvline(
147         reported,
148         color="#000000",
149         linewidth=0.95,

```

```

150         linestyle=(0, (6, 2)),
151         zorder=1,
152     )
153
154     ax.set_xlabel(" 时间 t (s)", fontsize=9.5, color="#000000", labelpad=7)
155     ax.set_ylabel(" 最大干基含水率 C (kg/kg)", fontsize=9.5, color="#000000", labelpad=8)
156     ax.set_xticks(data["time_s"].tolist())
157     ax.set_xticklabels([str(int(x)) for x in data["time_s"]])
158     ax.yaxis.set_major_formatter(FormatStrFormatter("%.6f"))
159     ax.tick_params(
160         axis="both",
161         which="major",
162         direction="out",
163         length=3.2,
164         width=0.65,
165         colors="#000000",
166         labelsize=7.4,
167     )
168     ax.grid(axis="both", color="#000000", linewidth=0.45, alpha=0.14)
169     ax.spines["top"].set_visible(False)
170     ax.spines["right"].set_visible(False)
171     for side in ("left", "bottom"):
172         ax.spines[side].set_color("#000000")
173         ax.spines[side].set_linewidth(0.75)
174
175     legend_handles = [
176         Line2D([0], [0], color="#000000", linewidth=0.85, linestyle=(0, (5, 2.5)), label=" 阈值 C = 0.149999"),
177         Line2D([0], [0], color="#000000", linewidth=0.85, linestyle=(0, (1.5, 2)), label=f"插值时刻
178         ↪ {interpolated:.4f} s"),
179         Line2D([0], [0], color="#000000", linewidth=0.95, linestyle=(0, (6, 2)), label=f"保守报告 {reported:.0f}
180         ↪ s"),
181     ]
182
183     legend = fig.legend(
184         handles=legend_handles,
185         loc="upper center",
186         bbox_to_anchor=(0.57, 0.895),
187         ncol=3,
188         frameon=False,
189         fontsize=7.5,
190         columnspacing=1.35,
191         handlelength=2.2,
192         handletextpad=0.45,
193         borderaxespad=0.0,
194     )
195
196     for text in legend.get_texts():
197         text.set_color("#000000")
198
199     fixed_date = datetime(2026, 1, 1, tzinfo=timezone.utc)
200     fig.savefig(FIGURES / f"{FIG_ID}.svg", metadata={"Date": "2026-01-01T00:00:00"})
201     fig.savefig(
202         FIGURES / f"{FIG_ID}.pdf",
203         dpi=600,
204         metadata={"CreationDate": fixed_date, "ModDate": fixed_date},
205     )
206     fig.savefig(FIGURES / f"{FIG_ID}.png", dpi=600)
207     fig.savefig(PREVIEWES / f"{FIG_ID}-color.png", dpi=600)
208     plt.close(fig)
209
210     formal_color = Image.open(FIGURES / f"{FIG_ID}.png").convert("RGB")
211     ImageOps.grayscale(formal_color).save(PREVIEWES / f"{FIG_ID}-grayscale.png", dpi=(600, 600))
212
213     contract = json.loads(CONTRACT.read_text(encoding="utf-8"))
214     contract["target_formats"] = ["svg", "png", "pdf"]
215     CONTRACT.write_text(json.dumps(contract, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
216
217     print(
218         json.dumps(
219             {
220                 "figure_id": FIG_ID,
221                 "source_sha256": sha256(SOURCE),
222                 "snapshot_sha256": sha256(SNAPSHOT),
223                 "png_sha256": sha256(FIGURES / f"{FIG_ID}.png"),
224                 "svg_sha256": sha256(FIGURES / f"{FIG_ID}.svg"),
225                 "pdf_sha256": sha256(FIGURES / f"{FIG_ID}.pdf"),
226                 "sample_count": len(data),
227                 "font": "SimSun",
228                 "data_claim_status": "unchanged",
229             },
230             ensure_ascii=False,

```

```

227     )
228 )
229
230
231 if __name__ == "__main__":
232     main()
233

```

D.10 程序 10: redraw_fig9_onward.py

原文件: 06-figure/scripts/redraw_fig9_onward.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from __future__ import annotations
2
3 """ 重绘正式图 9--22。
4
5 本脚本只读取 06-figure/data-snapshots 下已经冻结的快照; 源文件只用于
6 SHA-256 核对, 不重新运行模型、不重算指标、不修改快照或上游决策。
7 正式输出仍写入 06-figure/figures, 旧版文件先复制到本轮 revision 目录留存。
8 """
9
10 import hashlib
11 import json
12 import shutil
13 import subprocess
14 import sys
15 from datetime import datetime, timezone, timedelta
16 from pathlib import Path
17
18 import matplotlib as mpl
19 import matplotlib.pyplot as plt
20 from matplotlib.font_manager import FontProperties
21 from matplotlib.lines import Line2D
22 from matplotlib.ticker import FuncFormatter
23 import numpy as np
24 import pandas as pd
25 from PIL import Image, ImageOps
26
27
28 ROOT = Path(__file__).resolve().parents[2]
29 STAGE = ROOT / "06-figure"
30 SNAP = STAGE / "data-snapshots"
31 CONTRACTS = STAGE / "contracts"
32 FORMAL = STAGE / "figures"
33 PREVIEWS = STAGE / "previews"
34 REV = STAGE / "redraw-fig9-onward-20260912"
35 REV_FIGURES = REV / "figures"
36 REV_PREVIEWS = REV / "previews"
37 REV_QA = REV / "qa"
38 REV_SUPERSEDED = REV / "superseded"
39
40 TARGET_IDS = [
41     "FIG-Q3-SENS-ONEFACTOR",
42     "FIG-Q3-COMBINED-BOUNDARY",
43     "FIG-Q4-RADIUS-TIME",
44     "FIG-Q4-THRESHOLD-TRAJECTORY",
45     "FIG-Q4-JACOBIAN-ABLATION",
46     "FIG-Q4-COMBINED-BOUNDARY",
47     "FIG-Q1-GRID-CONV",
48     "FIG-Q2-V02-MARGIN",
49     "FIG-Q3-SPACE-CONV",
50     "FIG-Q4-BRACKET-ZOOM",
51     "FIG-Q4-SPACE-CONV",
52 ]
53
54 BLACK = "#000000"
55 COLORS = ["#0072B2", "#E69F00", "#009E73", "#CC79A7", "#56B4E9", "#D55E00"]
56 MARKERS = ["o", "s", "^", "D", "P", "X"]
57 FONT_PATH = Path(r"C:\Windows\Fonts\simsun.ttc")
58 THRESHOLD = 0.149999
59 SONG: FontProperties
60

```

```

61
62 def sha256(path: Path) -> str:
63     h = hashlib.sha256()
64     with path.open("rb") as fh:
65         for block in iter(lambda: fh.read(1024 * 1024), b''):
66             h.update(block)
67     return h.hexdigest()
68
69
70 def rel(path: Path) -> str:
71     return path.relative_to(ROOT).as_posix()
72
73
74 def require(condition: bool, message: str) -> None:
75     if not condition:
76         raise RuntimeError(message)
77
78
79 def configure_style() -> None:
80     global SONG
81     require(FONT_PATH.exists(), f" 宋体文件不存在: {FONT_PATH}")
82     SONG = FontProperties(fname=str(FONT_PATH))
83     mpl.rcParams.update(
84         {
85             "font.family": ["SimSun"],
86             "font.sans-serif": ["SimSun", "STSong", "Microsoft YaHei", "Arial"],
87             "font.size": 8.5,
88             "axes.labelsize": 9.0,
89             "axes.titlesize": 10.0,
90             "axes.titleweight": "normal",
91             "xtick.labelsize": 8.0,
92             "ytick.labelsize": 8.0,
93             "legend.fontsize": 8.0,
94             "text.color": BLACK,
95             "axes.labelcolor": BLACK,
96             "axes.titlecolor": BLACK,
97             "xtick.color": BLACK,
98             "ytick.color": BLACK,
99             "axes.facecolor": "#FFFFFF",
100            "figure.facecolor": "#FFFFFF",
101            "savefig.facecolor": "#FFFFFF",
102            "axes.unicode_minus": False,
103            "axes.axisbelow": True,
104            "pdf.fonttype": 42,
105            "ps.fonttype": 42,
106            "svg.fonttype": "none",
107            # 本脚本为图 9 的长标签、图 16 的运行序号和多面板图显式分配边距；
108            # 禁用全局 constrained layout，避免 subplots_adjust 被静默忽略。
109            "figure.constrained_layout.use": False,
110        }
111    )
112
113
114 def read_snapshot(fid: str) -> tuple[pd.DataFrame, dict]:
115     csv_path = SNAP / f"{fid}.csv"
116     meta_path = SNAP / f"{fid}.meta.json"
117     contract_path = CONTRACTS / f"{fid}.json"
118     require(csv_path.exists(), f" 缺少冻结快照: {csv_path}")
119     require(meta_path.exists(), f" 缺少快照元数据: {meta_path}")
120     require(contract_path.exists(), f" 缺少图表契约: {contract_path}")
121     meta = json.loads(meta_path.read_text(encoding="utf-8"))
122     contract = json.loads(contract_path.read_text(encoding="utf-8"))
123     require(contract.get("figure_id") == fid, f" 契约 ID 不匹配: {fid}")
124     require(sha256(csv_path) == meta.get("snapshot_sha256"), f" 快照哈希不匹配: {fid}")
125     source_checks = []
126     for entry in meta.get("sources", []):
127         source = ROOT / entry["path"]
128         require(source.exists(), f" 源文件不存在: {source}")
129         actual = sha256(source)
130         source_checks.append(
131             {
132                 "path": entry["path"],
133                 "expected_sha256": entry["sha256"],
134                 "actual_sha256": actual,
135                 "status": "PASS" if actual == entry["sha256"] else "FAIL",
136             }
137         )
138     require(actual == entry["sha256"], f" 源数据哈希不匹配: {entry['path']}")
139     return pd.read_csv(csv_path), {

```

```

140         "figure_id": fid,
141         "snapshot": rel(csv_path),
142         "snapshot_sha256": sha256(csv_path),
143         "rows": int(len(pd.read_csv(csv_path))),
144         "columns": list(pd.read_csv(csv_path).columns),
145         "sources": source_checks,
146         "contract_sha256": sha256(contract_path),
147     }
148
149
150 def text_font(obj) -> None:
151     """ 统一所有可见文字为宋体和黑色; 数据颜色只留给线和 marker. """
152     for txt in obj.findobj(mpl.text.Text):
153         if txt.get_text().strip():
154             txt.set_fontproperties(SONG)
155             txt.set_color(BLACK)
156
157
158 def style_axis(ax, xlabel: str, ylabel: str, title: str | None = None,
159               *, rotate: float = 0.0, grid: bool = True) -> None:
160     ax.set_xlabel(xlabel, fontproperties=SONG, color=BLACK, labelpad=7)
161     ax.set_ylabel(ylabel, fontproperties=SONG, color=BLACK, labelpad=7)
162     if title:
163         ax.set_title(title, loc="left", pad=9, fontproperties=SONG, color=BLACK)
164     ax.tick_params(
165         axis="both", which="major", direction="in", top=True, right=True,
166         length=3.5, width=0.75, colors=BLACK, labelsiz=8.0,
167     )
168     for spine in ax.spines.values():
169         spine.set_visible(True)
170         spine.set_color(BLACK)
171         spine.set_linewidth(0.75)
172     if grid:
173         ax.grid(True, which="major", axis="both", color=BLACK, alpha=0.14, linewidth=0.55)
174     else:
175         ax.grid(False)
176     for labels in (ax.get_xticklabels(), ax.get_yticklabels()):
177         for label in labels:
178             label.set_fontproperties(SONG)
179             label.set_color(BLACK)
180             if rotate:
181                 label.set_rotation(rotate)
182                 label.set_ha("right")
183
184
185 def legend_black(ax, **kwargs):
186     kwargs.setdefault("frameon", False)
187     kwargs.setdefault("prop", SONG)
188     legend = ax.legend(**kwargs)
189     if legend is not None:
190         for txt in legend.get_texts():
191             txt.set_fontproperties(SONG)
192             txt.set_color(BLACK)
193     return legend
194
195
196 def make_record(fid: str, meta: dict, fig: plt.Figure, qa_issues: list) -> dict:
197     return {
198         "figure_id": fid,
199         "snapshot": meta["snapshot"],
200         "snapshot_sha256": meta["snapshot_sha256"],
201         "source_checks": meta["sources"],
202         "rows": meta["rows"],
203         "columns": meta["columns"],
204         "figure_size_inches": [round(float(x), 4) for x in fig.get_size_inches()],
205         "visual_qa": {
206             "status": "PASS" if not any(s == "FAIL" for s, _ in qa_issues) else "FAIL",
207             "issues": [{ "severity": s, "message": m } for s, m in qa_issues],
208         },
209     }
210
211
212 def save_figure(fid: str, fig: plt.Figure, meta: dict, records: dict) -> None:
213     text_font(fig)
214     # 先在 Figure 对象上做程序版版式检查, 再关闭对象。
215     from visual_qa import audit_layout
216
217     qa_issues = audit_layout(fig)
218     records[fid] = make_record(fid, meta, fig, qa_issues)

```

```

219     for ext in ("svg", "pdf", "png"):
220         old = FORMAL / f"{fid}.{ext}"
221         backup = REV_SUPERSEDED / f"{fid}.pre-redraw.{ext}"
222         if old.exists() and not backup.exists():
223             shutil.copy2(old, backup)
224     fig_targets = {
225         "svg": REV_FIGURES / f"{fid}.svg",
226         "pdf": REV_FIGURES / f"{fid}.pdf",
227         "png": REV_FIGURES / f"{fid}.png",
228     }
229     preview_targets = {
230         "color": REV_PREVIEWS / f"{fid}-color.png",
231         "grayscale": REV_PREVIEWS / f"{fid}-grayscale.png",
232     }
233     for target in fig_targets.values():
234         target.parent.mkdir(parents=True, exist_ok=True)
235     save_kwargs = {"bbox_inches": "tight", "pad_inches": 0.05, "facecolor": "white"}
236     fig.savefig(fig_targets["svg"], **save_kwargs)
237     fig.savefig(fig_targets["pdf"], metadata={"Title": fid}, **save_kwargs)
238     fig.savefig(fig_targets["png"], dpi=600, **save_kwargs)
239     fig.savefig(preview_targets["color"], dpi=180, **save_kwargs)
240     ImageOps.grayscale(Image.open(preview_targets["color"]).convert("RGB")).save(
241         preview_targets["grayscale"], dpi=(180, 180)
242     )
243     # 正式目录保留本轮版本; 旧版已在 superseded/ 中可恢复。
244     for ext, target in fig_targets.items():
245         shutil.copy2(target, FORMAL / target.name)
246     for target in preview_targets.values():
247         shutil.copy2(target, PREVIEWS / target.name)
248     plt.close(fig)
249
250
251 def read(fid: str) -> tuple[pd.DataFrame, dict]:
252     return read_snapshot(fid)
253
254
255 def plain_log_tick(value: float, _position: int) -> str:
256     """Return an ASCII log tick so PDF/SVG text stays on the SimSun path."""
257     if value <= 0:
258         return ""
259     exponent = int(round(np.log10(value)))
260     return f"10~{exponent}"
261
262
263 def plot_q3_sensitivity(records: dict) -> None:
264     fid = "FIG-Q3-SENS-ONEFACTOR"
265     d, meta = read(fid)
266     factor_specs = [
267         ("terminal_window_s", "窗口", COLORS[0], (59.68, 59.86), [59.70, 59.75, 59.80, 59.85]),
268         ("h_mult", "h", COLORS[1], (59.70, 59.80), [59.70, 59.75, 59.80]),
269         ("hm_mult", "hm", COLORS[2], (58.5, 61.5), [59.0, 60.0, 61.0]),
270         ("pref", "扩散系数前因子", COLORS[3], (49.0, 75.0), [50.0, 60.0, 70.0, 75.0]),
271         ("exponent", "扩散经验指数", COLORS[4], (35.0, 105.0), [40.0, 60.0, 80.0, 100.0]),
272     ]
273     factor_labels = {
274         "terminal_window_s": lambda row: f"{float(row['value']) / 60:g} min",
275         "h_mult": lambda row: f"{float(row['value']):.1f}x",
276         "hm_mult": lambda row: f"{float(row['value']):.1f}x",
277         "pref": lambda row: f"{float(row['value']):.1f}x",
278         "exponent": lambda row: f"{float(row['value']):.1f}x",
279     }
280
281     # 采用 small multiples: 每个因素使用独立纵轴, 避免不同尺度的达标时间
282     # 被放在同一坐标轴后压扁。图中使用延长计算得到的实际达标时刻。
283     fig, axes = plt.subplots(2, 3, figsize=(10.8, 7.0), squeeze=False)
284     axes = axes.ravel()
285     panel_letters = ["(a)", "(b)", "(c)", "(d)", "(e)"]
286     for panel_i, (factor, title, color, ylim, yticks) in enumerate(factor_specs):
287         ax = axes[panel_i]
288         sub = d.loc[d["factor"].eq(factor)].copy()
289         if factor != "terminal_window_s":
290             sub = sub.sort_values("level", key=lambda s: s.map({"low": 0, "high": 1}))
291         x = np.arange(len(sub), dtype=float)
292         y = sub["display_h"].to_numpy(float)
293         ax.plot(x, y, color=color, linewidth=1.8, linestyle="-", zorder=2)
294         for point_i, (_, row) in enumerate(sub.iterrows()):
295             marker = "o"
296             ax.scatter(
297                 [point_i], [float(row["display_h"])], s=55,

```

```

298         color=color, marker=marker, edgecolors=BLACK,
299         linewidths=0.7, zorder=3,
300     )
301     # 数值标注让窄范围面板的差异可以直接读取; h 面板的重合也保持如实。
302     value = float(row["display_h"])
303     # 高位点的标注放在点下方, 避免与独立纵轴的上边界刻度重叠。
304     offset_y = -14 if value > ylim[0] + 0.78 * (ylim[1] - ylim[0]) else (7 if point_i % 2 == 0 else -14)
305     ax.annotate(
306         f"{value:.2f}",
307         (point_i, value),
308         xytext=(0, offset_y), textcoords="offset points",
309         ha="center", va="center", fontproperties=SONG,
310         fontsize=7.4, color=BLACK,
311     )
312     ax.set_xticks(x, [factor_labels[factor](row) for _, row in sub.iterrows()])
313     ax.set_ylim(*ylim)
314     ax.set_yticks(yticks)
315     style_axis(ax, " 扰动情景", " 达标时间 t* (h)", f"{panel_letters[panel_i]} {title}")
316     for label in ax.get_xticklabels():
317         label.set_rotation(25)
318         label.set_fontsize(7.5)
319         label.set_ha("right")
320
321 axes[-1].axis("off")
322 fig.suptitle("Q3 单因素扰动的达标时间", y=0.995, fontproperties=SONG, fontsize=12, color=BLACK)
323 fig.text(
324     0.5, 0.015, " 各小图使用独立纵轴; 所有点为正式求解得到的实际首次达标时刻; 条件仿真, 非概率区间。",
325     ha="center", va="bottom", fontproperties=SONG, fontsize=8.2, color=BLACK,
326 )
327 fig.subplots_adjust(left=0.075, right=0.985, bottom=0.17, top=0.86, wspace=0.34, hspace=0.48)
328 save_figure(fid, fig, meta, records)
329
330
331 def plot_combined(fid: str, title: str, records: dict) -> None:
332     d, meta = read(fid)
333     x = np.arange(len(d), dtype=float)
334     y = d["display_h"].to_numpy(float)
335     fig, ax = plt.subplots(figsize=(6.7, 3.8))
336     ax.plot(x, y, color=COLORS[0], linewidth=1.8, linestyle="-", zorder=2)
337     for i, (_, row) in enumerate(d.iterrows()):
338         failed = float(row["display_h"]) >= 72.0 and float(row["final_max_C"]) > THRESHOLD
339         marker = "X" if failed else "s"
340         color = COLORS[5] if failed else COLORS[1]
341         ax.scatter([i], [float(row["display_h"])], color=color, marker=marker, s=70,
342             edgecolors=BLACK, linewidths=0.7, zorder=3)
343         text = "72 h: 未达标" if failed else f"{float(row['display_h']):.2f} h"
344         ax.annotate(text, (i, float(row["display_h"])), xytext=(0, 9 if failed else 7),
345             textcoords="offset points", ha="center", va="bottom",
346             fontproperties=SONG, color=BLACK, fontsize=8.0)
347     ax.set_xticks(x, [" 不利组合", " 有利组合"])
348     ax.set_ylim(0, 75)
349     ax.set_yticks([0, 18, 36, 54, 72])
350     ax.axhline(72, color=BLACK, linestyle="--", linewidth=0.85)
351     style_axis(ax, " 组合扰动情景", " 达标时间 t* (h; × 表示未达标)", title)
352     legend_black(
353         ax,
354         handles=[
355             Line2D([0], [0], color=COLORS[1], marker="s", linestyle="-", lw=1.6, markersize=5, label=" 达标"),
356             Line2D([0], [0], color=COLORS[5], marker="X", linestyle="None", markersize=6, label="72 h 内未达标"),
357         ],
358         loc="upper center", bbox_to_anchor=(0.5, 1.01), ncol=2,
359     )
360     fig.subplots_adjust(left=0.11, right=0.98, bottom=0.19, top=0.82)
361     save_figure(fid, fig, meta, records)
362
363
364 def plot_radius(records: dict) -> None:
365     fid = "FIG-Q4-RADIUS-TIME"
366     d, meta = read(fid)
367     x = d["time_s"].to_numpy(float) / 3600.0
368     y = d["radius_m"].to_numpy(float) * 100.0
369     marks = np.unique(np.linspace(0, len(d) - 1, 6, dtype=int))
370     fig, ax = plt.subplots(figsize=(6.7, 3.8))
371     ax.plot(x, y, color=COLORS[0], linewidth=1.7, linestyle="-", marker="o",
372         markersize=4.0, markevery=marks, markerfacecolor=COLORS[0],
373         markeredgecolor=BLACK, markeredgewidth=0.55)
374     ax.annotate(f"R(0) = {y[0]:.2f} cm", (x[0], y[0]), xytext=(9, 8),
375         textcoords="offset points", fontproperties=SONG, color=BLACK, fontsize=8)
376     ax.annotate(f"R(t*) = {y[-1]:.2f} cm", (x[-1], y[-1]), xytext=(-68, 8),

```

```

377         textcoords="offset points", ha="left", fontproperties=SONG, color=BLACK, fontsize=8)
378     style_axis(ax, " 时间 t (h)", " 半径 R(t) (cm)", " 移动边界半径随时间收缩")
379     save_figure(fid, fig, meta, records)
380
381
382 def plot_threshold(fid: str, title: str, records: dict) -> None:
383     d, meta = read(fid)
384     x = d["time_s"].to_numpy(float) / 3600.0
385     fields = [("max_C", " 最大值", COLORS[0], "o"), ("mean_C", " 均值", COLORS[1], "s"),
386              ("center_C", " 中心", COLORS[2], "^"), ("surface_C", " 表面", COLORS[3], "D")]
387     marks = np.unique(np.linspace(0, len(d) - 1, 6, dtype=int))
388     fig, ax = plt.subplots(figsize=(6.7, 4.05))
389     for col, label, color, marker in fields:
390         ax.plot(x, d[col].to_numpy(float), color=color, linewidth=1.35, linestyle="--",
391                marker=marker, markersize=4.0, markevery=marks, markeredgewidth=0.45, label=label)
392     ax.axhline(THRESHOLD, color=BLACK, linestyle="--", linewidth=0.9, label=" 判据 0.149999")
393     ax.axvline(183840 / 3600.0, color=BLACK, linestyle=":", linewidth=0.85)
394     ax.text(183840 / 3600.0 - 0.25, THRESHOLD + 0.035, " 报告时刻 183840 s",
395            ha="right", va="bottom", fontproperties=SONG, color=BLACK, fontsize=8)
396     ax.set_ylim(0, 2.65)
397     ax.set_yticks([0, 0.5, 1.0, 1.5, 2.0, 2.5])
398     style_axis(ax, " 时间 t (h)", " 干基含水率 C (kg/kg)", title)
399     legend_black(ax, ncol=3, loc="upper center", bbox_to_anchor=(0.5, 1.01), columnspacing=1.1)
400     fig.subplots_adjust(left=0.105, right=0.99, bottom=0.16, top=0.82)
401     save_figure(fid, fig, meta, records)
402
403
404
405 def plot_implementation(records: dict) -> None:
406     fid = "FIG-Q4-IMPLEMENTATION-AGREEMENT"
407     d, meta = read(fid)
408     base = float(d["event_time_s"].iloc[0])
409     raw = d["event_time_s"].to_numpy(float)
410     delta = abs(float(raw[1] - raw[0]))
411     relative_delta = delta / abs(base)
412     normalized = raw / base
413     fig, (ax_a, ax_b) = plt.subplots(1, 2, figsize=(7.2, 3.85),
414                                    gridspec_kw={"width_ratios": [1.25, 1.0]})
415     x = np.arange(len(d), dtype=float)
416     # 主面板表达“同一量级、同一事件”：用相对于 reference 的归一化时刻，
417     # 而不是把纳秒级绝对差值作为主视觉。两点在当前尺度上重合是有意的。
418     ax_a.axhline(1.0, color=BLACK, linestyle="--", linewidth=0.9, zorder=1)
419     ax_a.scatter(x, normalized, color=[COLORS[0], COLORS[1]],
420                marker="o", s=74, edgecolors=BLACK, linewidths=0.7, zorder=3)
421     ax_a.set_xticks(x, ["reference", "moving-FV"])
422     ax_a.set_xlim(-0.45, 1.45)
423     ax_a.set_ylim(0.9999997, 1.0000003)
424     ax_a.set_yticks([1.0])
425     ax_a.set_yticklabels(["1.0000000000000000"])
426     style_axis(ax_a, " 数值实现", " 归一化达标时刻 t/tref",
427            " 两种实现的事件时刻重合")
428     ax_a.text(0.5, 0.86, " 两点在当前尺度上重合\n 插值时刻约 183789.331749 s\n 均报告 183840 s",
429            transform=ax_a.transAxes, ha="center", va="center",
430            fontproperties=SONG, color=BLACK, fontsize=8)
431
432     # 次面板只量化相对差值，并使用对数横轴表达其相对于事件时刻的极小量级。
433     ax_b.set_xscale("log")
434     ax_b.set_xlim(1e-16, 1e-14)
435     ax_b.set_ylim(-0.7, 0.7)
436     ax_b.set_yticks([])
437     ax_b.set_xticks([1e-16, 1e-15, 1e-14])
438     ax_b.set_xticklabels(["10-16", "10-15", "10-14"])
439     ax_b.axvline(relative_delta, color=COLORS[1], linewidth=1.8, zorder=2)
440     ax_b.scatter([relative_delta], [0], color=COLORS[1], marker="o", s=78,
441                edgecolors=BLACK, linewidths=0.7, zorder=3)
442     style_axis(ax_b, r" 相对差值 $\Delta t/t_{\text{ref}}$", " ",
443            " 差值相对于事件时刻的量级")
444     ax_b.text(0.5, 0.79,
445            f" 相对差值 {relative_delta:.2e}",
446            transform=ax_b.transAxes, ha="center", va="center",
447            fontproperties=SONG, color=BLACK, fontsize=8)
448     ax_b.text(0.5, 0.16, " 相对于约 1.84e5 s 的事件时刻",
449            transform=ax_b.transAxes, ha="center", va="center",
450            fontproperties=SONG, color=BLACK, fontsize=8)
451     fig.text(0.52, 0.035, " 实现一致性不等于现实准确性",
452            ha="center", fontproperties=SONG, color=BLACK, fontsize=8)
453     fig.subplots_adjust(left=0.14, right=0.99, bottom=0.25, top=0.82, wspace=0.48)
454     save_figure(fid, fig, meta, records)
455

```

```

456
457 def plot_jacobian(records: dict) -> None:
458     fid = "FIG-Q4-JACOBIAN-ABLATION"
459     d, meta = read(fid)
460     x = np.arange(len(d), dtype=float)
461     y = d["balance_error"].to_numpy(float)
462     fig, ax = plt.subplots(figsize=(6.7, 3.8))
463     ax.plot(x, y, color=COLORS[0], linewidth=1.7, linestyle="-", zorder=2)
464     ax.scatter([0], [y[0]], color=COLORS[0], marker="o", s=66, edgecolors=BLACK, linewidths=0.7, zorder=3)
465     ax.scatter([1], [y[1]], color=COLORS[5], marker="X", s=78, edgecolors=BLACK, linewidths=0.7, zorder=3)
466     ax.set_yscale("log")
467     ax.set_ylim(1e-9, 1.2)
468     ax.set_yticks([1e-8, 1e-6, 1e-4, 1e-2, 1e0])
469     ax.yaxis.set_major_formatter(FuncFormatter(plain_log_tick))
470     ax.set_xticks(x, d["route"].tolist())
471     style_axis(ax, " 数值路线", " 归一化平衡误差 (对数轴)", "Jacobian 修正对守恒的影响")
472     ax.annotate("9.645161e-8", (0, y[0]), xytext=(8, 8), textcoords="offset points",
473               fontproperties=SONG, color=BLACK, fontsize=8)
474     ax.annotate("0.5936319\nV10 失败", (1, y[1]), xytext=(-8, -26), textcoords="offset points",
475               ha="right", fontproperties=SONG, color=BLACK, fontsize=8)
476     legend_black(ax, handles=[
477         Line2D([0], [0], color=COLORS[0], marker="o", linestyle="-", lw=1.6, markersize=5, label=" 批准路线"),
478         Line2D([0], [0], color=COLORS[5], marker="X", linestyle="None", markersize=6, label="Jacobian 消融 (失败)"),
479     ], loc="upper center", bbox_to_anchor=(0.5, 1.01), ncol=2)
480     fig.subplots_adjust(left=0.12, right=0.985, bottom=0.20, top=0.82)
481     save_figure(fid, fig, meta, records)
482
483
484 def plot_balance(records: dict) -> None:
485     fid = "FIG-VAL-BALANCE-RESIDUAL"
486     d, meta = read(fid)
487     x = np.arange(len(d), dtype=float)
488     y1 = d["normalized_moisture_balance_error"].to_numpy(float)
489     y2 = d["max_linear_relative_residual"].to_numpy(float)
490     fig, ax = plt.subplots(figsize=(7.2, 4.05))
491     ax.plot(x, y1, color=COLORS[0], linewidth=1.3, linestyle="-", marker="o", markersize=3.7,
492            markevery=5, markeredgewidth=0.4, label=" 平衡误差")
493     ax.plot(x, y2, color=COLORS[1], linewidth=1.3, linestyle="-", marker="s", markersize=3.7,
494            markevery=5, markeredgewidth=0.4, label=" 线性残差")
495     ax.set_yscale("log")
496     ax.set_ylim(1e-12, 1.0)
497     ax.yaxis.set_major_formatter(FuncFormatter(plain_log_tick))
498     tick = np.array([0, 10, 20, 30, 40, 50, 61], dtype=int)
499     ax.set_xticks(tick, [str(i + 1) for i in tick])
500     style_axis(ax, " 正式运行序号", " 无量纲误差 (对数轴)", " 正式运行的守恒与线性残差")
501     legend_black(ax, loc="upper right", ncol=2)
502     fig.subplots_adjust(left=0.10, right=0.99, bottom=0.17, top=0.86)
503     save_figure(fid, fig, meta, records)
504
505
506 def plot_q1_grid(records: dict) -> None:
507     fid = "FIG-Q1-GRID-CONV"
508     d, meta = read(fid)
509     fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.75), sharey=False)
510     for ax, family, color_shift in zip(axes, ["M1", "M2"], [0, 1]):
511         sub = d.loc[d["family"].eq(family)]
512         for j, (metric, label, marker) in enumerate(
513             [("final_max_C", " 最大值", "o"), ("final_mean_C", " 均值", "s")]):
514             s = sub.loc[sub["metric"].eq(metric)].sort_values("nr")
515             ax.plot(s["nr"].to_numpy(float), s["value"].to_numpy(float), color=COLORS[j],
516                   linewidth=1.55, linestyle="-", marker=marker, markersize=4.5,
517                   markeredgewidth=0.5, label=label)
518             ax.set_title(family, loc="left", pad=9, fontproperties=SONG, color=BLACK)
519             style_axis(ax, " 径向网格数 n_r", " 终值 C (kg/kg 干基)")
520             legend_black(ax, loc="best", ncol=1)
521     fig.subplots_adjust(left=0.08, right=0.99, bottom=0.20, top=0.88, wspace=0.35)
522     save_figure(fid, fig, meta, records)
523
524
525
526 def plot_v02(records: dict) -> None:
527     fid = "FIG-Q2-V02-MARGIN"
528     d, meta = read(fid)
529     value = float(d["absolute_change"].iloc[0])
530     threshold = float(d["threshold"].iloc[0])
531     margin = float(d["margin"].iloc[0])
532     fig, ax = plt.subplots(figsize=(6.7, 3.5))
533     ax.plot([0, value], [0, 0], color=COLORS[1], linewidth=1.8, linestyle="-", zorder=2)

```

```

534 ax.scatter([value], [0], color=COLORS[1], marker="o", s=75, edgecolors=BLACK, linewidths=0.7, zorder=3)
535 ax.axvline(threshold, color=BLACK, linestyle="--", linewidth=0.9)
536 ax.set_xlim(0, 5.5e-5)
537 ax.set_ylim(-0.32, 0.40)
538 ax.set_yticks([0], ["V02"])
539 ax.xaxis.set_major_formatter(FuncFormatter(lambda v, _: f"{v / 1e-5:g}e-5"))
540 style_axis(ax, " 最大登记场变化 (kg/kg)", " 验证项", "V02 与验收边界")
541 ax.text(threshold, 0.19, " 验收线 5e-5", ha="right", fontproperties=SONG, color=BLACK, fontsize=8)
542 ax.annotate(f"4.00114e-5\n 余量 {margin:.6g}", (value, 0), xytext=(-5, 12),
543            textcoords="offset points", ha="right", fontproperties=SONG, color=BLACK, fontsize=8)
544 fig.subplots_adjust(left=0.12, right=0.98, bottom=0.22, top=0.83)
545 save_figure(fid, fig, meta, records)
546
547
548 def plot_space(fid: str, title: str, records: dict) -> None:
549     d, meta = read(fid)
550     x = d["nr"].to_numpy(float)
551     y = d["interpolated_event_time_s"].to_numpy(float) / 3600.0
552     fig, ax = plt.subplots(figsize=(6.7, 3.7))
553     ax.plot(x, y, color=COLORS[0], linewidth=1.7, linestyle="-", marker="o", markersize=5.0,
554            markeredgecolor=BLACK, markeredgewidth=0.6)
555     for xi, yi, mk in zip(x, y, MARKERS[:len(x)]):
556         ax.scatter([xi], [yi], color=COLORS[0], marker=mk, s=56, edgecolors=BLACK, linewidths=0.6, zorder=3)
557     ax.set_xticks(x, [str(int(v)) for v in x])
558     lo, hi = float(np.min(y)), float(np.max(y))
559     ax.set_ylim(lo - 0.003, hi + 0.003)
560     style_axis(ax, " 径向网格数 n_r", " 插值事件时刻 (h)", title)
561     ax.annotate(f" 最细 {float(d['interpolated_event_time_s'].iloc[-1]):.2f} s",
562               (x[-1], y[-1]), xytext=(-8, 9), textcoords="offset points", ha="right",
563               fontproperties=SONG, color=BLACK, fontsize=8)
564     save_figure(fid, fig, meta, records)
565
566
567 def plot_bracket(records: dict) -> None:
568     fid = "FIG-Q4-BRACKET-ZOOM"
569     d, meta = read(fid)
570     x = d["time_s"].to_numpy(float)
571     y = d["max_C"].to_numpy(float)
572     fig, ax = plt.subplots(figsize=(6.7, 3.8))
573     ax.plot(x, y, color=COLORS[0], linewidth=1.65, linestyle="-", zorder=2)
574     for i, (xi, yi) in enumerate(zip(x, y)):
575         ax.scatter([xi], [yi], color=COLORS[i % len(COLORS)], marker=MARKERS[i], s=54,
576                    edgecolors=BLACK, linewidths=0.6, zorder=3)
577     ax.axhline(THRESHOLD, color=BLACK, linestyle="--", linewidth=0.9)
578     ax.axvline(183789.33174915268, color=BLACK, linestyle=":", linewidth=0.85)
579     ax.axvline(183840, color=BLACK, linestyle="-. ", linewidth=0.85)
580     ax.text(183789.33174915268, float(np.max(y)) + 0.000004, " 插值 183789.33 s",
581            ha="center", va="bottom", fontproperties=SONG, color=BLACK, fontsize=8)
582     ax.text(183840, float(np.min(y)) - 0.000004, " 报告 183840 s",
583            ha="center", va="top", fontproperties=SONG, color=BLACK, fontsize=8)
584     ax.set_xlim(float(x.min()) - 30, float(x.max()) + 30)
585     ax.set_ylim(float(y.min()) - 0.000015, float(y.max()) + 0.000015)
586     style_axis(ax, " 时间 t (s)", " 最大干基含水率 C (kg/kg)", "Q4 判据邻域的时间夹逼")
587     fig.subplots_adjust(left=0.14, right=0.985, bottom=0.19, top=0.84)
588     save_figure(fid, fig, meta, records)
589
590
591 def plot_dry_solid(records: dict) -> None:
592     fid = "FIG-Q4-DRY-SOLID-CONTINUITY"
593     d, meta = read(fid)
594     x = np.arange(len(d), dtype=float)
595     y = d["plot_value"].to_numpy(float)
596     fig, ax = plt.subplots(figsize=(6.7, 3.8))
597     ax.plot(x, y, color=COLORS[0], linewidth=1.65, linestyle="-", zorder=2)
598     for i, yi in enumerate(y):
599         marker = "o" if i < 2 else "s"
600         face = "none" if i == 2 else COLORS[0]
601         ax.scatter([i], [yi], color=COLORS[0], marker=marker, facecolors=face,
602                    s=64, edgecolors=BLACK, linewidths=0.7, zorder=3)
603     ax.set_yscale("log")
604     ax.set_ylim(1e-18, 1e-12)
605     ax.set_yticks([1e-18, 1e-16, 1e-14, 1e-12])
606     ax.yaxis.set_major_formatter(FuncFormatter(plain_log_tick))
607     ax.set_xticks(x, [" 干固体存量", " 局部连续性", " 几何"])
608     style_axis(ax, "V12 分解指标", " 误差/残差 (对数轴; 0 单独标注)", " 干固体连续性与几何一致性")
609     for i, row in d.iterrows():
610         label = "0" if float(row["value"]) == 0 else f"{float(row['value']):.3e}"
611         ax.annotate(label, (i, float(row["plot_value"])), xytext=(0, 9), textcoords="offset points",
612                    ha="center", fontproperties=SONG, color=BLACK, fontsize=8)
613

```

```

613     fig.subplots_adjust(left=0.12, right=0.985, bottom=0.20, top=0.84)
614     save_figure(fid, fig, meta, records)
615
616
617 def write_contact_sheet(ids: list[str]) -> None:
618     for kind in ("color", "grayscale"):
619         images = [Image.open(REV_PREVIEWS / f"{fid}-{kind}.png").convert("RGB") for fid in ids]
620         thumb_w, thumb_h = 520, 300
621         sheet = Image.new("RGB", (thumb_w * 3, thumb_h * ((len(images) + 2) // 3)), "white")
622         for i, image in enumerate(images):
623             image.thumbnail((thumb_w - 14, thumb_h - 14), Image.Resampling.LANCZOS)
624             x = (i % 3) * thumb_w + (thumb_w - image.width) // 2
625             y = (i // 3) * thumb_h + (thumb_h - image.height) // 2
626             sheet.paste(image, (x, y))
627         sheet.save(REV_PREVIEWS / f"contact-sheet-{kind}.png", dpi=(150, 150))
628
629
630 def update_manifest(records: dict) -> None:
631     path = STAGE / "figure-manifest.json"
632     manifest = json.loads(path.read_text(encoding="utf-8"))
633     for item in manifest["items"]:
634         fid = item["id"]
635         if fid not in records:
636             continue
637         item.update(
638             {
639                 "pdf": f"06-figure/figures/{fid}.pdf",
640                 "png_sha256": sha256(FORMAL / f"{fid}.png"),
641                 "svg_sha256": sha256(FORMAL / f"{fid}.svg"),
642                 "pdf_sha256": sha256(FORMAL / f"{fid}.pdf"),
643                 "visual_revision": "redraw-fig9-onward-20260912-simsun-solid-marker",
644                 "data_claim_status": "unchanged",
645                 "format_check": "SVG + PDF + PNG@600dpi + color/grayscale preview",
646                 "font_spec": "SimSun for Chinese, variables, units, ticks and legend",
647                 "non_data_color_spec": "black axes, grid, title, legend, ticks and text",
648             }
649         )
650     manifest["last_visual_revision"] = {
651         "scope": "figure 9 onward (14 figures)",
652         "revision": "redraw-fig9-onward-20260912-simsun-solid-marker",
653         "data_claim_status": "unchanged",
654     }
655     path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
656
657
658 def write_register(records: dict) -> None:
659     style = {
660         "language": "zh-CN",
661         "font": "SimSun",
662         "data_encoding": "high-contrast color + solid line + distinct marker",
663         "non_data_encoding": "black axes, grid, title, legend, ticks and text",
664         "formats": ["svg", "pdf", "png@600dpi"],
665         "scope": "figure 9 onward; figure 1-8 unchanged",
666         "data_policy": "frozen snapshots only; no model recomputation, deletion or metric change",
667     }
668     payload = {
669         "schema_version": "1.0",
670         "status": "VALIDATED_PENDING_TEAM_REVIEW",
671         "revision": "redraw-fig9-onward-20260912",
672         "visual_spec": style,
673         "items": [
674             {
675                 "figure_id": fid,
676                 "source_snapshot": records[fid]["snapshot"],
677                 "source_snapshot_sha256": records[fid]["snapshot_sha256"],
678                 "rows": records[fid]["rows"],
679                 "columns": records[fid]["columns"],
680                 "outputs": {
681                     "ext": {
682                         "path": f"06-figure/figures/{fid}.{ext}",
683                         "sha256": sha256(FORMAL / f"{fid}.{ext}"),
684                     }
685                     for ext in ("svg", "pdf", "png")
686                 },
687                 "visual_qa": records[fid]["visual_qa"],
688             }
689             for fid in TARGET_IDS
690         ],
691     }

```

```

692 (REV / "figure-register.json").write_text(
693     json.dumps(payload, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
694 )
695
696
697 def run_strict_qa() -> str:
698     checker = ROOT / "skills" / "math-modeling-v2" / "tools" / "figure" / "scripts" / "check_figure.py"
699     paths = [str(FORMAL / f"{fid}.{ext}") for fid in TARGET_IDS for ext in ("pdf", "svg", "png")]
700     result = subprocess.run(
701         [sys.executable, str(checker), *paths, "--min-dpi", "300", "--strict"],
702         cwd=ROOT, capture_output=True, text=True, encoding="utf-8", errors="replace",
703     )
704     output = result.stdout + ("\n" + result.stderr if result.stderr else "")
705     (REV_QA / "strict-figure-qa.log").write_text(output, encoding="utf-8")
706     require(result.returncode == 0, "check_figure.py --strict 未通过, 请查看
    ↳ redraw-fig9-onward-20260912/qa/strict-figure-qa.log")
707     return output
708
709
710 def update_visual_audit(records: dict, strict_output: str) -> None:
711     path = STAGE / "visual-audit.md"
712     existing = path.read_text(encoding="utf-8") if path.exists() else "# 视觉与程序审计\n"
713     marker = "\n## 图 9-22 重绘审计 (2026-09-12)\n"
714     if marker in existing:
715         existing = existing.split(marker, 1)[0].rstrip() + "\n"
716     passed = sum(1 for r in records.values() if r["visual_qa"]["status"] == "PASS")
717     section = marker + "\n"
718     section += "- 范围: 图 9-22, 共 14 张; 图 1-8 保持原正式版本不变。 \n"
719     section += "- 数据检查: 14/14 冻结快照哈希匹配, 且每个快照声明的源文件哈希匹配; 无重算、删点、换指标或改主张。 \n"
720     section += "- 样式检查: 中文、变量、单位、刻度和图例统一使用宋体; 数据线统一实线并用高对比颜色与 marker 区分; 坐标轴、
    ↳ 网格、标题、图例、刻度及文字统一黑色。 \n"
721     section += "- 输出检查: 14/14 均生成 SVG、PDF、600 DPI PNG, 以及彩色和灰度预览; SVG 不嵌入位图, PDF 使用 TrueType
    ↳ 字体设置。 \n"
722     section += f"- 程序版版式检查: {passed}/14 无 FAIL; 严格格式检查已通过, 详见
    ↳ `redraw-fig9-onward-20260912/qa/strict-figure-qa.log`。 \n"
723     section += "- 视觉复核重点: 无中文方框、无边缘裁切、无图例遮挡、灰度下 marker/线条仍可区分; 72 h 未达标、V10
    ↳ Jacobian 消融失败、窄裕量和守恒零值均保留。 \n"
724     section += "- 限制条件移至正式图注/正文: 压力测试非概率区间、条件仿真非现实验证、
    ↳ 冻结收缩律非尺寸实测等未作为图内装饰性备注。 \n"
725     section += "- 阶段边界: 仅更新 FIGURE 图稿和本轮交接证据, 不修改 H1/H2/H3、COMPUTE、EVIDENCE 或 PAPER,
    ↳ 不推进后续阶段。 \n\n"
726     path.write_text(existing + section, encoding="utf-8")
727
728
729 def update_handoff(records: dict) -> None:
730     path = STAGE / "handoff.json"
731     handoff = json.loads(path.read_text(encoding="utf-8"))
732     handoff["status"] = "PASS"
733     handoff["technical_validation"] = "PASS"
734     handoff["visual_review_status"] = "PASS"
735     handoff["redraw_scope"] = {
736         "figures": TARGET_IDS,
737         "figure_number_range": "9-22",
738         "revision": "redraw-fig9-onward-20260912",
739         "data_claim_status": "unchanged",
740         "team_review": "PENDING_TEAM_REVIEW",
741     }
742     handoff["claims"] = list(handoff.get("claims", []))
743     statement = "图 9-22 已按宋体、黑色非数据元素、彩色实线与 marker 规范重绘; 源数据和冻结快照一致, 未推进后续阶段。"
744     if statement not in handoff["claims"]:
745         handoff["claims"].append(statement)
746     handoff["evidence"] = list(handoff.get("evidence", []))
747     for evidence in [
748         "06-figure/redraw-fig9-onward-20260912/figure-register.json",
749         "06-figure/redraw-fig9-onward-20260912/data-verification.json",
750         "06-figure/redraw-fig9-onward-20260912/qa/strict-figure-qa.log",
751     ]:
752         if evidence not in handoff["evidence"]:
753             handoff["evidence"].append(evidence)
754     handoff["required_next_actions"] = [
755         "团队复核图 9-22 的正式视觉版本后, 继续沿当前流程处理; 本轮不自动推进 PAPER。",
756         "PAPER 如启动, 使用本轮 figure-manifest、图注和冻结快照, 不改变数据或主张限制。",
757     ]
758     output_entries = []
759     for p in sorted(STAGE.rglob("*")):
760         if p.is_file() and p != path:
761             output_entries.append({"path": rel(p), "sha256": sha256(p)})
762     handoff["outputs"] = output_entries
763     handoff["completed_at"] = datetime.now(timezone(timedelta(hours=8))).isoformat()

```

```

764     path.write_text(json.dumps(handoff, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
765
766
767 def main() -> None:
768     configure_style()
769     for directory in (REV_FIGURES, REV_PREVIEWS, REV_QA, REV_SUPERSEDED):
770         directory.mkdir(parents=True, exist_ok=True)
771     sys.path.insert(0, str(ROOT / "skills" / "math-modeling-v2" / "tools" / "figure" / "scripts"))
772
773     records: dict[str, dict] = {}
774     plot_q3_sensitivity(records)
775     plot_combined("FIG-Q3-COMBINED-BOUNDARY", "Q3 组合边界的达标状态", records)
776     plot_radius(records)
777     plot_threshold("FIG-Q4-THRESHOLD-TRAJECTORY", "Q4 收缩域的含水率轨迹", records)
778     plot_jacobian(records)
779     plot_combined("FIG-Q4-COMBINED-BOUNDARY", "Q4 组合扰动的成功与失败边界", records)
780     plot_q1_grid(records)
781     plot_v02(records)
782     plot_space("FIG-Q3-SPACE-CONV", "Q3 空间网格收敛", records)
783     plot_bracket(records)
784     plot_space("FIG-Q4-SPACE-CONV", "Q4 空间网格收敛", records)
785     plot_dry_solid(records)
786     require(set(records) == set(TARGET_IDS), " 目标图未全部生成")
787
788     write_contact_sheet(TARGET_IDS)
789     data_verification = {
790         "status": "PASS",
791         "scope": "figure 9 onward",
792         "figures": records,
793         "policy": " 读取冻结快照; 源文件仅用于 hash verification: 不重新计算模型",
794     }
795     (REV / "data-verification.json").write_text(
796         json.dumps(data_verification, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
797     )
798     update_manifest(records)
799     write_register(records)
800     strict_output = run_strict_qa()
801     update_visual_audit(records, strict_output)
802     update_handoff(records)
803     (REV / "render-record.json").write_text(
804         json.dumps(
805             {
806                 "status": "PASS",
807                 "revision": "redraw-fig9-onward-20260912",
808                 "target_ids": TARGET_IDS,
809                 "style": {
810                     "font": "SimSun",
811                     "data": "high-contrast color, solid line, distinct marker",
812                     "non_data": "black axes, grid, title, legend, ticks and text",
813                     "formats": ["svg", "pdf", "png@600dpi"],
814                 },
815                 "records": records,
816             },
817             ensure_ascii=False,
818             indent=2,
819         )
820         + "\n",
821         encoding="utf-8",
822     )
823     # render-record 写入在 handoff 之后, 更新一次 outputs 以便其哈希也纳入交接。
824     update_handoff(records)
825     print(json.dumps({"status": "PASS", "figures": len(TARGET_IDS), "revision": rel(REV)}, ensure_ascii=False))
826
827
828 if __name__ == "__main__":
829     main()
830

```

D.11 程序 11: redraw_q1_end_effect_time.py

原文件: 06-figure/scripts/redraw_q1_end_effect_time.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 """Replace FIG-Q1-END-EFFECT with the Q1 M1/M2 moisture trajectories.
2
3 The trajectory is generated with the already approved formal solver and the
4 same Q1 base configurations used by 04-compute. No model definition or
5 upstream result file is changed; only the figure-stage snapshot and outputs
6 are written.
7 """
8
9 from __future__ import annotations
10
11 import csv
12 import hashlib
13 import html
14 import importlib.util
15 import json
16 import math
17 import os
18 import sys
19 from pathlib import Path
20
21 from PIL import Image, ImageDraw, ImageFont, ImageOps
22
23
24 ROOT = Path(__file__).resolve().parents[2]
25 OUT = ROOT / "06-figure"
26 RES = ROOT / "04-compute" / "results"
27 SOLVER_PATH = ROOT / "04-compute" / "src" / "formal_compute.py"
28 FID = "FIG-Q1-END-EFFECT"
29
30
31 def sha256(path: Path) -> str:
32     digest = hashlib.sha256()
33     with path.open("rb") as handle:
34         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
35             digest.update(chunk)
36     return digest.hexdigest()
37
38
39 def rel(path: Path) -> str:
40     return path.relative_to(ROOT).as_posix()
41
42
43 def load_formal_solver():
44     spec = importlib.util.spec_from_file_location("formal_compute_for_q1_figure", SOLVER_PATH)
45     if spec is None or spec.loader is None:
46         raise RuntimeError(f"cannot load formal solver: {SOLVER_PATH}")
47     module = importlib.util.module_from_spec(spec)
48     sys.modules[spec.name] = module
49     spec.loader.exec_module(module)
50     return module
51
52
53 def read_run_summary() -> dict[str, float]:
54     values: dict[str, float] = {}
55     with (RES / "run-summary.csv").open(encoding="utf-8-sig", newline="") as handle:
56         for row in csv.DictReader(handle):
57             if row["label"] in {"Q1-M1-base", "Q1-M2-base"}:
58                 values[row["label"]] = float(row["final_mean_C"])
59     if set(values) != {"Q1-M1-base", "Q1-M2-base"}:
60         raise RuntimeError("Q1 base endpoint results are incomplete")
61     return values
62
63
64 def build_snapshot() -> tuple[Path, Path, dict[str, dict[str, float | int]]]:
65     solver = load_formal_solver()
66     configs = {
67         "M1 一维基线": solver.RunConfig(
68             "Q1-M1-base", 122, 1, 0.5, 1800, 1, "M1", sample_every=60.0
69         ),
70         "M2 二维主干": solver.RunConfig(
```

```

71         "Q1-M2-base", 93, 54, 0.5, 1800, 1, "M2", sample_every=60.0
72     ),
73 }
74 endpoint = read_run_summary()
75 records: list[dict[str, object]] = []
76 provenance: dict[str, dict[str, float | int]] = {}
77 for label, cfg in configs.items():
78     result = solver.simulate(cfg)
79     rows = result["records"]
80     if not rows or rows[-1]["time_s"] != 1800.0:
81         raise RuntimeError(f"trajectory endpoint missing for {label}")
82     expected = endpoint["Q1-M1-base" if cfg.model == "M1" else "Q1-M2-base"]
83     actual = float(rows[-1]["mean_C"])
84     if not math.isclose(actual, expected, rel_tol=0.0, abs_tol=2e-10):
85         raise RuntimeError(f"endpoint mismatch for {label}: {actual} vs {expected}")
86     provenance[label] = {
87         "rows": len(rows),
88         "nr": cfg.nr,
89         "nz": cfg.nz,
90         "dt_s": cfg.dt,
91         "duration_s": cfg.duration,
92         "sample_every_s": cfg.sample_every,
93         "endpoint_mean_C": actual,
94     }
95     for row in rows:
96         records.append(
97             {
98                 "model": label,
99                 "time_s": float(row["time_s"]),
100                 "mean_C": float(row["mean_C"]),
101             }
102         )
103
104 records.sort(key=lambda row: (str(row["model"]), float(row["time_s"])))
105 snapshot = OUT / "data-snapshots" / f"{FID}.csv"
106 snapshot.parent.mkdir(parents=True, exist_ok=True)
107 with snapshot.open("w", encoding="utf-8-sig", newline="") as handle:
108     writer = csv.DictWriter(handle, fieldnames=["model", "time_s", "mean_C"])
109     writer.writeheader()
110     for row in records:
111         writer.writerow(
112             {
113                 "model": row["model"],
114                 "time_s": f"{float(row['time_s']):.16g}",
115                 "mean_C": f"{float(row['mean_C']):.16g}",
116             }
117         )
118
119 source_paths = [SOLVER_PATH, RES / "run-summary.csv"]
120 meta = {
121     "figure_id": FID,
122     "snapshot": rel(snapshot),
123     "snapshot_sha256": sha256(snapshot),
124     "sources": [{"path": rel(path), "sha256": sha256(path)} for path in source_paths],
125     "rows": len(records),
126     "columns": ["model", "time_s", "mean_C"],
127     "derived_metric": "solver record mean_C, volume-weighted mean dry-basis moisture",
128     "sampling": "every 60 s from t=0 to t=1800 s",
129     "configurations": provenance,
130     "endpoint_check": "matches 04-compute/results/run-summary.csv within 2e-10 kg/kg",
131 }
132 meta_path = snapshot.with_suffix(".meta.json")
133 meta_path.write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
134 return snapshot, meta_path, provenance
135
136
137 def load_snapshot(snapshot: Path) -> dict[str, list[tuple[float, float]]]:
138     series: dict[str, list[tuple[float, float]]] = {}
139     with snapshot.open(encoding="utf-8-sig", newline="") as handle:
140         for row in csv.DictReader(handle):
141             series.setdefault(row["model"], []).append((float(row["time_s"]), float(row["mean_C"])))
142     if set(series) != {"M1 一维基线", "M2 二维主干"}:
143         raise RuntimeError("snapshot must contain exactly the M1 and M2 series")
144     return series
145
146
147 def render(snapshot: Path) -> tuple[Path, Path, Path, Path]:
148     series = load_snapshot(snapshot)
149     figures = OUT / "figures"

```

```

150 previews = OUT / "previews"
151 figures.mkdir(parents=True, exist_ok=True)
152 previews.mkdir(parents=True, exist_ok=True)
153 svg = figures / f"{FID}.svg"
154 png = figures / f"{FID}.png"
155 color_preview = previews / f"{FID}-color.png"
156 gray_preview = previews / f"{FID}-grayscale.png"
157
158 # 6.7 x 4.2 in at 600 dpi. This keeps the figure renderer dependency-light
159 # while preserving the approved white-background academic style.
160 width, height = 4020, 2520
161 left, right, top, bottom = 400, 3840, 260, 2050
162 ymin, ymax = 2.24, 2.58
163 dark = "#222222"
164 grid_gray = "#C7C7C7"
165 axis_width = 6
166 title = "图 2: 一维与二维模型平均含水率随时间变化"
167 title_y = 2390
168 x_label_y = 2210
169 colors = {"M1 一维基线": "#0072B2", "M2 二维主干": "#D55E00"}
170
171 def font(size: int, *paths: str) -> ImageFont.FreeTypeFont | ImageFont.ImageFont:
172     for path in paths:
173         if Path(path).exists():
174             return ImageFont.truetype(path, size=size)
175     return ImageFont.load_default()
176
177 font_title = font(96, r"C:\Windows\Fonts\simhei.ttf", r"C:\Windows\Fonts\msyh.ttc",
178 ↪ r"C:\Windows\Fonts\simsun.ttc")
179 font_tick = font(78, r"C:\Windows\Fonts\arial.ttf", r"C:\Windows\Fonts\msyh.ttc",
180 ↪ r"C:\Windows\Fonts\simsun.ttc")
181 font_label = font(88, r"C:\Windows\Fonts\simsun.ttc", r"C:\Windows\Fonts\msyh.ttc")
182 font_legend = font(72, r"C:\Windows\Fonts\simsun.ttc", r"C:\Windows\Fonts\msyh.ttc")
183 font_note = font(58, r"C:\Windows\Fonts\simsun.ttc", r"C:\Windows\Fonts\msyh.ttc",
184 ↪ r"C:\Windows\Fonts\arial.ttf")
185
186 def xpix(value: float) -> int:
187     return round(left + (value / 1800.0) * (right - left))
188
189 def ypix(value: float) -> int:
190     return round(bottom - ((value - ymin) / (ymax - ymin)) * (bottom - top))
191
192 def centered(draw: ImageDraw.ImageDraw, xy: tuple[int, int], text: str, fnt, fill=dark) -> None:
193     box = draw.textbbox((0, 0), text, font=fnt)
194     draw.text((xy[0] - (box[2] - box[0]) / 2, xy[1] - (box[3] - box[1]) / 2), text, font=fnt, fill=fill)
195
196 image = Image.new("RGB", (width, height), "white")
197 draw = ImageDraw.Draw(image)
198 draw.rectangle((left, top, right, bottom), outline=dark, width=axis_width)
199 x_ticks = [0, 300, 600, 900, 1200, 1500, 1800]
200 y_ticks = [2.24, 2.30, 2.36, 2.42, 2.48, 2.54, 2.58]
201
202 def dashed_line(points: list[tuple[int, int]], fill: str, width: int, dash: int = 22, gap: int = 14) -> None:
203     for start, end in zip(points, points[1:]):
204         x0, y0 = start
205         x1, y1 = end
206         length = math.hypot(x1 - x0, y1 - y0)
207         if length == 0:
208             continue
209         ux, uy = (x1 - x0) / length, (y1 - y0) / length
210         position = 0.0
211         while position < length:
212             end_position = min(position + dash, length)
213             draw.line(
214                 (
215                     x0 + ux * position,
216                     y0 + uy * position,
217                     x0 + ux * end_position,
218                     y0 + uy * end_position,
219                 ),
220                 fill=fill,
221                 width=width,
222             )
223             position += dash + gap
224
225 centered(draw, ((left + right) // 2, title_y), title, font_title, fill="#000000")
226 for value in x_ticks[1:-1]:
227     x = xpix(value)
228     dashed_line([(x, top), (x, bottom)], fill=grid_gray, width=2, dash=18, gap=12)

```

```

226 for value in y_ticks[1:-1]:
227     y = ypix(value)
228     dashed_line([(left, y), (right, y)], fill=grid_gray, width=2, dash=18, gap=12)
229 for value in x_ticks:
230     x = xpix(value)
231     draw.line((x, bottom, x, bottom - 18), fill=dark, width=axis_width)
232     draw.line((x, top, x, top + 18), fill=dark, width=axis_width)
233     centered(draw, (x, bottom + 48), str(value), font_tick)
234 for value in y_ticks:
235     y = ypix(value)
236     draw.line((left, y, left + 18, y), fill=dark, width=axis_width)
237     draw.line((right, y, right - 18, y), fill=dark, width=axis_width)
238     box = draw.textbbox((0, 0), f"{value:.2f}", font=font_tick)
239     draw.text((left - 28 - (box[2] - box[0]), y - (box[3] - box[1]) / 2), f"{value:.2f}", font=font_tick,
240               ↪ fill=dark)
241 for label in ("M1 一维基线", "M2 二维主干"):
242     values = series[label]
243     points = [(xpix(x), ypix(y)) for x, y in values]
244     if label.startswith("M1"):
245         draw.line(points, fill=colors[label], width=7, joint="curve")
246     else:
247         dashed_line(points, fill=colors[label], width=7)
248     for index in range(0, len(points), 5):
249         x, y = points[index]
250         if label.startswith("M1"):
251             draw.ellipse((x - 12, y - 12, x + 12, y + 12), fill=colors[label])
252         else:
253             draw.rectangle((x - 12, y - 12, x + 12, y + 12), fill=colors[label])
254     x, y = points[-1]
255     if label.startswith("M1"):
256         draw.ellipse((x - 12, y - 12, x + 12, y + 12), fill=colors[label])
257     else:
258         draw.rectangle((x - 12, y - 12, x + 12, y + 12), fill=colors[label])
259
260 # Legend in the open upper-right area; line style and marker redundantly
261 # encode the two model dimensions in color and grayscale.
262 legend_x, legend_y = 2920, 350
263 for offset, label in enumerate(("M1 一维基线", "M2 二维主干")):
264     y = legend_y + offset * 78
265     draw.line((legend_x, y, legend_x + 105, y), fill=colors[label], width=7)
266     if label.startswith("M1"):
267         draw.ellipse((legend_x + 45 - 12, y - 12, legend_x + 45 + 12, y + 12), fill=colors[label])
268     else:
269         draw.rectangle((legend_x + 45 - 12, y - 12, legend_x + 45 + 12, y + 12), fill=colors[label])
270     draw.text((legend_x + 135, y - 28), label, font=font_legend, fill=dark)
271
272 centered(draw, ((left + right) // 2, x_label_y), "时间 t (s)", font_label)
273 y_label = Image.new("RGBA", (1200, 120), (255, 255, 255, 0))
274 ImageDraw.Draw(y_label).text((10, 10), "平均干基含水率 C (kg/kg)", font=font_label, fill=dark)
275 y_label = y_label.rotate(90, expand=True)
276 image.paste(y_label, (30, (height - y_label.height) // 2), y_label)
277 draw = ImageDraw.Draw(image)
278 note = "末值差 0.01855 kg/kg"
279 box = draw.textbbox((0, 0), note, font=font_note)
280 draw.text((right - 30 - (box[2] - box[0]), bottom - 80), note, font=font_note, fill="#444444")
281 image.save(str(png), dpi=(600, 600))
282 image.resize((2010, 1260), Image.Resampling.LANCZOS).save(str(color_preview), dpi=(150, 150))
283 ImageOps.grayscale(Image.open(str(color_preview)).convert("RGB")).save(str(gray_preview), dpi=(150, 150))
284
285 def esc(text: str) -> str:
286     return html.escape(text, quote=True)
287
288 svg_lines: list[str] = [
289     f'<svg xmlns="http://www.w3.org/2000/svg" width="6.7in" height="4.2in" viewBox="0 0 {width} {height}">',
290     '<rect width="100%" height="100%" fill="white"/>',
291     f'<rect x="{left}" y="{top}" width="{right-left}" height="{bottom-top}" fill="white" stroke="{dark}"',
292     ↪ 'stroke-width="{axis_width}">',
293     f'<text x="{(left+right)//2}" y="{title_y+50}" text-anchor="middle" font-family="SimHei,Microsoft',
294     ↪ 'YaHei,SimSun,sans-serif" font-size="96" font-weight="bold" fill="#000000">{esc(title)}</text>',
295 ]
296 for value in x_ticks[1:-1]:
297     x = xpix(value)
298     svg_lines.append(f'<path d="M{x} {top}V{bottom}" stroke="{grid_gray}" stroke-width="2"',
299                     ↪ 'stroke-dasharray="18 12"/>')
300 for value in y_ticks[1:-1]:
301     y = ypix(value)
302     svg_lines.append(f'<path d="M{left} {y}H{right}" stroke="{grid_gray}" stroke-width="2"',
303                     ↪ 'stroke-dasharray="18 12"/>')

```

```

300     for value in x_ticks:
301         x = xpix(value)
302         svg_lines.append(f'<path d="M{x} {bottom}V{bottom-18} M{x} {top}V{top+18}" stroke="{dark}"
↪ stroke-width="{axis_width}" />')
303         svg_lines.append(f'<text x="{x}" y="{bottom+70}" text-anchor="middle"
↪ font-family="Arial,SimSun,sans-serif" font-size="78" fill="{dark}">{value}</text>')
304     for value in y_ticks:
305         y = ypix(value)
306         svg_lines.append(f'<path d="M{left} {y}H{left+18} M{right} {y}H{right-18}" stroke="{dark}"
↪ stroke-width="{axis_width}" />')
307         svg_lines.append(f'<text x="{left-28}" y="{y+22}" text-anchor="end" font-family="Arial,SimSun,sans-serif"
↪ font-size="78" fill="{dark}">{value:.2f}</text>')
308     for label in ("M1 一维基线", "M2 二维主干"):
309         pts = " ".join(f"{xpix(x)}, {ypix(y)}" for x, y in series[label])
310         dash = "" if label.startswith("M1") else 'stroke-dasharray="16 10"'
311         marker = "circle" if label.startswith("M1") else "rect"
312         svg_lines.append(f'<polyline points="{pts}" fill="none" stroke="{colors[label]}" stroke-width="7"{dash}>'
↪ ')
313         for index in list(range(0, len(series[label]), 5)) + [len(series[label]) - 1]:
314             x, y = series[label][index]
315             if marker == "circle":
316                 svg_lines.append(f'<circle cx="{xpix(x)}" cy="{ypix(y)}" r="12" fill="{colors[label]}" />')
317             else:
318                 svg_lines.append(f'<rect x="{xpix(x)-12}" y="{ypix(y)-12}" width="24" height="24"
↪ fill="{colors[label]}" />')
319     svg_lines.extend(
320     [
321         f'<text x="{legend_x+135}" y="{legend_y+20}" font-family="SimSun,Microsoft YaHei,Arial,sans-serif"
↪ font-size="72" fill="{dark}">M1 一维基线</text>',
322         f'<text x="{legend_x+135}" y="{legend_y+98}" font-family="SimSun,Microsoft YaHei,Arial,sans-serif"
↪ font-size="72" fill="{dark}">M2 二维主干</text>',
323         f'<path d="M{legend_x} {legend_y}H{legend_x+105}" stroke="{colors["M1 一维基线"]}"
↪ stroke-width="7" />',
324         f'<path d="M{legend_x} {legend_y+78}H{legend_x+105}" stroke="{colors["M2 二维主干"]}" stroke-width="7"
↪ stroke-dasharray="16 10" />',
325         f'<circle cx="{legend_x+45}" cy="{legend_y}" r="12" fill="{colors["M1 一维基线"]}" />',
326         f'<rect x="{legend_x+33}" y="{legend_y+66}" width="24" height="24" fill="{colors["M2 二维主干"]}" />',
327         f'<text x="{(left+right)//2}" y="{x_label_y+28}" text-anchor="middle" font-family="SimSun,Microsoft
↪ YaHei,Arial,sans-serif" font-size="88" fill="{dark}">时间 t (s)</text>',
328         f'<text x="110" y="{height//2}" text-anchor="middle" transform="rotate(-90 110 {height//2})"
↪ font-family="SimSun,Microsoft YaHei,Arial,sans-serif" font-size="88" fill="{dark}">平均干基含水率 C
↪ (kg/kg)</text>',
329         f'<text x="{right-30}" y="{bottom-30}" text-anchor="end" font-family="Arial,Microsoft
↪ YaHei,sans-serif" font-size="58" fill="#444444">末值差 0.01855 kg/kg</text>',
330         '</svg>',
331     ]
332 )
333     svg.write_text("\n".join(svg_lines), encoding="utf-8")
334     return svg, png, color_preview, gray_preview
335
336
337 def write_contract(snapshot: Path, meta_path: Path) -> Path:
338     contract = {
339         "figure_id": FID,
340         "claim_ids": ["Q1-OBS-01"],
341         "question": "一维与二维模型的平均含水率如何随时间变化?",
342         "title": "图 2: 一维与二维模型平均含水率随时间变化",
343         "title_font": "SimHei",
344         "title_position": "below_plot_center",
345         "grid": "gray dashed auxiliary gridlines",
346         "source_files": [
347             {"path": rel(SOLVER_PATH), "sha256": sha256(SOLVER_PATH)},
348             {"path": rel(RES / "run-summary.csv"), "sha256": sha256(RES / "run-summary.csv")},
349         ],
350         "x": {"field": "time_s", "label": "时间 t", "unit": "s"},
351         "y": {"field": "mean_C", "label": "平均干基含水率 C", "unit": "kg/kg"},
352         "groups": ["M1 一维基线", "M2 二维主干"],
353         "uncertainty": None,
354         "required_comparisons": ["M1/M2 随时间的平均含水率曲线"],
355         "must_show_failures": False,
356         "prohibited_operations": [
357             "recompute with altered model parameters",
358             "change the moisture metric",
359             "omit one of the two base-model series",
360             "present conditional simulation as observation",
361         ],
362         "target_formats": ["svg", "png"],
363         "minimum_dpi": 600,
364         "placement": "主文",

```

```

365     "mandatory_limit": " 模型差异非精度提升; 曲线为条件仿真",
366     "snapshot": rel(snapshot),
367     "snapshot_sha256": sha256(snapshot),
368     "snapshot_meta": rel(meta_path),
369 }
370 path = OUT / "contracts" / f"{FID}.json"
371 path.parent.mkdir(parents=True, exist_ok=True)
372 path.write_text(json.dumps(contract, ensure_ascii=False, indent=2), encoding="utf-8")
373 return path
374
375
376 def refresh_manifest_and_handoff(paths: list[Path]) -> None:
377     manifest_path = OUT / "figure-manifest.json"
378     manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
379     item = next(item for item in manifest["items"] if item["id"] == FID)
380     item["png_sha256"] = sha256(OUT / "figures" / f"{FID}.png")
381     item["svg_sha256"] = sha256(OUT / "figures" / f"{FID}.svg")
382     manifest_path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
383     paths.append(manifest_path)
384
385     handoff_path = OUT / "handoff.json"
386     handoff = json.loads(handoff_path.read_text(encoding="utf-8"))
387     changed = {rel(path): sha256(path) for path in paths}
388     for record in handoff["outputs"]:
389         if record["path"] in changed:
390             record["sha256"] = changed[record["path"]]
391     handoff["assumptions"] = [
392         "FIG-Q1-END-EFFECT was revised to display the approved M1/M2 base trajectories generated with the",
393         ↪ "unchanged formal solver configurations.",
394         "No model definition, parameter, upstream COMPUTE result file, or other figure was changed.",
395         "All plotted values remain conditional simulations and do not constitute empirical accuracy validation.",
396     ]
397     handoff["warnings"] = [
398         "The time-series version supersedes the former endpoint dumbbell rendering for FIG-Q1-END-EFFECT.",
399         "Do not present the M1/M2 difference as an accuracy improvement without internal observations.",
400     ]
401     handoff_path.write_text(json.dumps(handoff, ensure_ascii=False, indent=2), encoding="utf-8")
402
403 def update_stage_docs(paths: list[Path]) -> None:
404     caption_path = OUT / " 图注.md"
405     caption = caption_path.read_text(encoding="utf-8")
406     start = caption.index("## 主文图 2 (FIG-Q1-END-EFFECT) ")
407     end = caption.index("## 主文图 3", start)
408     replacement = (
409         "## 主文图 2 (FIG-Q1-END-EFFECT) \n\n"
410         "一维与二维模型的平均含水率如何随时间变化? 0-1800 s 的体积加权平均干基含水率轨迹; 1800 s 末值差0.0185457085。"
411         "限定: 模型差异非精度提升, 曲线为条件仿真。数据快照与来源哈希见 `data-snapshots/FIG-Q1-END-EFFECT.meta.json`。"
412         ↪ "\n\n"
413     )
414     caption_path.write_text(caption[:start] + replacement + caption[end:], encoding="utf-8")
415     paths.append(caption_path)
416
417     audit_path = OUT / "visual-audit.md"
418     audit = audit_path.read_text(encoding="utf-8")
419     note = (
420         "- FIG-Q1-END-EFFECT 本轮按用户要求由端点哑铃图改为 M1 一维基线与 M2 二维主干的平均含水率—时间曲线;"
421         "将黑体标题 '图2: 一维与二维模型平均含水率随时间变化' 移至图下方居中, 进一步放大标题、坐标轴标签和横纵轴刻度数字,"
422         ↪ "并加粗横纵坐标轴主框线与刻度线, 保留灰色虚线网格辅助线;"
423         "使用 0-1800 s、60 s 采样, 末值与 run-summary.csv 校验通过, 未改动其他图。"
424     )
425     audit = "".join(
426         line for line in audit.splitlines(True)
427         if not line.startswith("- FIG-Q1-END-EFFECT 本轮按用户要求")
428     )
429     audit = audit.replace("- 灰度检查: ", note + "- 灰度检查: ")
430     audit_path.write_text(audit, encoding="utf-8")
431     paths.append(audit_path)
432
433 def main() -> None:
434     snapshot_path = OUT / "data-snapshots" / f"{FID}.csv"
435     if os.environ.get("USE_EXISTING_SNAPSHOT") == "1" and snapshot_path.exists():
436         snapshot = snapshot_path
437         meta_path = snapshot.with_suffix(".meta.json")
438         if not meta_path.exists():
439             raise RuntimeError(f"snapshot metadata is missing: {meta_path}")
440     else:
441         snapshot, meta_path, _ = build_snapshot()

```

```

441     svg, png, color_preview, gray_preview = render(snapshot)
442     contract = write_contract(snapshot, meta_path)
443     changed = [snapshot, meta_path, contract, svg, png, color_preview, gray_preview, Path(__file__)]
444     update_stage_docs(changed)
445     refresh_manifest_and_handoff(changed)
446     print(
447         json.dumps(
448             {
449                 "figure_id": FID,
450                 "snapshot": rel(snapshot),
451                 "svg": rel(svg),
452                 "png": rel(png),
453                 "rows": 62,
454                 "endpoint_check": "PASS",
455             },
456             ensure_ascii=False,
457         )
458     )
459
460
461 if __name__ == "__main__":
462     main()
463

```

D.12 程序 12: render_q2_profiles.py

原文件: 06-figure/scripts/render_q2_profiles.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助；参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from __future__ import annotations
2
3 import csv
4 import hashlib
5 from datetime import datetime, timezone
6 from pathlib import Path
7
8 import matplotlib as mpl
9 import matplotlib.pyplot as plt
10 import pandas as pd
11 from PIL import Image, ImageOps
12
13
14 ROOT = Path(__file__).resolve().parents[2]
15 FIGURE_DIR = ROOT / "06-figure"
16 SOURCE = ROOT / "04-compute" / "results" / "q2-samples.csv"
17 SNAPSHOT = FIGURE_DIR / "data-snapshots" / "FIG-Q2-C-PROFILES.csv"
18 FIGURES = FIGURE_DIR / "figures"
19 PREVIEWS = FIGURE_DIR / "previews"
20 FIG_ID = "FIG-Q2-C-PROFILES"
21 FIELDS = ["time_s", "radius_cm", "C_kg_per_kg"]
22
23
24 def sha256(path: Path) -> str:
25     return hashlib.sha256(path.read_bytes()).hexdigest()
26
27
28 def rows(path: Path) -> list[tuple[int, float, float]]:
29     with path.open("r", encoding="utf-8-sig", newline="") as handle:
30         return sorted(
31             (
32                 (
33                     int(row["time_s"]),
34                     float(row["radius_cm"]),
35                     float(row["C_kg_per_kg"]),
36                 )
37                 for row in csv.DictReader(handle)
38             ),
39             key=lambda row: (row[0], row[1]),
40         )
41
42
43 def same_numeric_rows(left: list[tuple[int, float, float]], right: list[tuple[int, float, float]]) -> bool:
44     if len(left) != len(right):

```

```

45         return False
46     return all(
47         a[0] == b[0]
48         and abs(a[1] - b[1]) <= 1e-12
49         and abs(a[2] - b[2]) <= 1e-12
50         for a, b in zip(left, right)
51     )
52
53
54 def main() -> None:
55     FIGURES.mkdir(parents=True, exist_ok=True)
56     PREVIEWS.mkdir(parents=True, exist_ok=True)
57     assert same_numeric_rows(rows(SOURCE), rows(SNAPSHOT)), "source and frozen figure snapshot differ"
58
59     data = pd.read_csv(SOURCE)[FIELDS].sort_values(
60         ["time_s", "radius_cm"], kind="stable"
61     )
62     mpl.rcParams.update(
63         {
64             "font.family": "SimSun",
65             "font.sans-serif": ["SimSun", "STSong", "FangSong", "Microsoft YaHei", "Arial"],
66             "font.serif": ["SimSun", "STSong", "FangSong", "Microsoft YaHei", "Arial"],
67             "font.size": 8.5,
68             "text.color": "#000000",
69             "axes.labelcolor": "#000000",
70             "xtick.color": "#000000",
71             "ytick.color": "#000000",
72             "axes.unicode_minus": False,
73             "axes.facecolor": "#FFFFFF",
74             "figure.facecolor": "#FFFFFF",
75             "savefig.facecolor": "#FFFFFF",
76             "axes.axisbelow": True,
77             "pdf.fonttype": 42,
78             "ps.fonttype": 42,
79             "svg.fonttype": "none",
80             "svg.hashsalt": "FIG-Q2-C-PROFILES",
81         }
82     )
83
84     # 图线按半径分组：使用高对比彩色、统一实线，并用 marker 进一步区分半径。
85     colors = ["#005AB5", "#E69F00", "#009E73", "#D55E00", "#7B2CBF"]
86     linestyle = ["-"] * 5
87     markers = ["o", "s", "^", "D", "P"]
88     labels = ["r = 0 cm", "r = 0.5 cm", "r = 1 cm", "r = 1.5 cm", "r = 2 cm"]
89
90     fig, ax = plt.subplots(figsize=(6.9, 4.5), dpi=600)
91     fig.subplots_adjust(left=0.135, right=0.985, bottom=0.205, top=0.745)
92     for i, (radius_cm, group) in enumerate(data.groupby("radius_cm", sort=True)):
93         group = group.sort_values("time_s")
94         ax.plot(
95             group["time_s"] / 3600.0,
96             group["C_kg_per_kg"],
97             color=colors[i],
98             linestyle=linestyle[i],
99             linewidth=1.75,
100             marker=markers[i],
101             markersize=5.0,
102             markerfacecolor=colors[i],
103             markeredgecolor=colors[i],
104             markeredgewidth=0.65,
105             label=labels[i],
106             zorder=3 + i * 0.01,
107         )
108
109     ax.set_xlim(0.45, 3.05)
110     ax.set_ylim(0.92, 2.66)
111     ax.set_xticks([0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
112     ax.set_yticks([1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4, 2.6])
113     ax.set_xlabel("时间 t (h)", fontsize=9.5, color="#000000", labelpad=7)
114     ax.set_ylabel("干基含水率 C (kg/kg)", fontsize=9.5, color="#000000", labelpad=8)
115     ax.tick_params(
116         axis="both",
117         which="major",
118         direction="out",
119         length=3.2,
120         width=0.65,
121         colors="#000000",
122         labelsize=8.3,
123     )

```

```

124     for tick_label in [*ax.get_xticklabels(), *ax.get_yticklabels()]:
125         tick_label.set_fontfamily("SimHei")
126     ax.grid(axis="y", color="#000000", linewidth=0.5, alpha=0.15)
127     ax.grid(axis="x", visible=False)
128     ax.spines["top"].set_visible(False)
129     ax.spines["right"].set_visible(False)
130     for side in ("left", "bottom"):
131         ax.spines[side].set_color("#000000")
132         ax.spines[side].set_linewidth(0.75)
133
134     fig.text(
135         0.135,
136         0.935,
137         "M3",
138         ha="left",
139         va="top",
140         fontsize=11.5,
141         fontfamily="SimHei",
142         color="#000000",
143     )
144     fig.text(
145         0.175,
146         0.935,
147         "不同半径位置含水率随时间演化",
148         ha="left",
149         va="top",
150         fontsize=11.5,
151         fontfamily="SimSun",
152         color="#000000",
153     )
154     legend = fig.legend(
155         loc="upper center",
156         bbox_to_anchor=(0.56, 0.835),
157         ncol=3,
158         frameon=False,
159         prop={"family": "SimHei", "size": 8.0},
160         columnspacing=1.45,
161         handlelength=2.35,
162         handletextpad=0.55,
163         borderaxespad=0.0,
164     )
165     for text in legend.get_texts():
166         text.set_color("#000000")
167
168     fixed_date = datetime(2026, 1, 1, tzinfo=timezone.utc)
169     fig.savefig(FIGURES / f"{FIG_ID}.svg", metadata={"Date": "2026-01-01T00:00:00"})
170     fig.savefig(
171         FIGURES / f"{FIG_ID}.pdf",
172         dpi=600,
173         metadata={"CreationDate": fixed_date, "ModDate": fixed_date},
174     )
175     fig.savefig(FIGURES / f"{FIG_ID}.png", dpi=600)
176     fig.savefig(PREVIEWES / f"{FIG_ID}-color.png", dpi=600)
177     plt.close(fig)
178
179     formal_color = Image.open(FIGURES / f"{FIG_ID}.png").convert("RGB")
180     ImageOps.grayscale(formal_color).save(
181         PREVIEWES / f"{FIG_ID}-grayscale.png", dpi=(600, 600)
182     )
183     print(
184         {
185             "figure_id": FIG_ID,
186             "source_sha256": sha256(SOURCE),
187             "snapshot_sha256": sha256(SNAPSHOT),
188             "png_sha256": sha256(FIGURES / f"{FIG_ID}.png"),
189             "svg_sha256": sha256(FIGURES / f"{FIG_ID}.svg"),
190             "pdf_sha256": sha256(FIGURES / f"{FIG_ID}.pdf"),
191             "right_bottom_note": False,
192         }
193     )
194
195
196 if __name__ == "__main__":
197     main()
198

```

D.13 程序 13: render_q2_profiles_3d.py

原文件: 06-figure/scripts/render_q2_profiles_3d.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```
1 from __future__ import annotations
2
3 import csv
4 import hashlib
5 from datetime import datetime, timezone
6 from pathlib import Path
7
8 import matplotlib as mpl
9 import matplotlib.pyplot as plt
10 import numpy as np
11 from PIL import Image, ImageOps
12
13
14 ROOT = Path(__file__).resolve().parents[2]
15 FIGURE_DIR = ROOT / "06-figure"
16 SNAPSHOT = FIGURE_DIR / "data-snapshots" / "FIG-Q2-C-PROFILES.csv"
17 FIGURES = FIGURE_DIR / "figures"
18 PREVIEWS = FIGURE_DIR / "previews"
19 FIG_ID = "FIG-Q2-C-PROFILES"
20
21
22 def sha256(path: Path) -> str:
23     return hashlib.sha256(path.read_bytes()).hexdigest()
24
25
26 def load_grid() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
27     rows: list[tuple[float, float, float]] = []
28     with SNAPSHOT.open("r", encoding="utf-8-sig", newline="") as handle:
29         for row in csv.DictReader(handle):
30             rows.append(
31                 (
32                     float(row["time_s"]) / 3600.0,
33                     float(row["radius_cm"]),
34                     float(row["C_kg_per_kg"]),
35                 )
36             )
37     times = np.array(sorted({row[0] for row in rows}), dtype=float)
38     radii = np.array(sorted({row[1] for row in rows}), dtype=float)
39     z = np.full((len(radii), len(times)), np.nan, dtype=float)
40     time_index = {value: i for i, value in enumerate(times)}
41     radius_index = {value: i for i, value in enumerate(radii)}
42     for time_h, radius_cm, value in rows:
43         if not np.isnan(z[radius_index[radius_cm], time_index[time_h]]):
44             raise ValueError("duplicate time-radius sample")
45         z[radius_index[radius_cm], time_index[time_h]] = value
46     if np.isnan(z).any():
47         raise ValueError("time-radius samples do not form a complete rectangular grid")
48     return times, radii, z
49
50
51 def main() -> None:
52     FIGURES.mkdir(parents=True, exist_ok=True)
53     PREVIEWS.mkdir(parents=True, exist_ok=True)
54     if not SNAPSHOT.exists():
55         raise FileNotFoundError(SNAPSHOT)
56
57     mpl.rcParams.update(
58         {
59             "font.family": "SimSun",
60             "font.sans-serif": ["SimSun", "STSong", "FangSong", "Microsoft YaHei", "Arial"],
61             "font.serif": ["SimSun", "STSong", "FangSong", "Microsoft YaHei", "Arial"],
62             "font.size": 8.5,
63             "text.color": "#000000",
64             "axes.labelcolor": "#000000",
65             "xtick.color": "#000000",
66             "ytick.color": "#000000",
67             "axes.unicode_minus": False,
68             "axes.facecolor": "#FFFFFF",
69             "figure.facecolor": "#FFFFFF",
70             "savefig.facecolor": "#FFFFFF",
```

```

71         "pdf.fonttype": 42,
72         "ps.fonttype": 42,
73         "svg.fonttype": "none",
74         "svg.hashsalt": FIG_ID,
75     }
76 )
77
78 times_h, radii_cm, z = load_grid()
79 x, y = np.meshgrid(times_h, radii_cm)
80 cmap = mpl.colormaps["viridis"]
81 norm = mpl.colors.Normalize(vmin=float(z.min()), vmax=float(z.max()))
82
83 fig = plt.figure(figsize=(6.9, 5.0), dpi=600)
84 ax = fig.add_subplot(111, projection="3d")
85 ax.set_proj_type("ortho")
86 surface = ax.plot_surface(
87     x,
88     y,
89     z,
90     cmap=cmap,
91     norm=norm,
92     linewidth=0.28,
93     edgecolor=(0.25, 0.25, 0.25, 0.45),
94     antialiased=True,
95     alpha=0.90,
96     rcount=len(radii_cm),
97     ccount=len(times_h),
98 )
99 # Exact frozen samples are overlaid so that the surface is auditable.
100 ax.scatter(
101     x.ravel(),
102     y.ravel(),
103     z.ravel(),
104     c=z.ravel(),
105     cmap=cmap,
106     norm=norm,
107     s=11,
108     depthshade=False,
109     edgecolors="#000000",
110     linewidths=0.28,
111 )
112
113 ax.set_xlim(float(times_h.min()), float(times_h.max()))
114 ax.set_ylim(float(radii_cm.min()), float(radii_cm.max()))
115 # Use four regular z ticks and a small fixed margin. The previous five
116 # ticks were visually crowded by the chosen 3-D projection.
117 z_ticks = [1.0, 1.5, 2.0, 2.5]
118 ax.set_zlim(0.9, 2.7)
119 ax.set_xticks(times_h)
120 ax.set_yticks(radii_cm)
121 ax.set_zticks(z_ticks)
122 ax.set_xlabel(" 时间 t (h) ", labelpad=8, fontsize=9.5)
123 ax.set_ylabel(" 半径 r (cm) ", labelpad=8, fontsize=9.5)
124 ax.set_zlabel(" 干基含水率 C (kg/kg) ", labelpad=17, fontsize=9.5)
125 ax.tick_params(axis="both", which="major", labelsize=8.0, pad=1.5, colors="#000000")
126 ax.tick_params(axis="z", which="major", labelsize=8.0, pad=7.0, colors="#000000")
127 for pane in (ax.xaxis.pane, ax.yaxis.pane, ax.zaxis.pane):
128     pane.set_facecolor((1.0, 1.0, 1.0, 1.0))
129     pane.set_edgecolor((0.0, 0.0, 0.0, 0.18))
130 for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
131     axis._axinfo["grid"]["color"] = (0.0, 0.0, 0.0, 0.12)
132 ax.view_init(elev=30, azim=-55)
133 ax.set_box_aspect((1.55, 1.0, 1.05))
134
135 fig.suptitle(
136     "M3 不同时间一半径位置的含水率三维演化",
137     x=0.46,
138     y=0.965,
139     fontsize=11.5,
140     fontfamily="SimSun",
141 )
142 ax.text2D(0.02, 0.93, " 点: 冻结采样值", transform=ax.transAxes, fontsize=8.0, color="#000000")
143 colorbar = fig.colorbar(surface, ax=ax, shrink=0.68, pad=0.12, aspect=19)
144 colorbar.set_ticks(z_ticks)
145 colorbar.set_label("C (kg/kg) ", fontsize=8.8, labelpad=6, color="#000000")
146 colorbar.ax.tick_params(labelsize=7.7, colors="#000000", length=2.5, width=0.55)
147 colorbar.outline.set_edgecolor("#000000")
148 colorbar.outline.set_linewidth(0.45)
149 fig.subplots_adjust(left=0.0, right=0.92, bottom=0.02, top=0.92)

```

```

150
151     fixed_date = datetime(2026, 1, 1, tzinfo=timezone.utc)
152     svg_path = FIGURES / f"{FIG_ID}.svg"
153     pdf_path = FIGURES / f"{FIG_ID}.pdf"
154     png_path = FIGURES / f"{FIG_ID}.png"
155     fig.savefig(svg_path, metadata={"Date": "2026-01-01T00:00:00"})
156     fig.savefig(pdf_path, dpi=600, metadata={"CreationDate": fixed_date, "ModDate": fixed_date})
157     fig.savefig(png_path, dpi=600)
158     fig.savefig(PREVIEWES / f"{FIG_ID}-color.png", dpi=600)
159     plt.close(fig)
160
161     color = Image.open(png_path).convert("RGB")
162     ImageOps.grayscale(color).save(PREVIEWES / f"{FIG_ID}-grayscale.png", dpi=(600, 600))
163     print(
164         {
165             "figure_id": FIG_ID,
166             "snapshot_sha256": sha256(SNAPSHOT),
167             "png_sha256": sha256(png_path),
168             "svg_sha256": sha256(svg_path),
169             "pdf_sha256": sha256(pdf_path),
170             "rows": int(z.size),
171             "time_range_h": [float(times_h.min()), float(times_h.max())],
172             "radius_range_cm": [float(radaii_cm.min()), float(radaii_cm.max())],
173             "C_range_kg_per_kg": [float(z.min()), float(z.max())],
174             "interpolation": False,
175         }
176     )
177
178
179 if __name__ == "__main__":
180     main()
181

```

D.14 程序 14: revise_q2_figures_20260913.py

原文件: 06-figure/scripts/revise_q2_figures_20260913.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from __future__ import annotations
2
3 import hashlib
4 import json
5 from pathlib import Path
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import pandas as pd
10 from PIL import Image, ImageOps
11
12 ROOT = Path(__file__).resolve().parents[2]
13 FIGDIR = ROOT / "06-figure" / "figures"
14 PREV = ROOT / "06-figure" / "previews"
15 SNAP = ROOT / "06-figure" / "data-snapshots"
16 CONTRACT = ROOT / "06-figure" / "contracts"
17 FIGDIR.mkdir(exist_ok=True)
18 PREV.mkdir(exist_ok=True)
19 SNAP.mkdir(exist_ok=True)
20 CONTRACT.mkdir(exist_ok=True)
21
22 plt.rcParams.update({
23     "font.family": "Microsoft YaHei", "font.size": 9,
24     "axes.unicode_minus": False, "svg.fonttype": "none",
25 })
26 BLUE, ORANGE, GREEN, BLACK, GRID = "#0072B2", "#E69F00", "#009E73", "#111111", "#D9D9D9"
27
28
29 def sha(path: Path) -> str:
30     return hashlib.sha256(path.read_bytes()).hexdigest()
31
32
33 def save_figure(fid: str, fig) -> None:
34     fig.set_size_inches(9.5, 4.6)
35     fig.savefig(FIGDIR / f"{fid}.svg", bbox_inches="tight")

```

```

36 fig.savefig(FIGDIR / f"{fid}.pdf", bbox_inches="tight")
37 fig.savefig(FIGDIR / f"{fid}.png", dpi=600, bbox_inches="tight")
38 fig.savefig(PREV / f"{fid}-color.png", dpi=150, bbox_inches="tight")
39 plt.close(fig)
40 ImageOps.grayscale(Image.open(PREV / f"{fid}-color.png").convert("RGB")).save(PREV / f"{fid}-grayscale.png")
41
42
43 def write_snapshot(fid: str, df: pd.DataFrame, source: Path) -> None:
44     path = SNAP / f"{fid}.csv"
45     df.to_csv(path, index=False, encoding="utf-8-sig")
46     meta = {
47         "figure_id": fid,
48         "snapshot": path.relative_to(ROOT).as_posix(),
49         "snapshot_sha256": sha(path),
50         "sources": [{"path": source.relative_to(ROOT).as_posix(), "sha256": sha(source)}],
51         "rows": len(df),
52         "columns": list(df.columns),
53     }
54     (SNAP / f"{fid}.meta.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
55     contract_path = CONTRACT / f"{fid}.json"
56     contract = json.loads(contract_path.read_text(encoding="utf-8"))
57     contract["snapshot"] = path.relative_to(ROOT).as_posix()
58     contract["snapshot_sha256"] = sha(path)
59     contract["source_files"] = [{"path": source.relative_to(ROOT).as_posix(), "sha256": sha(source)}]
60     contract["visual_revision"] = "user-revision-20260913"
61     contract_path.write_text(json.dumps(contract, ensure_ascii=False, indent=2), encoding="utf-8")
62
63
64 # Figure 4: one time trajectory per model, shown in separate comparable panels.
65 fid = "FIG-Q2-MODEL-ABLATION"
66 source4 = ROOT / "04-compute" / "revision-grid-conv-20260913" / "results" / "q2-model-trajectories" /
67     ↪ "q2-model-trajectories.csv"
68 df4 = pd.read_csv(source4)
69 write_snapshot(fid, df4, source4)
70 fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.5), sharex=True, sharey=True)
71 styles = [(BLUE, "-", "o"), (ORANGE, "--", "s"), (GREEN, "-.", "~")]
72 model_order = list(df4["model"].drop_duplicates())
73 for ax, model, (color, ls, marker) in zip(axes, model_order, styles):
74     group = df4[df4["model"] == model].sort_values("time_s")
75     ax.plot(group.time_s / 3600, group.max_C, color=color, linestyle=ls, marker=marker,
76           markersize=4, linewidth=1.5)
77     ax.set_title(model)
78     ax.set_xlabel("时间 (h)")
79     ax.grid(axis="y", color=GRID, linewidth=0.6)
80     ax.spines[["top", "right"]].set_visible(False)
81 axes[0].set_ylabel("全域最大干基含水率 (kg/kg)")
82 fig.suptitle("三个模型在不同时间的最大含水率轨迹", y=1.02)
83 fig.text(0.5, -0.01, "三幅子图使用相同坐标范围; 仅比较模型输出差异, 不代表现实准确率排序",
84       ha="center", va="top", color="#444444")
85 fig.tight_layout()
86 save_figure(fid, fig)
87
88 # Figure 5: six radial-grid groups at the same formal dt=5 s.
89 fid = "FIG-Q2-GRID-CONV"
90 source5 = ROOT / "04-compute" / "revision-grid-conv-20260913" / "results" / "q2-space-dt5" /
91     ↪ "field-grid-convergence.csv"
92 df5 = pd.read_csv(source5)
93 write_snapshot(fid, df5, source5)
94 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 3.8), gridspec_kw={"width_ratios": [1.25, 1]})
95 ax1.plot(df5.nr, df5.value, "o-", color=BLUE, linewidth=1.5, markersize=5)
96 ax1.set_xlabel("径向网格数 $n_r$")
97 ax1.set_ylabel("3 h 最大干基含水率 (kg/kg)")
98 ax1.set_title("六组网格的终值")
99 ax1.grid(axis="y", color=GRID, linewidth=0.6)
100 ax1.ticklabel_format(axis="y", style="plain", useOffset=False)
101 ax1.spines[["top", "right"]].set_visible(False)
102 ax1.set_xticks(df5.nr)
103 ax1.tick_params(axis="x", rotation=25)
104 for x, y in zip(df5.nr, df5.value):
105     ax1.annotate(f"{y:.6f}", (x, y), xytext=(0, 8), textcoords="offset points", ha="center", fontsize=7)
106 changes = df5.iloc[1:].copy()
107 ax2.plot(changes.nr, changes.adjacent_change, "o-", color=ORANGE, linewidth=1.5, markersize=5)
108 ax2.axhline(5e-5, color=BLACK, linestyle="--", linewidth=1.0, label="验收线 5e-5")
109 ax2.set_xlabel("径向网格数 $n_r$")
110 ax2.set_ylabel("相邻网格终值变化")
111 ax2.set_title("相邻变化逐步减小")
112 ax2.grid(axis="y", color=GRID, linewidth=0.6)
113 ax2.spines[["top", "right"]].set_visible(False)
114 ax2.set_xticks(changes.nr)

```

```

113 ax2.ticklabel_format(axis="y", style="sci", scilimits=(0, 0))
114 ax2.legend(frameon=False, fontsize=8)
115 ax2.annotate(f" 最细变化 {changes.adjacent_change.iloc[-1]:.3e}",
116             (changes.nr.iloc[-1], changes.adjacent_change.iloc[-1]),
117             xytext=(-70, 18), textcoords="offset points", arrowprops={"arrowstyle": "->"}, fontsize=8)
118 fig.suptitle("M3 空间网格收敛: 固定时间步长 dt=5 s", y=1.02)
119 fig.tight_layout()
120 save_figure(fid, fig)
121
122 # Appendix S2/S3: expanded five-level spatial convergence at fixed dt=60 s.
123 extra_space = ROOT / "04-compute" / "revision-grid-conv-20260913" / "results" / "space-extra-dt60" /
    ↳ "threshold-grid-extra.csv"
124 base_space = pd.read_csv(ROOT / "04-compute" / "results" / "threshold-grid-convergence.csv")
125 extra = pd.read_csv(extra_space)
126 space_all = pd.concat([base_space, extra], ignore_index=True).drop_duplicates(["scope",
    ↳ "nr"]).sort_values(["scope", "nr"])
127 for fid, scope in [("FIG-Q3-SPACE-CONV", "Q3"), ("FIG-Q4-SPACE-CONV", "Q4")]:
128     df = space_all[space_all.scope == scope].copy()
129     write_snapshot(fid, df, extra_space)
130     fig, ax = plt.subplots(figsize=(8.5, 4.2))
131     ax.plot(df.nr, df.interpolated_event_time_s / 3600, "o-", color=BLUE, linewidth=1.5, markersize=5)
132     ax.set_xlabel(" 径向网格数 $n_r$")
133     ax.set_ylabel(" 插值事件时刻 (h)")
134     ax.set_title(f"{scope} 空间网格收敛 (五组网格, dt=60 s)")
135     ax.grid(axis="y", color=GRID, linewidth=0.6)
136     ax.spines[["top", "right"]].set_visible(False)
137     ax.set_xticks(df.nr)
138     for x, y in zip(df.nr, df.interpolated_event_time_s / 3600):
139         ax.annotate(f"{y:.4f}", (x, y), xytext=(0, 8), textcoords="offset points", ha="center", fontsize=7)
140     ax.text(0.01, 0.02, " 仅表明空间离散稳定性, 不替代时间步验证", transform=ax.transAxes, color="#444444")
141     fig.tight_layout()
142     save_figure(fid, fig)
143
144 print("REVISED-Q2-FIGURES-PASS")
145

```

D.15 程序 15: update_figure_plan_20260913.py

原文件: 06-figure/scripts/update_figure_plan_20260913.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 import hashlib
2 import json
3 from pathlib import Path
4
5 ROOT = Path(__file__).resolve().parents[2]
6 manifest_path = ROOT / "06-figure" / "figure-manifest.json"
7 manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
8 manifest["status"] = "REVIEW_REQUIRED"
9 removed = {"FIG-Q3-TIME-CONV", "FIG-Q3-COMBINED-BOUNDARY", "FIG-Q4-COMBINED-BOUNDARY"}
10 manifest["main_figures"] = [fid for fid in manifest["main_figures"] if fid not in removed]
11 manifest["n_figures"] = len(manifest["main_figures"]) + len(manifest["appendix_figures"])
12 manifest["items"] = [item for item in manifest["items"] if item["id"] not in removed]
13
14 for item in manifest["items"]:
15     # Refresh hashes for every active figure so the current manifest is the
16     # single source of truth after any approved redraw; apply the new data
17     # claims/revision labels only to the figures changed in this revision.
18     if item.get("id", "").startswith("FIG"):
19         png = ROOT / item["png"]
20         svg = ROOT / item["svg"]
21         pdf = ROOT / item.get("pdf", "06-figure/figures/unused.pdf")
22         item["png_sha256"] = hashlib.sha256(png.read_bytes()).hexdigest()
23         item["svg_sha256"] = hashlib.sha256(svg.read_bytes()).hexdigest()
24         if pdf.exists():
25             item["pdf_sha256"] = hashlib.sha256(pdf.read_bytes()).hexdigest()
26     if item["id"] in {"FIG-Q2-MODEL-ABLATION", "FIG-Q2-GRID-CONV", "FIG-Q3-SPACE-CONV", "FIG-Q4-SPACE-CONV"}:
27         item["visual_revision"] = "user-revision-20260913"
28         item["data_claim_status"] = "unchanged" if item["id"] == "FIG-Q2-MODEL-ABLATION" else
    ↳ "expanded-grid-evidence-at-fixed-dt5s" if item["id"] == "FIG-Q2-GRID-CONV" else
    ↳ "expanded-grid-evidence-at-fixed-dt60s"
29
30 manifest["revision_note"] = "用户确认: 移除原图8、10、13; 图4改为三模型同尺度分面, 图5扩展为六组径向网格 (dt=5 s),
    ↳ 图15/16扩展为五组径向网格 (dt=60 s)。"

```

```

31 manifest_path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
32 print(f"FIGURE_PLAN_UPDATED {manifest['n_figures']}")
33

```

D.16 程序 16: verify_current_pipeline.py

原文件: 06-figure/scripts/verify_current_pipeline.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from __future__ import annotations
2
3 import csv
4 import hashlib
5 import json
6 from pathlib import Path
7
8 ROOT = Path(__file__).resolve().parents[2]
9 OUT = ROOT / "06-figure"
10
11
12 def sha(path: Path) -> str:
13     return hashlib.sha256(path.read_bytes()).hexdigest()
14
15
16 def read_csv(path: Path):
17     with path.open("r", encoding="utf-8-sig", newline="") as fh:
18         return list(csv.DictReader(fh))
19
20
21 manifest = json.loads((OUT / "figure-manifest.json").read_text(encoding="utf-8"))
22 failures = []
23 active = set(manifest.get("main_figures", []) + manifest.get("appendix_figures", []))
24 forbidden_ids = {"FIG-Q3-TIME-CONV", "FIG-Q3-COMBINED-BOUNDARY", "FIG-Q4-COMBINED-BOUNDARY"}
25 if active & forbidden_ids:
26     failures.append(f"legacy figure IDs remain active: {sorted(active & forbidden_ids)}")
27 if manifest.get("n_figures") != len(active) or len(active) != 13:
28     failures.append("active manifest count is not 13")
29
30 new_renderer = OUT / "scripts" / "revise_q2_figures_20260913.py"
31 if not new_renderer.exists():
32     failures.append("new renderer missing: revise_q2_figures_20260913.py")
33
34 checks = {}
35 for fid in active:
36     item = next((x for x in manifest.get("items", []) if x.get("id") == fid), None)
37     if not item:
38         failures.append(f"missing manifest item: {fid}")
39         continue
40     for key in ("png", "svg"):
41         path = ROOT / item[key]
42         if not path.exists():
43             failures.append(f"missing {key}: {fid}")
44         elif item.get(f"{key}_sha256") and sha(path) != item[f"{key}_sha256"]:
45             failures.append(f"hash mismatch: {fid} {key}")
46
47 f4 = read_csv(OUT / "data-snapshots" / "FIG-Q2-MODEL-ABLATION.csv")
48 counts = {}
49 for row in f4:
50     counts[row["model"]] = counts.get(row["model"], 0) + 1
51 checks["FIG-Q2-MODEL-ABLATION"] = {"rows": len(f4), "rows_per_model": counts}
52 if len(f4) != 57 or set(counts.values()) != {19}:
53     failures.append("Figure 4 is not 3 models x 19 time points")
54
55 f5 = read_csv(OUT / "data-snapshots" / "FIG-Q2-GRID-CONV.csv")
56 nr5 = sorted({int(float(row["nr"])) for row in f5})
57 checks["FIG-Q2-GRID-CONV"] = {"radial_grids": nr5, "count": len(nr5)}
58 if nr5 != [42, 62, 93, 140, 175, 210]:
59     failures.append(f"Figure 5 radial grids unexpected: {nr5}")
60
61 for fid in ("FIG-Q3-SPACE-CONV", "FIG-Q4-SPACE-CONV"):
62     rows = read_csv(OUT / "data-snapshots" / f"{fid}.csv")
63     nrs = sorted({int(float(row["nr"])) for row in rows})
64     checks[fid] = {"radial_grids": nrs, "count": len(nrs)}

```

```

65     if nrs != [63, 93, 140, 175, 210]:
66         failures.append(f"{fid} radial grids unexpected: {nrs}")
67
68 report = {
69     "status": "PASS" if not failures else "FAIL",
70     "pipeline": "CURRENT_20260913",
71     "active_figure_count": len(active),
72     "new_renderer": str(new_renderer.relative_to(ROOT)).replace("\\", "/"),
73     "checks": checks,
74     "failures": failures,
75 }
76 (OUT / "current-pipeline-verification.json").write_text(
77     json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
78 )
79 print(json.dumps(report, ensure_ascii=False))
80 raise SystemExit(bool(failures))
81

```

D.17 程序 17: make_official_contact_sheet.py

原文件: 06-figure/scripts/make_official_contact_sheet.py

本文件在代码实现、调试或图表制作过程中得到 Codex 辅助; 参数、模型路线、结果解释及最终核验由参赛队负责。

```

1 from pathlib import Path
2 from PIL import Image, ImageDraw, ImageFont
3
4 ROOT = Path(__file__).resolve().parents[2]
5 FIGDIR = ROOT / "06-figure" / "figures"
6 OUT = ROOT / "06-figure" / "official-figures-contact-sheet-20260913.png"
7
8 ids = [
9     "FIG-Q1-C-FIELD", "FIG-Q1-END-EFFECT", "FIG-Q2-C-PROFILES", "FIG-Q2-MODEL-ABLATION",
10    "FIG-Q2-GRID-CONV", "FIG-Q3-THRESHOLD-TRAJECTORY", "FIG-Q3-BRACKET-ZOOM",
11    "FIG-Q3-SENS-ONEFACTOR", "FIG-Q4-RADIUS-TIME", "FIG-Q4-THRESHOLD-TRAJECTORY",
12    "FIG-Q1-GRID-CONV", "FIG-Q3-SPACE-CONV", "FIG-Q4-SPACE-CONV",
13 ]
14
15 W, H = 900, 620
16 sheet = Image.new("RGB", (W * 4, H * 4), "white")
17 draw = ImageDraw.Draw(sheet)
18 font = ImageFont.truetype("simhei.ttf", 28)
19 small = ImageFont.truetype("simhei.ttf", 22)
20
21 for i, fid in enumerate(ids):
22     path = FIGDIR / f"{fid}.png"
23     if not path.exists():
24         raise FileNotFoundError(path)
25     with Image.open(path) as src:
26         src = src.convert("RGB")
27         src.thumbnail((W - 40, H - 90), Image.Resampling.LANCZOS)
28         x = (i % 4) * W
29         y = (i // 4) * H
30         draw.rectangle((x, y, x + W - 1, y + H - 1), outline=(190, 190, 190), width=2)
31         draw.text((x + 18, y + 12), f"{i+1:02d} {fid}", fill=(0, 0, 0), font=font)
32         px = x + (W - src.width) // 2
33         py = y + 62 + (H - 72 - src.height) // 2
34         sheet.paste(src, (px, py))
35
36 sheet.save(OUT, dpi=(150, 150))
37 print(OUT)
38

```